

CS100 Recitation 1

GKxx

目录

- 开始
- 编译、运行一个 C 程序
- 初识函数
- `scanf` 和 `printf`
- 算术类型

开始

为什么要学“任何东西”？

为什么要学 C/C++ ？

- 为什么要学微积分/离散数学/XXXX/.....？

为什么要学“任何东西”？

为什么要学 C/C++ ？

- 为什么要学微积分/离散数学/XXXX/.....？

重走一个学科/领域从无到有的发现历程

- 理解学科中的基本思想、基本方法、里程碑、走过的弯路
- 锻炼学习的能力
- 应用、创新、革命

怎样学好这门课

关键：把课件、课本等资料上的示例代码独立地实现一遍

- 作业也会有很多直接来自于课件（包括习题课）的题目
 - 看懂、掌握基本的知识
 - （尝试）独立地实现
 - 对比课件上的写法，有什么区别？哪种更好？
- 仅仅做到能“看懂”是**不够的**，只有独立实现才能内化为自己的知识。
 - 学数学也是如此。

关于学术诚信

我们采取相对宽松的规则。

- 简单题、教学题不查重，鼓励模仿和借鉴**好的写法、好的思路**。
 - 部分题目会设置与 TA 的线下 check 。
- **允许**帮别人 debug 。
- 鼓励使用 AI 工具辅助学习，比如让 ChatGPT/Copilot 帮你解释代码。
 - 注意谨慎求证。
- 但是禁止**直接抄袭**来自他人、网络、AI 生成的代码。

关于 TA

有问题找 TA，任何问题都可以问。我们会在合理的范围内回答、提供帮助。

编译、运行一个 C 程序

编译、运行一个 C 程序

```
#include <stdio.h>

int main(void) {
    printf("Hello, world\n");
    return 0;
}
```

`gcc hello.c -o hello` 使用 `gcc` 编译，生成一个可执行文件。

- Windows 上叫 `hello.exe`，Linux/Mac 上叫 `hello` 。

`./hello` (Linux/Mac), `.\hello` (Windows) 运行这个程序。

- 当然你也可以双击它，但它会“一闪而过”。

环境变量 PATH

当你执行 `gcc hello.c -o hello` 的时候，你唤醒的 `gcc` 是你安装在某处的 `gcc` (Linux/Mac) / `gcc.exe` (Windows)

例如，Windows 上的 `D:\mingw64\bin\gcc.exe`

终端如何认识到你想找的 `gcc` 是 `D:\mingw64\bin\gcc.exe` ？

环境变量 PATH

当你执行 `gcc hello.c -o hello` 的时候，你唤醒的 `gcc` 是你安装在某处的 `gcc` (Linux/Mac) / `gcc.exe` (Windows)

例如，Windows 上的 `D:\mingw64\bin\gcc.exe`

终端如何认识到你想找的 `gcc` 是 `D:\mingw64\bin\gcc.exe` ？

- 将 `D:\mingw64\bin` 添加到环境变量 `PATH` 。

在 VSCode/CLion 的终端中执行以上指令

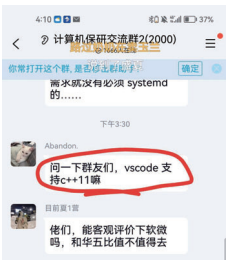
VSCode: `ctrl + `` 开启终端，默认 `working directory` 为当前打开的文件夹。

CLion: 最下面 "Terminal"/"终端"，或 `Alt + F12`

可以先 `cd` 到方便的位置（如果需要的话）

编译、运行、调试一个 C 程序

VSCode/CLion 本身不具备任何编译、运行、调试的功能！它只是把编译器、可执行文件和 GDB 的结果打在公屏上，然后给你提供一些方便操作的按钮。



初识函数

定义一个函数

语法: `ReturnType FunctionName(Parameters) { FunctionBody }`

- 数学上，函数是 $f : S \mapsto T$
- 一个函数的参数的类型对应 S ，返回值类型对应 T 。

例：下取整函数 $f(x) = \lfloor x \rfloor$ 的参数和返回值类型分别是什么？

定义一个函数

语法: `ReturnType FunctionName(Parameters) { FunctionBody }`

- 数学上，函数是 $f : S \mapsto T$
- 一个函数的参数的类型对应 S ，返回值类型对应 T 。

例：下取整函数 $f(x) = \lfloor x \rfloor$ 的参数和返回值类型分别是什么？

```
int floor(double x) {
    // ...
}
```

定义一个函数

一个函数可以接受多个参数 ($f : S_1 \times S_2 \times \dots \times S_n \mapsto T$)，每个参数的类型都需要写出来。

- 哪怕它和前一个参数类型相同，也不可以省去。 `(int x, int y)` 不可以省略为 `(int x, y)`

```
ReturnType myFunction(Type1 a1, Type2 a2, Type3 a3) {
    // ...
}
```

定义一个函数

一个函数可以没有返回值，这样的函数返回值类型为 `void`。

和数学函数不同，C 函数不仅仅是“接受一些参数、返回一个值”，它实际上就是将一小段具有特定功能的代码单独拎出来，并且给它起个名字。

定义一个函数

一个函数可以不接受参数 ($f : \emptyset \mapsto T$)，写法是在参数列表里放一个 `void`

```
ReturnType do_some_work(void) {
    // ...
}
```

和 C++ 不同，参数列表空着并不代表不接受参数，而是表示 "accepting unknown parameters"。这项规则自 C23 起才被删除。

- 这是 C 的一个历史遗留问题。上古时期的 C 并不注重参数类型检查，这可能更接近汇编的风格，但也更容易出错。

另一个历史遗留问题

在 C99 之前，一个函数如果不写返回值类型，则返回值类型默认为 `int`。

事实上在旧的 C 语言中存在着大量的“默认为 `int`”的规则，在很多场合下这种写法极易引发难以预料的问题。

- 不幸的是，为了兼容已经存在的上亿行旧代码，编译器大概率会允许这种写法，并仅仅给出一个 warning。

你可能见过一些人写这样的 `main` 函数：

```
main() {
    // ...
}
```

这不符合现代 C 语言的语法，也是极不推荐的写法。

main 函数的返回值

函数的返回值是传给它的调用者的。

- `main` 的“调用者”是谁？——程序的调用者/“宿主环境” (hosted environment)

`main` 函数返回 `0` ——程序正常退出

`main` 函数返回其它值——代表某种错误

- 一些程序会用不同的返回值表达不同的含义。
- 也有的程序只是在“任何错误”发生时都返回 `1`。

更糟糕的历史遗留问题

如果你调用了 一个未经声明的函数

```
#include <stdio.h>

int main(void) {
    int x = my_mysterious_function();
    printf("%d\n", x);
}
```

C++ 编译器会给出一个十分正常的报错: "... was not declared in this scope"

```
a.cpp: In function 'int main()':
a.cpp:4:11: error: 'my_mysterious_function' was not declared in this scope
4 |     int x = my_mysterious_function();
  |           ^~~~~~
```

scanf 和 printf

虽然文档有点复杂，但也要试着在这里检索你需要的信息。

[scanf](#)

[printf](#)

更糟糕的历史遗留问题

如果你调用了 一个未经声明的函数

```
#include <stdio.h>

int main(void) {
    int x = my_mysterious_function();
    printf("%d\n", x);
}
```

而 C 编译器会允许，并且给出一个令人困惑的 warning: "implicit declaration"

```
a.c: In function 'main':
a.c:4:11: warning: implicit declaration of function 'my_mysterious_function'
[-Wimplicit-function-declaration]
4 |     int x = my_mysterious_function();
  |           ^~~~~~
```

当然，它是无法运行的，链接器会抱怨“找不到名为 `my_mysterious_function` 的函数”。

"A+B" problem

```
#include <stdio.h>

int main(void) {
    int a, b;
    scanf("%d%d", &a, &b);
    printf("%d\n", a + b);
    return 0;
}
```

更糟糕的历史遗留问题

如果你调用了 一个未经声明的函数

```
#include <stdio.h>

int main(void) {
    int x = my_mysterious_function();
    printf("%d\n", x);
}
```

C 标准认为你“隐式地声明” (implicitly declare) 了这个函数，于是压力全都给到链接器。

这一规则 C99 起被删除，但为了向后兼容 (backward compatibility)，编译器很可能仍然支持，只是给一个 warning。

A+B Problem

约定输入格式：两行，每行一个整数，分别表示 `a` 和 `b`。

- `scanf("%d\n%d\n", &a, &b)` 会发生什么？试一试。

main 函数的返回值

函数的返回值是传给它的调用者的。

- `main` 的“调用者”是谁？

A+B Problem

约定输入格式：两行，每行一个整数，分别表示 `a` 和 `b`。

- `scanf("%d\n%d\n", &a, &b)` 会卡住，直到你输入了一个非空白字符为止。
- 看看标准：

whitespace characters: any single whitespace character in the format string consumes all available consecutive whitespace characters from the input (determined as if by calling `isspace` in a loop). Note that there is no difference between `"\n"`, `" "`, `"\t\t"`, or other whitespace in the format string.
- whitespace character: “空白字符”，包括空格、换行、回车、制表等。

A+B Problem

约定输入格式：两行，每行一个整数，分别表示 `a` 和 `b`。

- `scanf("%d%d", &a, &b)` 可以吗？试一试。

31/41

A+B Problem

约定输入格式：两行，每行一个整数，分别表示 `a` 和 `b`。

- `scanf("%d%d", &a, &b)` 可以。
- 中间的换行符不写也可以？看看标准：

```
"%d" matches a decimal integer.
```

The format of the number is the same as expected by `strtol` with the value 10 for the base argument.
- `strtol` 是什么鬼？[点进去看看](#)：

Interprets an integer value in a byte string pointed to by str.

Discards any whitespace characters (as identified by calling `isspace`) until the first non-whitespace character is found, then...

32/41

printf

`double` 需要用 `%lf` 输出。如果我用了 `%d` 输出，会怎样？

```
#include <stdio.h>

int main(void) {
    double pi = 3.14;
    printf("%d\n", pi);
    return 0;
}
```

它和先将 `pi` 转换为 `int`（小数部分截去）再输出的效果一样吗？

```
double pi = 3.14;
int pi_int = pi; // pi_int 值为 3
printf("%d\n", pi_int);
```

33/41

printf

在 `scanf` 和 `printf` 中，conversion specifier 与对应的变量类型不匹配是 **undefined behavior**：你不能对程序的行为做任何假定。

以下说法统统是错的：

- 它会进行某种类型转换再输出
- 它会输出一个值，只是我不知道它是多少罢了

程序完全有可能在这里直接崩溃。此外，编译器可以假定你的程序不含 undefined behavior，所以它如果发现这里有 undefined behavior，就可以直接把这行代码删掉，这完全符合语言标准。

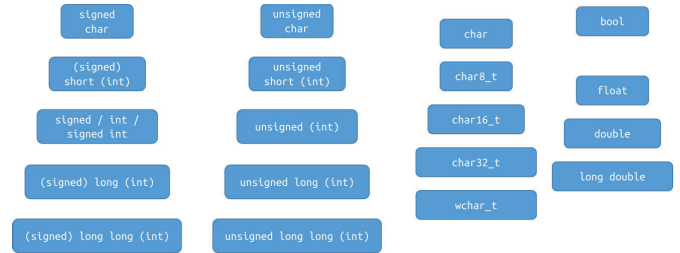
34/41

算术类型

正课 slides 已经足够完整，这里仅作为回顾。

算术类型：整数，字符，布尔，浮点数

有时候字符和布尔也认为是整数。



36/41

整数类型的大小

type	width (at least)	width (usually)
<code>short</code>	16 bits	16 bits
<code>int</code>	16 bits	32 bits
<code>long</code>	32 bits	32 or 64 bits
<code>long long</code>	64 bits	64 bits

- `sizeof(signed T) == sizeof(unsigned T)`
- `1 == sizeof(char) <= sizeof(short) <= sizeof(int) <= sizeof(long) <= sizeof(long long)`

37/41

位 (bit) 和字节 (byte)

一个 32 位整数长这样：



通常一个字节是 8 位。 `sizeof(T)` 可以获得 `T` 所占的**字节**数量。

```
printf("%zu\n", sizeof(double));
```

- 据说 MinGW 对 `%zu` 的支持有问题，不行就把结果转成 `int` 再用 `%d`

```
printf("%d\n", (int)sizeof(double));
```

38/41

浮点数不是万能的

定义一个函数，接受两个 64 位整数，返回它们的平均值。你会使用什么返回值类型？

浮点数不是万能的

定义一个函数，接受两个 64 位整数，返回它们的平均值。你会使用什么返回值类型？

看起来唯一的选择是 `double`

```
double midpoint(long long a, long long b) {
    return (a + b) / 2.0;
}
```

它对吗？试试看 `9007199254740993 9007199254740993`

35/41

40/41

浮点数误差是难免的

判断一个浮点数是否等于零，必须使用 $|x| < \epsilon$ ，其中 ϵ 可以是 10^{-8} 这样的小数。
判断两个浮点数是否相等，必须使用 $|a - b| < \epsilon$ 。

CS100 Recitation 2

GKxx

目录

- 字面值
- 运算符
- 控制流

字面值 (literals)

字面值 (literals)

像 42 这样把值写在脸上的就是字面值。
C 语言的每一个表达式都有一个明确的、编译期确定的类型。所以，42 是什么类型？
3.14 又是什么类型？

字面值 (literals)

像 42 这样把值写在脸上的就是字面值。
C 语言的每一个表达式都有一个明确的、编译期确定的类型。所以，42 是什么类型？
3.14 又是什么类型？

- 42 是 int，3.14 是 double，尽管它们也许可以用其它类型表示。

如何写出一个其它类型的字面值？

字面值 (literals)

像 42 这样把值写在脸上的就是字面值。

- 整型字面值 (integer literals): 42, 100L, 011, 405ul
 - 不写后缀，默认是 int，如果 int 不够大就是 long，还不够大就是 long long。还不够大的话：
 - 如果编译器支持 __int128 并且它够大，那就是 __int128
 - 否则报错 (ill-formed)。
 - 不存在负字面值：-42 是将一元负号 - 作用在字面值 42 上形成的表达式。
 - 后缀有 u (unsigned), l (long), ll (long long)。大小写不敏感，但是不可以是 ll 或 Ll。
 - 后缀可以任意组合。

字面值 (literals)

像 42 这样把值写在脸上的就是字面值。

- 整型字面值 (integer literal): 42, 100L, 011, 405ul
 - 还可以有十六进制字面值: 0xBAADF00D
 - 以及八进制字面值: 052
 - 以及 C23 的二进制字面值: 0b101010
 - 实际上编译器早就支持了，隔壁 C++14 就有二进制字面值了。
 - 这里所有的字母的大小写都随意。

字面值 (literals)

浮点数字面值: 3.14, 3.14f, 3.14l, 1e8, 3e-8

- 不写后缀，默认是 double。f 是 float，l 是 long double，大小写不敏感。
- 1e8 表示 10^8 ，但它是 double 而非整数。

字符字面值: 'a'

- 类型是 int？—— C++ 里它就是 char 了。

字符？一个小整数罢了

```
char c = 'a';
```

c 所存储的内容就是整数 97，即 'a' 的 ASCII 码，而非任何神秘的图像。

不存在“字符和它对应的 ASCII 码之间的转换”！它们本就是同一个东西

假如字符 c 是一个数字字符，如何获得它所表示的数值？

字符？一个小整数罢了

```
char c = 'a';
```

c 所存储的内容就是整数 97，即 'a' 的 ASCII 码，而非任何神秘的图像。

不存在“字符和它对应的 ASCII 码之间的转换”！它们本就是同一个东西

假如字符 c 是一个数字字符，c - '0' 就是它所表示的数值。

练习：假如你只能使用 getchar() 读取一个字符，如何读入一个非负整数？

读入整数

暂时假设输入不会出错。

```
int read(void) {
    int x = 0;
    char c = getchar();
    while (isspace(c)) // 跳过空白
        c = getchar();
    while (isdigit(c)) {
        x = x * 10 + c - '0';
        c = getchar();
    }
    return x;
}
```

- 使用声明在标准库头文件 <ctype.h> 中的函数
- isupper(c) 判断 c 是不是大写字母
- islower(c) 判断 c 是不是小写字母
- isdigit(c) 判断 c 是不是数字字符
- isspace(c) 判断 c 是不是空白字符
-

运算符

优先级，结合性，求值顺序

优先级和结合性表

- 优先级：f() + g() * h() 被解析为 f() + (g() * h()) 而非 (f() + g()) * h()
- 结合性：f() - g() + h() 被解析为 (f() - g()) + h() 而非 f() - (g() + h())

但是 f(), g(), h() 三者的调用顺序是 unspecified 的！

类似的还有：func(f(), g(), h()) 这里的 f(), g(), h() 三个函数的调用顺序是 unspecified 的。

我的亲身经历：add_edge(read(), read(), read());

优先级，结合性，求值顺序

如果两个表达式 A 和 B 的求值顺序是 unspecified 的，而它们

- 都修改了一个变量的值，或者
- 一个修改了某个变量的值，另一个读取了那个变量的值，

那么这就是 undefined behavior。

例：i = i++ + 2 的结果是什么？

优先级，结合性，求值顺序

如果两个表达式 A 和 B 的求值顺序是 unspecified 的，而它们

- 都修改了一个变量的值，或者
- 一个修改了某个变量的值，另一个读取了那个变量的值，

那么这就是 undefined behavior。

例：i = i++ + 2 的结果是什么？

- 一旦你开始分析它是 +1 还是 +2，你就掉进了 undefined behavior 的陷阱里。
- ++ 对 i 进行了修改，赋值也对 i 进行了修改，两个修改的顺序 unspecified，所以就是 undefined behavior。

求值顺序确定的运算符

常见的运算符中，其运算对象的求值顺序确定的只有四个：&&, ||, ?:, ,

- && 和 ||：短路求值，先求左边，非必要不求右边。
- cond ? t : f：先求 cond，根据它的真假性选择求 t 还是求 f。
- ,：一种存在感极低的运算符。

位运算符

~, &, ^, |, <<, >>, 以及复合赋值运算符 &=, |=, ^=, <<=, >>=

Operator	Operator name	Example	Result
~	bitwise NOT	~a	the bitwise NOT of a
&	bitwise AND	a & b	the bitwise AND of a and b
	bitwise OR	a b	the bitwise OR of a and b
^	bitwise XOR	a ^ b	the bitwise XOR of a and b
<<	bitwise left shift	a << i	a left shifted by i
>>	bitwise right shift	a >> i	a right shifted by i

位运算符

- ~a：返回 a 的每个二进制位都取反后的结果。例如 10010110_{two} 求反后等于 1101001_{two}
- a & b 的第 i 位是 1 当且仅当 a 和 b 的第 i 位都是 1。
- a | b 的第 i 位是 1 当且仅当 a 和 b 的第 i 位至少有一个是 1。
- a ^ b 的第 i 位是 1 当且仅当 a 和 b 的第 i 位不同。

例：a & 1 就是拿 a 和 $\underbrace{0\dots001}_{n-1\text{个}}$ 求 bitwise AND，就相当于 a % 2。

位运算符

假设 a 是无符号整数。

- a << i 返回将 a 的二进制位集体左移 i 位的结果。
 - 例如，a << 1 就是 a * 2，a << i 就是 a 乘以 2ⁱ。
 - 左边超出的部分丢弃。
- a >> i 返回将 a 的二进制位集体右移 i 位的结果。
 - 例如，a >> 1 就是 a / 2。右边超出的部分被丢弃。

例：如何得到 2ⁿ？

位运算符

假设 `a` 是无符号整数。

- `a << i` 返回将 `a` 的二进制位集体左移 `i` 位的结果。
 - 例如, `a << 1` 就是 `a * 2`, `a << i` 就是 `a` 乘以 2^i 。
 - 左边超出的部分丢弃。
- `a >> i` 返回将 `a` 的二进制位集体右移 `i` 位的结果。
 - 例如, `a >> 1` 就是 `a / 2`。右边超出的部分被丢弃。

例: 如何得到 2^n ? `1 << n`

- 但是 `1` 是 `int` 类型的字面值。
- 如果 `n` 比较大, 你可能需要 `1u << n`, `1ll << n` 等等。

位运算符

如何截取一个无符号整数 `x` 的第 $[low, high)$ 位?

```
unsigned bit_slice(unsigned x, unsigned low, unsigned high) {  
      
}
```

位运算符

如何获得一个无符号整数 `x` 的第 `i` 位?

- 我们约定 $i \in [0, N)$, 其中 N 是 `x` 的类型的总位数。第 `0` 位是最右边的位 (least significant bit)。

```
unsigned test_bit(unsigned x, unsigned i) {  
      
}
```

位运算符

如何截取一个无符号整数 `x` 的第 $[low, high)$ 位?

```
unsigned bit_slice(unsigned x, unsigned low, unsigned high) {  
    x >>= low; // 先右移 low 位  
    // 然后我们需要和一个 0...00011111 这样的数 & 一下  
    // 如何获得一个这样的数?  
}
```

位运算符

如何获得一个无符号整数 `x` 的第 `i` 位?

```
unsigned test_bit(unsigned x, unsigned i) {  
    return (x >> i) & 1u;  
}
```

或者

```
unsigned test_bit(unsigned x, unsigned i) {  
    return (x & (1u << i)) >> i;  
}
```

位运算符

如何截取一个无符号整数 `x` 的第 $[low, high)$ 位?

- 先右移 `low` 位, 然后和一个末 `high - low` 位全是 `1`、其它全是 `0` 的数 `&` 一下

```
unsigned bit_slice(unsigned x, unsigned low, unsigned high) {  
    return (x >> low) & ((1u << (high - low)) - 1);  
}
```

还有没有别的办法?

位运算符

如何翻转一个无符号整数 `x` 的第 `i` 位?

```
unsigned bit_flip(unsigned x, unsigned i) {  
      
}
```

位运算符

如何截取一个无符号整数 `x` 的第 $[low, high)$ 位?

- 先右移 `low` 位, 然后和一个末 `high - low` 位全是 `1`、其它全是 `0` 的数 `&` 一下

```
unsigned bit_slice(unsigned x, unsigned low, unsigned high) {  
    return (x >> low) & ((1u << (high - low)) - 1);  
}
```

或者, 先和一个末 `high` 位全是 `1`、其它全是 `0` 的数 `&` 一下, 再右移 `low` 位

```
unsigned bit_slice(unsigned x, unsigned low, unsigned high) {  
    return (x & ((1u << high) - 1)) >> low;  
}
```

位运算符

如何翻转一个无符号整数 `x` 的第 `i` 位?

```
unsigned bit_flip(unsigned x, unsigned i) {  
    return x ^ (1u << i);  
}
```

神奇的异或 (XOR)

输入 n 个 32 位非负整数, 其中有一个数出现了奇数次, 其它数都出现了偶数次。求那个出现了奇数次的数。

神奇的异或 (XOR)

输入 n 个 32 位非负整数，其中有一个数出现了奇数次，其它数都出现了偶数次。求那个出现了奇数次的数。

答案: $x_1 \oplus x_2 \oplus \cdots \oplus x_n$ 。

```
int main(void) {
    int n; scanf("%d", &n);
    unsigned result = 0;
    while (n--) {
        unsigned x; scanf("%u", &x);
        result ^= x; // 啊?
    }
    printf("%u\n", result);
    return 0;
}
```

30/39

神奇的异或 (XOR)

输入 n 个 32 位非负整数，其中有一个数出现了奇数次，其它数都出现了偶数次。求那个出现了奇数次的数。

首先证明，异或具有交换律和结合律：

- $a \oplus b = b \oplus a$
- $(a \oplus b) \oplus c = a \oplus (b \oplus c)$

并且 $a \oplus a = 0, a \oplus 0 = a$ 。

那么对于 $x_1 \oplus \cdots \oplus x_n$ ，任意交换它们的顺序，把相同的数碰到一起，它们就会消失，剩下的就是那个出现了奇数次的。

31/39

神奇的异或 (XOR)

输入 n 个 32 位非负整数，其中有一个数出现了奇数次，其它数都出现了偶数次。求那个出现了奇数次的数。

另一种想法：考虑那个出现了奇数次的数 r

- 如果 r 的第 i 位是 1，意味着所有的数中，第 i 位一共有奇数个 1。
- 如果 r 的第 i 位是 0，意味着所有的数中，第 i 位一共有偶数个 1。

所以我们需要一种神奇的操作，让偶数个 1 变成 0，让奇数个 1 变成 1。

- 或者说，这是二进制意义下的不进位加法。

32/39

神奇的异或 (XOR)

输入 n 个 32 位非负整数，其中有一个数出现了奇数次，其它数都出现了偶数次。求那个出现了奇数次的数。

另一种想法：考虑那个出现了奇数次的数 r

- 如果 r 的第 i 位是 1，意味着所有的数中，第 i 位一共有奇数个 1。
- 如果 r 的第 i 位是 0，意味着所有的数中，第 i 位一共有偶数个 1。

所以我们需要一种神奇的操作，让偶数个 1 变成 0，让奇数个 1 变成 1。

- 或者说，这是二进制意义下的不进位加法。——这就是异或！

33/39

控制流

主要讲两个作用域问题

34/39

do-while 的作用域问题

每次输入一个整数，做一些处理，如果输入的是 0 那么处理完毕后停止。你会怎么写？

35/39

do-while 的作用域问题

每次输入一个整数，做一些处理，如果输入的是 0 那么处理完毕后停止。你会怎么写？

```
do {
    int x;
    scanf("%d", &x);
    do_something(x);
} while (x != 0);
```

```
a.c:9:12: error: ‘x’ undeclared (first use in this function)
    9 |     } while (x != 0);
```

36/39

do-while 的作用域问题

do-while 的循环体是一个内层的作用域，它以 { 开始、} 结束，不包含 while (cond) 的部分。

在 do-while 循环体内声明的变量，无法在 cond 部分使用。

```
do {
    int x;
    scanf("%d", &x);
    do_something(x);
} while (x != 0); // Error: `x` undeclared.
```

37/39

switch-case 的作用域问题

始终牢记，控制流跳转到一个 case 标签对应的语句后会一直往下执行，直到碰到 break; 或者到达末尾。

```
switch (expr) {
    case 1:
        int x = 42;
        do_something(x, expr);
    case 2:
        // 如果这里使用了 `x`，怎么办?
        printf("%d\n", x);
}
```

如果 expr == 2，控制流根本就没有经过 int x = 42; 这条语句，但是根据名字查找的规则却能找到 x。

38/39

switch-case 的作用域问题

```
switch (expr) {
    case 1: { // 用 {} 将 `x` 限定在内层作用域中。
        int x = 42;
        do_something(x, expr);
    }
    case 2:
        printf("%d\n", x); // Error: `x` was not declared in this scope.
}
```

如果 expr == 2，控制流根本就没有经过 int x = 42; 这条语句，但是根据名字查找的规则却能找到 x。

为了解决这个问题，如果在某个 case 内部声明了变量，这个变量必须存在于一个内层作用域中。

- 简单来说就是要加 {}。

39/39

CS100 Recitation 3

GKxx

目录

- 预处理指令
- 指针和数组

预处理指令

预处理指令 (preprocessor directives)

“预处理指令”是那些以 # 开头的指令，例如 #include, #define, #if, #ifdef, #endif, ...

- 有一个可能比较特殊的是 #pragma，我们暂时忽略

编译一个 C/C++ 程序时，编译器会首先调用**预处理器** (preprocessor) 处理所有的预处理指令。

- 有时也认为预处理器是编译器的一部分

试一试：`gcc a.c --save-temps` 或者 `gcc a.c -E`

#include

#include 的含义极其简单：**文本替换**。它会按照某种规则，找到被 #include 的那个文件，将其中的内容原封不动地复制粘贴过来。

- #include 和其它语言的 import 是完全不同的，它远远不如 import 聪明。
- 缺乏更智能、更现代的 import / export / modules 也是 C/C++ 的槽点之一，但是 C++20 有 modules 了！（但是**主流编译器支持**还不知道要哪年才能完工）

#include <somefile> 会让编译器去两个地方找 somefile，一是它自己预先设定好的标准库文件的位置，二是你编译的时候通过 -I 指定的路径。（GCC）

- `gcc a.c -o a -I/home/gkxx/my_awesome_library/include`

#include "somefile" 除了以上两种情况外，还可以将 somefile 视为相对路径：

```
#include "../my_library/include/my_header.h"
```

#define

#define 的含义也是**文本替换**：

```
#define N 1000
#define MAX(A, B) A < B ? B : A
```

在这之后，所有 N 都会被替换为 1000，所有 MAX(expr1, expr2) 都会被替换为 expr1 < expr2 ? expr2 : expr1。

* 这样的 MAX 真的没问题吗？

#define

```
#define MAX(A, B) A < B ? B : A
```

在这之后，所有 MAX(expr1, expr2) 都会被替换为 expr1 < expr2 ? expr2 : expr1。

```
int i = 10, j = 15;
int k = MAX(i, j) + 1;
```

它被替换为

```
int i = 10, j = 15;
int k = i < j ? j : i + 1; // i < j ? j : (i + 1)
```

#define

```
#define MAX(A, B) (A < B ? B : A)
```

加一对括号就行了？

```
int i = 10, j = 15;
int k = MAX(i, i & j); // 比较 i 和 i & j
```

它被替换为

```
int i = 10, j = 15;
int k = (i < i & j ? i & j : i); // (i < i) & j
```

运算符优先级：比较运算符 > bitwise AND, bitwise XOR, bitwise OR

#define

```
#define MAX(A, B) ((A) < (B) ? (B) : (A))
```

全加括号总可以了吧？

```
int i = 10, j = 15;
int k = MAX(i, ++j);
```

它被替换为

```
int i = 10, j = 15;
int k = ((i) < (++j) ? (++j) : (i));
```

j 有可能被 ++ 两次！

#define

结论：不要用 #define 来代替函数。

要定义常量，在 C++ 里也有比 #define 更好的办法。

1/44

6/44

2/44

7/44

3/44

8/44

4/44

9/44

5/44

10/44

指针和数组

指针 (pointer)

一个指针**指向**一个变量。指针所储存的**值**是它所指向的变量的内存地址。

```
int i = 42;
int* pi = &i;
printf("%d\n", *pi);
```

- int* pi 声明了一个名为 pi 的指针，它指向的变量的类型是 int。
- & 是**取地址运算符**，用来获得一个变量的地址。
- *pi 中的 * 是**解引用运算符**，用来获得一个指针所指向的对象。

指针 (pointer)

```
int i = 42;
int* pi = &i;
printf("%d\n", *pi); // 42
*pi = 35;
printf("%d\n", *pi); // 35
printf("%d\n", i); // 35
```

指针 (pointer)

- 声明语句 Type* p; 中，* 表示声明的变量是**指针**。
- 表达式 *p 中的 * 是**解引用运算符**。
- 一个符号只有在**表达式**中才可能是**运算符**。

指针 (pointer)

Type* p; 和 Type* p; 是一样的：* 靠着谁都行，甚至可以 Type* p;

- 但是如果要是声明多个指针，每个变量名前面必须都有一个*：

```
int *p1, p2, *p3; // p1 和 p3 是 int *, 但 p2 是 int
int* q1, q2, q3; // 只有 q1 是 int *, q2 和 q3 都是 int
```

- Type* p 可能更直观（“指针修饰符也是类型的一部分”），但它具有欺骗性。
- 选择一种并坚持。如果你选择 Type* p，不要在一条语句里定义多个指针。

指针 (pointer)

如果指针没有被显式初始化：

- 局部非静态：未初始化，拥有未定义的值——不知道指向哪。
- 全局或局部静态：**空初始化**，值是 NULL，即**空指针**。

也可以显式地让一个指针置空：p = NULL;

p = 0; 也是合法的，但不推荐。

事实上自 C++11 和 C23 起，标准都引入了更类型安全 (type-safe) 的空指针 nullptr。

在 C++ 中，请使用 nullptr 而非 NULL。

参数传递

```
void fun(int x) {
    x = 42;
}
int main(void) {
    int i = 30;
    fun(i);
    printf("%d\n", i);
}
```

参数传递

```
void fun(int x) {
    x = 42;
}
int main(void) {
    int i = 30;
    fun(i);
    printf("%d\n", i); // 30
}
```

传参的过程中，相当于发生了 int x = i; 这样的初始化。修改 x 的值并不会同时修改 i 的值。

参数传递

如果想要让函数修改外部变量的值，我们需要传递那个变量的地址。

```
void fun(int *px) {
    *px = 42;
    // 将 px 指向的变量的值修改为 42，
    // 或者说，将 px 所表示的地址上存放的变量的值修改为 42。
}
int main(void) {
    int i = 30;
    fun(&i); // 传递 i 的地址
    printf("%d\n", i); // 42
}
```

参数传递

交换两个整数的 swap

```
void swap(int *pa, int *pb) {
    int tmp = *pa;
    *pa = *pb;
    *pb = tmp;
}
int main(void) {
    int i = 42, j = 35;
    swap(&i, &j);
    // ...
}
```

* 为何 scanf 的参数需要取地址，而 printf 的参数不需要？

参数传递

如果要交换的是两个 `int*` 怎么办?

- 如果要交换的是两个 `T` 类型的变量怎么办?

21/44

参数传递

如果要交换的是两个 `T` 类型的变量怎么办?

```
void swap(T *pa, T *pb) {
    T tmp = *pa;
    *pa = *pb;
    *pb = tmp;
}

int main(void) {
    T x = something(), y = something_else();
    swap(&x, &y);
    // ...
}
```

22/44

未定义的行为

如果一个指针没有指向一个实际的对象:

- 它可能是未初始化的 (俗称“野指针”)
- 可能是空指针
- 可能是指向的内存刚被释放掉 (“空悬指针”) (下次课再说)
- 或者其它无意义的地址: `int *p = 123;`

试图解引用这个指针是 **undefined behavior**, 并且通常是**严重的运行时错误**。

- 还记得逻辑运算符的“短路求值”吗?

```
if (p != NULL && *p == 42) { /* ... */ }
```

23/44

数组

一片连续的存储区域

```
Type name[N];
```

N 必须是**常量表达式**: 它的值能确定在编译时已知。

```
int a1[10]; // 正确, 字面值是常量表达式
#define MAXN 10
int a2[MAXN]; // 正确, MAXN 被预处理器替换为 10
int n;
scanf("%d", &n);
int a[n]; // 见下一页
```

24/44

数组

```
int n;
scanf("%d", &n);
int a[n];
```

如果数组 `a` 的大小是运行期确定的, 它就是一个 **Variable-Length Array (VLA)**。

- VLA 自 C99 被加入 C 语言标准, 但 C11 起编译器有权决定是否支持它。
- 我们**不推荐也不允许**使用 VLA。
 - 等学了动态内存再说。
- C++14 时人们曾经讨论过是否在 C++ 中引入 VLA, 但最终还是将它 vote out。
 - 等学 C++ 的时候再解释为何 C++ 难以支持 VLA。

可以看[我的博客](#)

25/44

数组下标

可以用 `a[i]` 来访问数组 `a` 的第 `i` 个元素, 读写都可以。

```
int a[10];

bool find(int value) {
    for (int i = 0; i != 10; ++i)
        if (a[i] == value)
            return true;
    return false;
}

int main(void) {
    int n; scanf("%d", &n);
    for (int i = 0; i != n; ++i)
        scanf("%d", &a[i]);
    for (int i = 0; i != n; ++i)
        a[i] *= 2;
    // ...
}
```

下标的范围是 $[0, N)$ 的整数。下标访问越界本质上等同于解引用无效的指针, 是**未定义的行为**, 并且是**严重的运行时错误**。

26/44

数组下标

下标的范围是 $[0, N)$ 的整数。下标访问越界本质上等同于解引用无效的指针, 是**未定义的行为**, 并且通常是**严重的运行时错误**。

还是那句话: 编译器可以假定你的程序没有未定义的行为。

```
int a[10];

bool find(int value) {
    for (int i = 0; i <= 10; ++i)
        if (a[i] == value)
            return true;
    return false;
}
```

- 这段代码可能被直接优化为

```
bool find(int value) {
    return true;
}
```

[更多例子](#)

27/44

数组的初始化

和普通变量类似, 如果没有显式初始化:

- 局部非静态的数组: 未初始化, 数组里的所有元素都具有未定义的值。
- 全局或局部静态的数组: **空初始化**, 就是对数组里的所有元素进行**空初始化**。

但数组还有特殊的初始化规则...

28/44

数组的初始化

但数组还有特殊的初始化规则: [使用大括号初始化列表](#)

- 可以对数组的前几个元素进行初始化:

```
int a[10] = {2, 3, 5, 7}; // Correct: Initializes a[0], a[1], a[2], a[3]
int b[2] = {2, 3, 5}; // Error: Too many initializers
int c[] = {2, 3, 5}; // Correct: 'c' has type int[3].
int d[100] = {}; // Correct in C++ and since C23.
```

- 还可以使用 **designators** (since C99):

```
int e[10] = {[0] = 2, 3, 5, [7] = 7, 11, [4] = 13};
```

29/44

数组的初始化

如果对数组进行了显式初始化, 所有没有指定初始值的元素都被**空初始化**!

```
int main(void) {
    int a[10] = {1, 2, 3}; // a[3] 及后续元素都是 0
    int b[100] = {0}; // b 的所有元素都是 0
    int c[100] = {1}; // c[0] 是 1, c[1] 及后续元素都是 0
    int d[100] = {}; // 自 C23 起合法, 所有元素都是 0
}
```

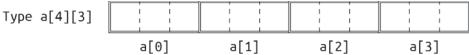
不要认为 `= {x}` 是将所有元素初始化为 `x`!

30/44

多维数组

C 没有真正意义上的“多维数组”（但 C++23 有多维下标运算符！！）

所谓的“多维数组”其实是数组的数组：



```
int a[10][20];

bool find(int value) {
    for (int i = 0; i != 10; ++i)
        for (int j = 0; j != 20; ++j)
            if (a[i][j] == value)
                return true;
    return false;
}
```

31/44

多维数组的初始化

标准几乎已经说得非常清楚了。

```
int a[4][3] = { // array of 4 arrays of 3 ints each (4x3 matrix)
    { 1 },      // row 0 initialized to {1, 0, 0}
    { 0, 1 },   // row 1 initialized to {0, 1, 0}
    { [2]=1 },  // row 2 initialized to {0, 0, 1}
    {}         // row 3 initialized to {0, 0, 0}
};

int b[4][3] = { // array of 4 arrays of 3 ints each (4x3 matrix)
    1, 3, 5, 2, 4, 6, 3, 5, 7 // row 0 initialized to {1, 3, 5}
    // row 1 initialized to {2, 4, 6}
    // row 2 initialized to {3, 5, 7}
    // row 3 initialized to {0, 0, 0}
};

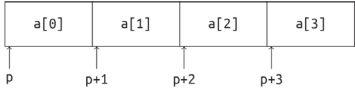
int y[4][3] = {[0][0]=1, [1][1]=1, [2][0]=1}; // row 0 initialized to {1, 0, 0}
                                                // row 1 initialized to {0, 1, 0}
                                                // row 2 initialized to {1, 0, 0}
                                                // row 3 initialized to {0, 0, 0}
```

32/44

指针的算术运算

对于指向 Type 类型的指针 p 和一个整数 i：

- p + i 得到的地址是 (char *)p + i * sizeof(Type)，即和 p 相距 i 个 Type。
- 所以如果让 p = &a[0]，那么 p + i 就等于 &a[i]，*(p + i) 就等价于 a[i]。



- i + p, p += i, p - i, p -= i, ++p, p++, --p, p-- 等运算也是类似。

33/44

数组向指针的隐式转换

如果 p = &a[0]，那么 p + i 就等于 &a[i]，*(p + i) 就等价于 a[i]。

鉴于数组和指针有如此紧密的联系，C/C++ 允许数组隐式转换为指向其首元素的指针：

- a → &a[0]
- T [N] → T *

因此，

- p = &a[0] 可以直接写成 p = a。
- *a 就是 a[0]。

34/44

数组向指针的隐式转换

如果 p = &a[0]，那么 p + i 就等于 &a[i]，*(p + i) 就等价于 a[i]。

鉴于数组和指针有如此紧密的联系，C/C++ 允许数组隐式转换为指向其首元素的指针。

- 可以用指针的方式遍历数组：

```
int a[10];

bool find(int value) {
    for (int *p = a; p != a + 10; ++p)
        if (*p == value)
            return true;
    return false;
}
```

35/44

指针相减



如果指针 p1 和 p2 分别指向某一个数组的下标为 i 和 j 的位置，则 p1 - p2

- 的结果是 i - j，
- 类型为 ptrdiff_t：一个定义在 <stddef.h> 里的类型，是一种带符号整数，其具体大小是 implementation-defined。
 - 例如，在 64 位系统上 ptrdiff_t 可能是 64 位。
- 特别地，这里的 i 和 j 可以等于 N，即 p1 或 p2 指向数组的最后一个元素的下一个位置（“尾后”，past-the-end）

36/44

指针的算术运算

简单来说，指针的算术运算只能在一个数组内部以及“尾后”位置的范围内发生。

以下统统是 undefined behavior:

- p1 - p2，其中 p1 和 p2 指向两个不同的数组中的位置
- p + 2 * N，其中 p 指向某个长度为 N 的数组
- p - 1，其中 p 指向某个数组的首元素 a[0]

37/44

向函数传递数组

C 语言没有办法声明一个数组参数（为什么？），所以传递数组的唯一方式是传递指向数组首元素的指针。

以下四种声明是完全等价的：参数 a 的类型是 int *。（如何验证？）

```
void fun(int *a);
void fun(int a[]);
void fun(int a[10]);
void fun(int a[2]);
```

38/44

向函数传递数组

C 语言没有办法声明一个数组参数（为什么？），所以传递数组的唯一方式是传递指向数组首元素的指针。

以下四种声明是完全等价的：参数 a 的类型是 int *。（如何验证？）

```
void fun(int *a);
void fun(int a[]);
void fun(int a[10]);
void fun(int a[2]);
```

验证：使用 sizeof 获得一个变量或类型所占的字节数。

39/44

向函数传递数组

就算以数组的形式声明一个参数，它的类型也是一个普通的指针：传递任何一个地址给它都是合法的，传递任何大小的数组给它也是合法的。

为何使用 scanf、gets 之类的函数读取字符串被视为不安全的？

为了让函数知道这个数组究竟有多长，常见的方法是显式地传递一个参数 n：

```
void print_array(int *a, int n) {
    for (int i = 0; i != n; ++i)
        printf("%d ", a[i]);
}
```

* 指针也能用下标？

40/44

向函数传递数组

练习：设计并编写一个函数，将一个整数数组里的奇数倒序拷贝给另一个数组。

41/44

向函数传递数组

练习：设计并编写一个函数，将一个整数数组里的奇数倒序拷贝给另一个数组。

```
void copy_odd_reversed(int *from, int n, int *to) {
    for (int i = n - 1, j = 0; i >= 0; --i)
        if (from[i] % 2 == 1)
            to[j++] = from[i];
}
```

42/44

指针也能用下标？

看看[标准](#)，你会发现下标运算符的语法是

- `pointer[integer]`
- `integer[pointer]`

也就是说：

- 下标运算符在语法上是为指针提供的，取数组的下标 `a[i]` 时实际上也发生了数组向首元素指针的隐式转换
- 不仅可以 `p[i]`，还可以 `i[p]` 是不是很奇怪
- 你只要知道“数组和指针都可以用下标”就行了，不是非得记住谁向谁转换。

43/44

数组向指针的隐式转换

事实上这种隐式转换无处不在：

```
Any lvalue expression of array type, when used in any context other than
    • as the operand of the address-of operator (&)
    • as the operand of sizeof
    • as the string literal used for array initialization
undergoes a conversion to the non-lvalue pointer to its first element.
```

以至于某些人会告诉你“数组名就是指针”，**但这是不对的！**

44/44

CS100 Recitation 4

GKxx

1/40

Contents

- 指针和数组（续）
- 字符串
- 动态内存

2/40

Homework 2

Solution 已发。对比一下自己的做法，看看能否改进。

3/40

指针和数组

指针类型

对于两个不同的类型 `T` 和 `U`，指针类型 `T*` 和 `U*` 是**不同的类型**（尽管它们有可能指向相同的地址）。

```
int i = 42;
float *fp = &i;
*fp += 1; // undefined behavior
```

尽管在 C 中，不同类型的指针之间可以相互（隐式）转换（在 C++ 中必须显式转换），但如果一个指针指向的对象和它被声明的类型不符，解引用这个指针几乎总是 undefined behavior。

- 除了一些情况（稍后会看到一种）

5/40

void *：“指向 void 的指针”

- 任何指针都可以（隐式）转换成 `void *`
- `void *` 可以（隐式）转换成任何指针
- 可以用 `printf("%p", ptr)` 输出一个 `void *` 类型的指针 `ptr` 的值。
 - 如果 `ptr` 是其它类型的指针，需要先转换：`printf("%p", (void *)ptr)`

6/40

`void *`

在没有 C++ 那样强的静态类型系统支持下，`void *` 经常被用来表示“任意类型的指针”、“（未知类型的）内存的地址”等等。

- `malloc` 的返回值类型就是 `void *`，`free` 的参数类型也是 `void *`。
- `pthread_create` 允许给线程函数传任意参数，方法就是用 `void *` 转交。
- 在 C 中，接受 `malloc` 的返回值时**不需要显式转换**。
 - `int *p = malloc(size)`；而非 `int *p = (int *)malloc(size)`；。

`void *` 是 C 类型系统真正意义上的天窗

向函数传递二维数组

以下声明中，参数 `a` 分别具有什么类型？哪些可以接受一个二维数组 `int [N][M]`？

1. `void fun(int a[N][M])`
2. `void fun(int (*a)[M])`
3. `void fun(int (*a)[N])`
4. `void fun(int **a)`
5. `void fun(int *a[])`
6. `void fun(int *a[N])`
7. `void fun(int a[100][M])`
8. `void fun(int a[N][100])`

退化 (decay)

- 数组向指向首元素指针的隐式转换（退化）：
 - `Type [N]` 会退化为 `Type *`
- “二维数组”其实是“数组的数组”：
 - `Type [N][M]` 是一个 `N` 个元素的数组，每个元素都是 `Type [M]`
 - `Type [N][M]` 应该退化为什么类型？

向函数传递二维数组

以下声明中，参数 `a` 分别具有什么类型？哪些可以接受一个二维数组 `int [N][M]`？

1. `void fun(int a[N][M])`：指向 `int [M]` 的指针，可以
2. `void fun(int (*a)[M])`：同 1
3. `void fun(int (*a)[N])`：指向 `int [N]` 的指针，当且仅当 `N==M` 时可以
4. `void fun(int **a)`：指向 `int *` 的指针，**不可以**
5. `void fun(int *a[])`：同 4
6. `void fun(int *a[N])`：同 4
7. `void fun(int a[100][M])`：同 1
8. `void fun(int a[N][100])`：指向 `int [100]` 的指针，当且仅当 `M==100` 时可以

退化 (decay)

- 数组向指向首元素指针的隐式转换（退化）：
 - `Type [N]` 会退化为 `Type *`
- “二维数组”其实是“数组的数组”：
 - `Type [N][M]` 是一个 `N` 个元素的数组，每个元素都是 `Type [M]`
 - `Type [N][M]` 退化为“指向 `Type [M]` 的指针”
- 如何定义一个“指向 `Type [M]` 的指针”？

const

`const` 变量：**一经初始化就不能再改变**（“常量”，所以当然**必须初始化**。

- “常量”这个词其实很容易引发歧义，C/C++ 中还有一种真正的“常量”，是指**值在编译期已知的量**。

可以定义“指向常量的指针”：`const Type *ptr` 或 `Type const *ptr`

稍微复杂一点儿的复合类型

指向数组的指针

存放指针的数组

```
int (*parr)[N];
int *arrp[N];
```

- 首先，记住**这两种写法都有，而且是不同的类型**。
- `int (*parr)[N]` 为何要加一个圆括号？当然是因为 `parr` 和“指针”的关系更近
 - 所以 `parr` **是指针**，
 - 指向的东西是 `int [N]`
- 那么另一种则相反：
 - `arrp` **是数组**，
 - 数组里存放的东西是指针。

pointer-to- const

一个“指向常量的指针”也可以指向一个 `non-const` variable

- 但它**自以为**自己指向了“常量”，所以不允许你通过它修改它所指向的变量的值。

```
int i = 42;
const int *cip = &i;
int *ip = &i;
++i; // OK
++*ip; // OK
++*cip; // Error
```

- “底层 `const`”（low-level `const`）

向函数传递二维数组

以下声明了**同一个函数**：参数类型为 `int (*)[N]`，即一个指向 `int [N]` 的指针。

```
void fun(int (*a)[N]);
void fun(int a[][N]);
void fun(int a[2][N]);
void fun(int a[10][N]);
```

可以传递 `int [K][N]` 给 `fun`，其中 `K` 可以是任意值。

- 第二维大小必须是 `N`。`Type [10]` 和 `Type [100]` 是不同的类型，指向它们的指针之间不兼容。

pointer-to- const

不能用 `pointer-to-non-const` 指向一个真正的 `const` 变量，也不能用一个 `pointer-to-const` 给它赋值或初始化（“不能去除底层 `const`”）

- 如果把 `const` 视为一把锁，这就是在试图拆掉锁。
- 你可以用 `explicit cast`（显式转换）强行拆锁，但由此引发的对于 `const` 变量的修改是 `undefined behavior`

```
const int ci = 42;
int *ip = (int *)&ci;
++*ip; // Undefined behavior

int i = 42;
const int *cip = &i;
int *ip = cip; // Warning in C, Error in C++
int *ip2 = (int *)cip; // OK
```

顶层 const (top-level const)

一个指针自己是 const 变量：它永远指向它初始化时指向的那个对象
有时称为“常量指针”

```
int ival = 42;
int *const ipc = &ival;
++*ipc; // Correct
int ival2 = 35;
ipc = &ival2; // Error. ipc is not modifiable.
```

当然也可以同时带有底层、顶层 const：

```
const int *const cipc = &ival;
```

17/40

字符串

“C 风格字符串” (C-style strings)

C 语言没有对应“字符串”的抽象，字符串就是一群字符排在一起。

- 可以是数组，也可以是动态分配的内存
- 末尾必须有一个 '\0'， '\0' 在哪末尾就在哪。

```
char s[10] = "abcde"; // s = {'a', 'b', 'c', 'd', 'e', '\0'}
printf("%s\n", s); // abcde
printf("%s\n", s + 1); // bcde
s[2] = ';'; // s = "ab;de"
printf("%s\n", s); // ab;de
s[2] = '\0';
printf("%s\n", s); // ab
```

19/40

结束符 '\0'

'\0' 是所谓的“空字符”，其 ASCII 值为 0。

C 风格字符串结束的唯一判断标志

所有号称接受、处理字符串的函数都会去找 '\0'

- 缺少 '\0' 会让他们不停地走下去，（很可能）导致越界访问。

用数组存储字符串时，记得为空字符多开一格。

```
char s[5] = "abcde"; // OK, but no place for '\0'.
puts(s); // undefined behavior (missing '\0')
```

20/40

字符串 IO

- scanf / printf : "%s"
 - scanf 读 "%s" 有内存安全问题：它并不知道你传给它的数组有多长
 - scanf 没被踢出去，（我猜）是因为它还有别的用途
- gets：自 C11 起被踢出标准，因为它只有这一个用途
 - 替代品 gets_s 在标准的附录里，很遗憾 GCC 没有支持它
- fgets：更通用，更安全

```
char str[100];
fgets(str, 100, stdin);
```

- puts(str)；输出字符串 str 并换行

21/40

拷贝字符串

练习：实现你自己的 strcpy 函数，将一个字符串拷贝到另一个地方。

拷贝字符串

大多数人总是按捺不住使用下标的冲动，从而先调用一遍 strlen：

```
char *strcpy(char *dest, const char *src) {
    size_t length = strlen(src);
    for (size_t i = 0; i <= length; ++i)
        dest[i] = src[i];
    return dest;
}
```

但是 strlen 也会把 src 这个字符串扫一遍。

- 为什么要扫两遍？

22/40

拷贝字符串

明明可以只扫一遍：终止条件是碰到 '\0'，而非 i == strlen(src)。

```
char *strcpy(char *dest, const char *src) {
    size_t i = 0;
    while (src[i] != '\0') {
        dest[i] = src[i];
        ++i;
    }
    dest[i] = '\0';
    return dest;
}
```

24/40

拷贝字符串：更直接的办法

直接移动指针就可以了，不需要下标。

- dest 和 src 都是你这个函数内部的变量，完全可以随便改！

```
char *strcpy(char *dest, const char *src) {
    char *ret = dest;
    while (*src != '\0')
        *dest++ = *src++;
    *dest = '\0';
    return ret;
}
```

```
char a[100], b[] = "hello";
strcpy(a, b); // 没问题。
```

25/40

strlen 没有魔法

strlen(s) 除了把 s 从头到尾扫一遍之外，没有任何更神奇的办法。

```
for (size_t i = 0; i < strlen(s); ++i)
    // ...
```

每次循环体执行完毕时，都要执行一次判断条件 i < strlen(s)，而每次算 strlen(s) 都需要遍历整个字符串，非常慢（时间复杂度为 $O(n^2)$ ）

应该改为

```
for (size_t i = 0, n = strlen(s); i < n; ++i)
    // ...
```

26/40

一个小问题

```
for (int i = 0; i < strlen(s); ++i)
// ...
```

编译器在 `i < strlen(s)` 上报了个 warning?

- `strlen` 返回值类型为 `size_t`: 无符号整数
 - 将 `int` 和 `size_t` 放在一起运算/比较时, `int` 值会被转换为 `size_t` 类型
 - `-1 < strlen(s)` 几乎肯定是 `false`
- * 不要混用带符号数和无符号数

字符串字面值 (string literals)

字符串字面值: 类似这种 `"abcde"` (双引号!!!)

- 类型为 `char [N+1]`, 其中 `N` 是它的长度, `+1` 是为了放空字符。
- 它会被放在只读的内存区域
 - 在 C++ 中, 它的类型是 `const char [N+1]`, 非常合理。
- 和 `42` 不同, 字符串字面值具有长久的生命期, 你甚至可以取它的地址 `&"abcd"`。

用不带底层 `const` 的指针指向一个 string literal 是合法的, 但极易导致 undefined behavior:

```
char *p = "abcde";
p[3] = 'a'; // undefined behavior, and possibly runtime-error.
```

字符串字面值 (string literals)

用不带底层 `const` 的指针指向一个 string literal 是合法的, 但极易导致 undefined behavior:

```
char *p = "abcde";
p[3] = 'a'; // undefined behavior, and possibly runtime-error.
```

正确的做法:

加上底层 `const` 的保护

或者将内容拷贝进数组

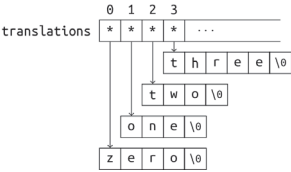
```
const char *str = "abcde";
```

```
char arr[] = "abcde";
```

字符串数组

```
const char *translations[] = {
    "zero", "one", "two", "three", "four",
    "five", "six", "seven", "eight", "nine"
};
```

- `translations` 是一个数组, 存放的元素是指针, 每个指针都指向一个 string literal
- `translations` 不是二维数组!



标准库函数

完整列表, 你想要的都在这里

HW3 会有一道造标准库函数的题。

What I cannot create, I do not understand. - Richard Feynman

动态内存

使用 `malloc` 和 `free`

创建一个“动态数组”: 大小在运行时确定

```
Type *ptr = malloc(sizeof(Type) * n);
for (int i = 0; i != n; ++i)
    ptr[i] = /* ... */
// ...
free(ptr);
```

使用 `malloc` 和 `free`

也可以动态创建一个对象

```
int *ptr = malloc(sizeof(int));
*ptr = 42;
// ...
free(ptr);
```

但是为什么需要这样? 直接创建 `int ival = 42;` 不香吗?

使用 `malloc` 和 `free`

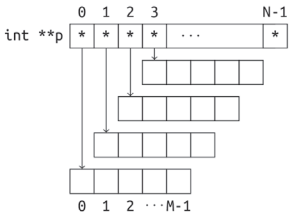
动态创建对象的好处: 生命周期跨越作用域

```
int *create_array(void) {
    int a[N];
    return a; // 返回了一个指向局部变量 a 的指针
              // 当函数返回时, 局部变量 a 已经被销毁, 该地址无意义
              // 解引用这个指针将导致 undefined behavior
}
int *create_array2(int n) {
    return malloc(sizeof(int) * n); // OK. 动态分配的内存直到被 free 才被销毁
}
```

使用 `malloc` 和 `free`

动态创建一个“二维数组”?

```
int **p = malloc(sizeof(int *) * n);
for (int i = 0; i != n; ++i)
    p[i] = malloc(sizeof(int) * m);
for (int i = 0; i != n; ++i)
    for (int j = 0; j != m; ++j)
        p[i][j] = /* ... */
for (int i = 0; i != n; ++i)
    free(p[i]);
free(p);
```



使用 malloc 和 free

动态创建一个“二维数组”——另一种方法：创建一维数组

```
int *p = malloc(sizeof(int) * n * m);
for (int i = 0; i != n; ++i)
    for (int j = 0; j != m; ++j)
        p[i * m + j] = /* ... */
// ...
free(p);
```

37/40

malloc, calloc 和 free

看标准! malloc calloc free

```
void *malloc(size_t size);
void *calloc(size_t num, size_t each_size);
void free(void *ptr);
```

- calloc 分配的内存大小至少是 num * each_size 字节
- malloc 分配未初始化的内存，每个元素都具有 indeterminate value
 - 但你可能试来试去发现都是 0？这是巧合吗？
- calloc 会将分配的内存的每个字节都初始化为零。
- free 释放动态分配的内存。在调用 free(ptr) 后，ptr 指向无效的内存 (dangling pointer)

38/40

malloc, calloc 和 free

malloc(0), calloc(0, N) 和 calloc(N, 0) 的行为是 implementation-defined

- 有可能不分配内存，返回空指针
- 也有可能分配一定量的内存，返回指向这个内存起始位置的指针
 - 但解引用这个指针是 undefined behavior
 - 这个内存同样需要记得 free，否则也构成内存泄漏

39/40

正确使用 free

- 忘记 free：内存泄漏，OJ 上无法通过测试。
- free(ptr) 的 ptr 必须是之前由 malloc, calloc, realloc 或 C11 的 aligned_alloc 返回的一个地址
 - 必须指向动态分配的内存的开头，不可以从中间某个位置开始 free
 - 会释放由 ptr 开头的整片内存，它自有办法知道这片内存有多长
- free 一个空指针是**无害的**，不需要额外判断 ptr != NULL
- free 过后，ptr 指向**无效的地址**，不可对其解引用。再次 free 这个地址的行为 (double free) 是 undefined behavior。

40/40

CS100 Recitation 5

GKxx

目录

- 重定向与文件 IO
- struct
- enum, bitfield, union

2/40

重定向与文件 IO

输入流、输出流

“流” (stream)：这些字符像水流一样，流过便不再回头。

stdin, stdout：标准输入流、标准输出流

- 当我们在终端运行一个程序的时候，在默认情况下，stdin 和 stdout 都被绑定到这个终端。
- ./program < my_input_file.txt：将 stdin 重定向到 my_input_file.txt
- ./program > my_output_file.txt：将 stdout 重定向到 my_output_file.txt
 - 如果这个文件不存在，就创建出来。
 - 如果这个文件存在，它原有的内容会被覆盖掉。
 - ./program >> my_output_file.txt：追加，而非覆盖。
- ./program < testcases/3.in > output.txt

4/40

freopen：在代码中实现“重定向” (redirect)

```
freopen("testcases/3.in", "r", stdin);
freopen("output.txt", "w", stdout);
```

File access mode string	meaning	explanation
"r"	read	open a file for reading
"w"	write	create a file for writing
"a"	append	append to a file
"r+"	read extended	open a file for read/write
"w+"	write extended	create a file for read/write
"a+"	append extended	open a file for read/write

5/40

freopen：在代码中实现“重定向” (redirect)

```
freopen("testcases/3.in", "r", stdin);
freopen("output.txt", "w", stdout);
```

事实上 freopen 能做的不止是给 stdin / stdout 重定向！

永远不要仅凭课件上的只言片语和例子学习标准库函数。

6/40

1/40

标准输入、输出流

`scanf`, `printf`, `getchar`, `putchar`, `puts` 这些函数都使用 `stdin` 和 `stdout` 。

例如,

- 如果在使用 `scanf` 前将 `stdin` 重定向到某个文件, `scanf` 就会从那个文件读入。
- 如果在使用 `puts` 前将 `stdout` 重定向到某个文件, `puts` 就会向那个文件写入。

7/40

文件 IO

```
FILE *infile = fopen("relative/path/to/my/input/file", "r");
int a, b;
fscanf(infile, "%d%d", &a, &b);
FILE *outfile = fopen("relative/path/to/my/output/file", "w");
fprintf(outfile, "%d\n", a + b);
fclose(infile);
fclose(outfile);
```

- `fopen` : 打开一个文件, 获得一个 `FILE *` (文件指针)。
- `fclose` : 关闭一个文件。参数是一个 `FILE *`。
- `fscanf`, `fprintf`, `fputs`, `fgets`, `fputc`, `fgetc` : 在原来的函数的基础上, 多传一个 `FILE *` 类型的参数。
 - 如果传入 `stdin` / `stdout`, 它就和普通的版本一样了。

8/40

struct

struct

把几个东西结合在一起, 定义成一个新的数据结构

```
struct Student {
    const char *name;
    const char *id;
    int entrance_year;
    int dorm;
};
```

```
struct Record {
    void *ptr;
    size_t size;
    int line_no;
    const char *file_name;
};
```

```
struct brainfuck_state {
    uint8_t *memory_buffer;
    size_t offset;
    // ...
};
```

```
struct Point3d {
    double x, y, z;
};
```

```
struct Line3d {
    struct Point3d p0, direction;
};
```

10/40

Product type

假设 `struct X` 具有如下成员

```
struct X {
    T_1 mem_1;
    T_2 mem_2;
    // ...
    T_n mem_n;
};
```

设类型 $T_1, T_2, \dots, T_n$ 的值的集合分别为 $\mathcal{T}_1, \mathcal{T}_2, \dots, \mathcal{T}_n$, 则 `struct X` 的值的集合为 $\mathcal{X} = \mathcal{T}_1 \times \dots \times \mathcal{T}_n = \{(t_1, \dots, t_n) \mid t_1 \in \mathcal{T}_1, \dots, t_n \in \mathcal{T}_n\}$

11/40

struct 类型

`struct` + 名字。C 中 `struct` 关键字不可省略, C++ 中必须省略。

```
struct Student student;
struct Record records[1000];
```

typedef 定义类型别名

```
typedef long long LL;
typedef struct { double x, y, z; } Point3d;

LL llval = 0; // llval is long long
Point3d p;
```

不要用 `#define` 代替 `typedef`

12/40

struct 的成员

`name.mem`

```
struct Student student;
student.name = "Alice";
student.id = "2023533000";
student.entrance_year = 2023;
student.dorm = 8;
printf("%d\n", student.dorm);
++student.entrance_year;
puts(student.name);
```

13/40

struct 的成员

`ptr->mem` : 等价于 `(*ptr).name`。不是 `*ptr.name` !! (. 优先级高于 *)

```
struct Student *ptr = &student;
ptr->name = "Alice";
ptr->id = "2023533000";
(*ptr).entrance_year = 2023; // equivalent to ptr->entrance_year = 2023;
ptr->dorm = 8;
printf("%d\n", ptr->dorm);
++ptr->entrance_year;
puts(ptr->name);
```

14/40

struct 初始化

老生常谈的问题: 不显式初始化时会发生什么?

```
struct Student gs;
int main(void) {
    struct Student ls;
}
```

15/40

struct 初始化

老生常谈的问题: 不显式初始化时会发生什么?

```
struct Student gs;
int main(void) {
    struct Student ls;
}
```

- 全局或局部 `static` : 空初始化: 结构体的所有成员都被空初始化。
- 局部非 `static` : 不初始化, 所有成员都具有未定义的值。

16/40

struct 的初始化

Initializer list:

```
struct Record r = {p, cnt * each_size, __LINE__, __FILE__};
```

隔壁 C++20 才有的 designators, C99 就有了! (是不是非常像 Python?)

```
struct Record r2 = {.ptr = p, .size = cnt * each_size,
                   .line_no = __LINE__, .file = __FILE__};
```

C 允许 designators 以任意顺序出现, C++不允许。

17/40

struct 的初始化

```
struct Record r = {p, cnt * each_size, __LINE__, __FILE__};
struct Record r2 = {.ptr = p, .size = cnt * each_size,
                   .line_no = __LINE__, .file = __FILE__};
```

赋值不行:

```
r = {p, cnt * each_size, __LINE__, __FILE__}; // Error
records[i] = {.ptr = p, .size = cnt * each_size,
             .line_no = __LINE__, .file = __FILE__}; // Error
```

但可以使用 compound literal :

```
r = (struct Record){p, cnt * each_size, __LINE__, __FILE__}; // OK
records[i] = (struct Record){.ptr = p, .size = cnt * each_size,
                             .line_no = __LINE__, .file = __FILE__}; // OK
```

18/40

拷贝一个 struct

一个 struct 可以被拷贝, 其行为是逐成员拷贝。

```
struct Student a = {"Alice", "2023533000", 2023, 8};
struct Student b = a; // OK, 初始化
struct Student c = something();
c = a; // OK, 赋值
```

- // struct Student b = a; 就如同下面的代码
struct Student b = {a.name, a.id, a.entrance_year, a.dorm};
- // c = a; 就如同下面的代码
c = (struct Student){a.name, a.id, a.entrance_year, a.dorm};

19/40

拷贝一个 struct

```
struct A {
    int array[10];
    // other members ...
};
```

虽然编译器拒绝直接拷贝数组, 但它其实有能力做到。

拷贝一个 struct A 时, 编译器会自动逐元素拷贝数组。

```
int a[10];
int b[10] = a; // Error!

struct A a;
struct A b = a; // OK
```

20/40

在函数之间传递 struct

传参的语义是拷贝。

```
void print_info(struct Record r) {
    printf("%p, %zu, %d, %s\n", r.ptr, r.size, r.line_no, r.file_name);
}

print_info(records[i]);
```

传参时发生了这样的初始化, 也就是逐成员拷贝

```
struct Record r = records[i];
```

就如同

```
struct Record r = {records[i].ptr, records[i].size,
                  records[i].line_no, records[i].file_name};
```

21/40

在函数之间传递 struct

返回一个 struct : 严格按照语法来说, 也是拷贝:

```
struct Record fun(void) {
    struct Record r = something();
    some_computations(r);
    return r;
}

records[i] = fun();
```

return r; : 发生了形如 struct Record tmp = r; 的拷贝, 临时对象 tmp 是表达式 fun() 的求值结果。然后发生了形如 records[i] = tmp; 的拷贝。

但实际上这个过程会被编译器优化, 标准也是允许这种优化的。(我们以后在 C++ 里进一步讨论这个问题)

22/40

struct 的大小

```
struct A {
    int x;
    char y;
    double z;
};
```

sizeof(struct A) 是多少?

23/40

struct 的大小

```
struct A {
    int x;
    char y;
    double z;
};
```

sizeof(struct A) >= sizeof(int) + sizeof(char) + sizeof(double)。由于内存对齐的问题, 编译器可能会在某些地方插入一定的空白。

一般地, 对于一个 struct X , 有

$$\text{sizeof}(\text{struct } X) \geq \sum_{\text{member} \in X} \text{sizeof}(\text{member}).$$

24/40

struct 的大小

```
struct A {
    int x;
    struct A a;
};
```

sizeof(struct A) 是多少?

25/40

struct 的大小

```
struct A {
    int x;
    struct A a; // Error: use of incomplete type 'struct A'.
};
```

sizeof(struct A) = +∞。因此这种行为是不允许的。

- 从物理上讲: 这样的东西无法存储。
- C 类型系统认为: 在定义完毕之前, 这个类型是不完全类型 (incomplete type)。对于不完全类型, 不能定义这个类型的对象, 不能访问这个类型的成员, 只能定义这个类型的指针, 并且不能解引用。

26/40

常见问题：如何返回两个值/多个值？

```
??? minmax(int *array, int n) {
    int min = INT_MAX;
    int max = INT_MIN;
    for (int i = 0; i < n; ++i) {
        // ...
    }
    return /* (min, max) */
}

??? read_string(void) {
    char *buffer = /* ... */;
    size_t size = 0;
    // read the string ...

    buffer[size] = '\0';
    return /* (buffer, size) ? */
}
```

“返回一个值”：将信息从函数内传给调用者。

- 除了 `return`，还有什么办法可以传递这个信息？

27/40

除了 `return`，还有什么办法可以把信息传给调用者？

让调用者把存放结果的内存地址传进来，我往那儿写。

```
int add(int a, int b) {
    return a + b;
}

int main(void) {
    int sum = add(1, 2);
    // ...
}

void add(int a, int b, int *result) {
    *result = a + b;
}

int main(void) {
    int sum;
    add(1, 2, &sum);
    // ...
}
```

28/40

传递存放结果的内存地址

```
void minmax(int *array, int n, int *min, int *max) {
    *min = INT_MAX;
    *max = INT_MIN;
    for (int i = 0; i < n; ++i) {
        // 对 *min 和 *max 进行修改
    }
}

int main(void) {
    int a[] = {1, 2, 3};
    int min, max;
    minmax(a, 3, &min, &max);
    // ...
}
```

想“返回”多少个值都可以：

```
void minmax(int *array, int n, int *min, int *max, int *secmin, int *secmax);
```

29/40

传递存放结果的内存地址

还可以与 `return` 结合使用：

```
char *read_string(size_t *size) {
    char *buffer = /* ... */;
    *size = 0;
    // read the string ...
    // ++*size whenever a new character is read and stored.
    buffer[*size] = '\0';
    return buffer;
}

int main(void) {
    size_t size;
    char *string = read_string(&size);
    // ...
}
```

但像 `minmax` 这样的函数，返回的两个值“地位对等”，不建议用两种不同的方式。

30/40

更直接：`return` 一个 `struct`

C 没有内置的“元组” (tuple)，但它可以用 `struct` 造出来：

```
struct minmax_result { int min; int max; };

struct minmax_result minmax(int *array, int n) {
    int min = INT_MAX;
    int max = INT_MIN;
    // ...
    return (struct minmax_result){min, max};
}

int main(void) {
    int a[] = {1, 2, 3};
    struct minmax_result minmax = minmax(a, 3);
    int min = minmax.min, max = minmax.max;
    // ...
}
```

练习：用这种方式改写 `read_string`

31/40

逗号运算符

一些来自其他语言的选手可能会想这么写：

```
int minmax(int *array, int n) {
    // ...
    return min, max;
}
```

它真的能通过编译，但效果就是 `return max;`

- 表达式 `expr1, expr2` 中的 `,` 是**逗号运算符**：先对 `expr1` 求值，再对 `expr2` 求值，返回 `expr2` 的值。优先级最低的运算符。
 - 为数不多的运算对象求值顺序能确定的运算符之一。
- 并非所有逗号都是逗号运算符，例如 `func(a, b, c)` 中的逗号是函数调用传参的语法的一部分，`int a, b;` 中的逗号是声明多个变量的语法的一部分。

32/40

标准库的相关设计

`strtoul`：将一个以字符串形式存在的整数的数值提取出来（得到一个 `long`）。

```
long strtoul(const char *str, char **str_end, int base);
```

```
char str[] = "1234abcdef";
char *next_pos;
long value = strtoul(str, &next_pos, 10);
// value 是 1234，next_pos 指向 'a'
```

The functions set the pointer pointed to by `str_end` to point to the character past the last numeric character interpreted. If `str_end` is a null pointer, it is ignored.

`strtoll`，`strtod` 等函数也是这样。

33/40

怎样设计一个函数，让它修改一个变量？

要么 `x = f(x)`，要么 `f(&x)`。

```
const char *move_forward(const char *s)
{
    while (*s != '\0' && condition(*s))
        ++s;
    return s;
}

// call site:
ptr = move_forward(ptr);

void move_forward(const char **s)
{
    while (**s != '\0' && condition(**s))
        ++*s;
}

// call site:
move_forward(&ptr);
```

34/40

enum, bitfield, union

enum

定义一个新的类型，其值是某几个特定的（有名字的）值之一。

```
struct Text {
    int color; // 0 黑色, 1 红色, 2 绿色
    char *contents;
};

// in some function
struct Text text = something();
text.color = 0; // 设为黑色

enum TextColor { Black, Red, Green };
struct Text {
    enum TextColor color;
    char *contents;
};

// in some function
struct Text text = something();
text.color = Black;
```

- 不好的设计：使用 magic numbers

- 好的设计：使用 `enum`。

36/40

enum 实际上也是一种整数

```
const char *colorToString(enum TextColor color) {
    switch (color) { // 可以放在 switch 里
        // enum items 是整型枚举常量, 可以放在 case label 上
        case Black: return "Black";
        case Red: return "Red";
        case Green: return "Green";
        default: return "unknown color";
    }
}

const char *colorToString(enum TextColor color) {
    const char *colors[] = {"Black", "Red", "Green"};
    return colors[color]; // color 可以作为下标
}
```

- 你觉得哪一种写法更好?

37/40

2/42

视频 错误更正：在视频中出现的C++小拇指控制力量代码：powerCon[1][2][6][7][11][52][57][58][65][66]；逻辑错误。原先的代码为：

```
{
    powerCon[1][ 10];
    powerCon[2][ 10];
    powerCon[6][ 10];
    powerCon[7][ 10];
    powerCon[11][ 10];
    powerCon[52][ 10];
    powerCon[57][ 10];
    powerCon[58][ 10];
    powerCon[65][ 10];
}
```

在过往学习中使用过并进行多个GPIO引脚的定义，但是并没有深入了解这个符号的含义，由此在该功能实现后源代码时错误使用了，并且没有加以验证，导致了错误。

比较合适的方法是将小拇指控制的按键单独定义成一个数组，在控制小拇指的函数中增加传入参数，并进行一个for循环，从而控制小拇指力量。

```
#define littleFingerNum 9
u8 littleFingerID 【littleFingerNum】=(1,2,6,7,11,52,57,58,65);
```

【此处符号为英文输入法输入的左右方括号，不知道为啥在B站评论这里打出来会自动加粗

```
void littleFingerForce(u8 Force) // 调整小拇指控制区域的力量
{
    for(u8 i = 0; i < littleFingerNum; i++)
    {
        powerCon[littleFingerID【i】][Force];
    }
}
```

38/40

3/42

Bitfield

注意：以下代码的行为是 implementation-dependent 的

```
struct Instruction {
    unsigned imm : 7;
    unsigned r2 : 3;
    unsigned r1 : 3;
    unsigned opcode : 3;
}; // sizeof(struct Instruction) == 7 + 3 + 3 + 3
```

```
unsigned short x; scanf("%hx", &x);
struct Instruction i = *(struct Instruction *)&x;
// 现在可以直接使用 i.opcode, i.r1, i.r2, i.imm
```

39/40

4/42

union

`union` 和 `struct` 一样，也有一系列**成员**。不同的是，`union` 是这些成员的“叠加”，这些成员共用一片内存。

- `struct` : "Cartesian product"

$$\mathcal{X} = \mathcal{T}_1 \times \cdots \times \mathcal{T}_n = \{(t_1, \dots, t_n) \mid t_1 \in \mathcal{T}_1, \dots, t_n \in \mathcal{T}_n\}$$

$$\text{sizeof}(\text{struct } X) \geq \sum_{\text{member} \in X} \text{sizeof}(\text{member}).$$
- `union` : (almost) "disjoint union"

$$\mathcal{X} = \mathcal{T}_1 \sqcup \cdots \sqcup \mathcal{T}_n = \{(t, i) \mid t_i \in \mathcal{T}_i, i \in \{1, \dots, n\}\}$$

$$\text{sizeof}(\text{union } X) \geq \max_{\text{member} \in X} \text{sizeof}(\text{member}).$$

使用 `union` 要非常小心。

40/40

5/42

CS100 Recitation 6

GKxx

目录

- C++ 的开始
- 初识 `iostream`
- `std::string`
- `std::vector`

C++ 的开始

C++ 的开始

首先，我们采用 C++17 语言标准。如果你使用 VSCode：

- `settings.json` : `code-runner.executorMap` 里 "cpp" 项, 确保编译指令中的语言标准是 `-std=c++17` 。
- `c_cpp_properties.json` : "cppStandard" 项设置为 `c++17`

调试：最简单粗暴的方法是把 `tasks.json` 和 `launch.json` 都删掉，然后调试 C++ 程序，VSCode 会自动生成。

- 调试 C++ 时要选 `g++.exe`。

4/42

Hello C++ World

```
#include <iostream>

int main() {
    std::cout << "Hello C++ World\n";
    return 0;
}
```

iostream

```
#include <iostream>

int main() {
    int a, b;
    std::cin >> a >> b;
    std::cout << a + b << '\n';
    return 0;
}
```

`std::cin` 和 `std::cout` 是定义在 `<iostream>` 中的两个对象，分别表示标准输入流和标准输出流。

6/42

1/42

iostream

`std::cin >> x` 输入一个东西给 `x`。

- `x` 可以是任何受支持的类型，例如整数、浮点数、字符、字符串。
- C++ 有能力识别 `x` 的类型并选择正确的方式输入，不需要丑陋的 `"%d"`、`"%c"` 来告诉它。
- C++ 有办法获得 `x` 的引用，因此不需要取 `x` 的地址。
- 表达式 `std::cin >> x` 执行完毕后会吧 `std::cin` 返回出来，所以可以连着写：
`std::cin >> x >> y >> z`

输出也是一样的：`std::cout << x << y << z`

7/42

iostream

`std::cout << std::endl;`

- `std::endl` 是一个“操纵符”。`std::cout << std::endl` 的含义是输出换行符，并刷新输出缓冲区。

如果你不手动刷新缓冲区，`std::cout` 自有规则在特定情况下刷新缓冲区。（C `stdout` 也是一样）

8/42

namespace std

C++ 有一套非常庞大的标准库，为了避免名字冲突，所有的名字（函数、类、类型别名、模板、全局对象等）都在一个名为 `std` 的命名空间下。

- 你可以用 `using std::cin;` 将 `cin` 引入当前作用域，那么在当前作用域内就可以省略 `std::cin` 的 `std::`。
- 你可以用 `using namespace std;` 将 `std` 中的所有名字都引入当前作用域，但这将使得命名空间形同虚设，并且重新引入了名字冲突的风险。（我个人极不推荐，并且我自己从来都不写）

我们不允许在头文件的全局作用域中使用以上任何一种 `using`。

9/42

对 C 标准库的兼容

C++ 标准库包含了 C 标准库的设施，但并不完全一样。

- 因为一些历史问题（向后兼容），C 有很多不合理之处，例如 `strchr` 接受 `const char *` 却返回 `char *`，某些本应该是函数的东西被实现为宏。
- C 缺乏 C++ 的 function overloading 等机制，因此某些设计显得繁琐。
- C++ 的编译期计算能力远远强过 C，例如 `<cmath>` 里的数学函数自 C++23 起可以在编译时计算。

C 的标准库文件 `<xxx.h>` 在 C++ 中的版本是 `<cxxx>`，并且所有名字也被引入了 `namespace std`。

```
#include <cstdio>
int main() { std::printf("Hello world\n"); }
```

* 在 C++ 中使用来自 C 的标准库文件时，请使用 `<cxxx>` 而非 `<xxx.h>`

10/42

C++ 中的 C

更合理的设计

- `bool`、`true`、`false` 是内置的，不需要额外头文件
- 逻辑运算符和关系运算符的返回值是 `bool` 而非 `int`
- `"hello"` 的类型是 `const char [6]` 而非 `char [6]`
- 字符面值 `'a'` 的类型是 `char` 而非 `int`
- 所有有潜在风险的类型转换都不允许隐式发生，不是 warning，而是 error。
- 由 `const int maxn = 100;` 声明的 `maxn` 是编译期常量，可以作为数组大小。
- `int fun()` 不接受参数，而非接受任意参数。

11/42

std::string

定义在标准库文件 `<string>` 中（不是 `<string.h>`，不是 `<cstring>`！！）

真正意义上的“字符串”。

定义并初始化一个字符串

```
std::string str = "Hello world";
// equivalent: std::string str("Hello world");
// equivalent: std::string str("Hello world"); (modern)
std::cout << str << std::endl;

std::string s1(7, 'a');
std::cout << s1 << std::endl; // aaaaaa

std::string s2 = s1; // s2 is a copy of s1
std::cout << s2 << std::endl; // aaaaaa

std::string s; // "" (empty string)
```

默认初始化一个 `std::string` 对象会得到空串，而非未定义的值！

12/42

一些问题

- `std::string` 的内存：自动管理，自动分配，需要时自动增长，自动释放
- 使用 `std::string` 时，关注字符串的内容本身，而非它的实现细节
 - 不必再考虑它的内存是怎么管理的，不必考虑末尾是不是有 `'\0'`。

14/42

std::string 的长度

s.size() 成员函数

```
std::string str{"Hello world"};
std::cout << str.size() << std::endl;
```

不是 `strlen`，更不是 `sizeof`！！！

s.empty() 成员函数

```
if (str.empty()) {
    // ...
}
```

15/42

字符串的连接

直接用 `+` 和 `+=` 就行了！

不需要考虑内存怎么分配，不需要 `strcat` 这样的函数。

```
std::string s1 = "Hello";
std::string s2 = "world";
std::string s3 = s1 + ' ' + s2; // "Hello world"
s1 += s2; // s1 becomes "Helloworld"
s2 += "C++string"; // s2 becomes "worldC++string"
```

16/42

字符串的连接

+ 两侧至少有一个得是 `std::string` 对象。

```
const char *old_bad_ugly_c_style_string = "hello";
std::string s = old_bad_ugly_c_style_string + "aaaaa"; // Error
```

下面这个是否合法？

```
std::string hello{"hello"};
std::string s = hello + "world" + "C++";
```

17/42

字符串的连接

+ 两侧至少有一个得是 `std::string` 对象。

```
const char *old_bad_ugly_c_style_string = "hello";
std::string s = old_bad_ugly_c_style_string + "aaaaa"; // Error
```

下面这个是否合法？

```
std::string hello{"hello"};
std::string s = hello + "world" + "C++";
```

合法：+ 是左结合的，(hello + "world") 是一个 `std::string` 对象。

18/42

使用 +=

`s1 = s1 + s2` 会先为 `s1 + s2` 构造一个临时对象，必然要拷贝一遍 `s1` 的内容。

而 `s1 += s2` 是直接 在 `s1` 后面连接 `s2`。

试一试：

```
std::string result;
for (int i = 0; i != n; ++i)
    result += 'a';

std::string result;
for (int i = 0; i != n; ++i)
    result = result + 'a';
```

19/42

字符串比较（字典序）

直接用 `<`, `<=`, `>`, `>=`, `==`, `!=`，不需要循环，不需要其它函数！

字符串的拷贝赋值

直接用 `=` 就行

```
std::string s1{"Hello"};
std::string s2{"world"};
s2 = s1; // s2 is a copy of s1
s1 += 'a'; // s2 is still "Hello"
```

`std::string` 的 `=` 会拷贝这个字符串的内容。

20/42

遍历字符串：基于范围的 for 语句

例：输出所有大写字母（`std::isupper` 在 `<cctype>` 里）

```
for (char c : s)
    if (std::isupper(c))
        std::cout << c;
std::cout << std::endl;
```

等价的方法：使用下标，但不够 modern，比较啰嗦。

```
for (std::size_t i = 0; i != s.size(); ++i)
    if (std::isupper(s[i]))
        std::cout << s[i];
std::cout << std::endl;
```

基于范围的 `for` 语句更好，更清晰，更简洁，更通用，更现代，更推荐。

- 你的意图是“遍历该字符串”，而非“创建一个整数并使它从 0 变化到 `s.size()`”。

21/42

字符串的 IO

`std::cin >> s` 可以输入字符串，`std::cout << s` 可以输出字符串。

- `std::cin >> s` 跳不跳过空白？是读到空白结束还是读一行结束？可以自己试试

`std::getline(std::cin, s)`：从当前位置开始读一行，换行符会读掉，但不会存进来

- 假如前一次输入恰好停在换行符处，`getline` 就会读进一个空串。

22/42

std::string 接口完整列表

https://en.cppreference.com/w/cpp/string/basic_string

23/42

std::vector

定义在标准库文件 `<vector>` 中

真正好用的“动态数组”

24/42

创建一个 std::vector 对象

`std::vector` 是一个类模板，只有给出了模板参数之后才成为一个真正的类型。

```
std::vector<int> vi; // An empty vector of ints
std::vector<std::string> vs; // An empty vector of strings
std::vector<double> vd; // An empty vector of doubles
```

对于两个不同的类型 `T` 和 `U`，`std::vector<T>` 和 `std::vector<U>` 是不同的类型。

`std::vector` 不是类型。

25/42

创建一个 std::vector 对象

```
std::vector<int> v{2, 3, 5, 7}; // A vector of ints,
// whose elements are {2, 3, 5, 7}.
std::vector<int> v2 = {2, 3, 5, 7}; // Equivalent to ↑

std::vector<std::string> vs{"hello", "world"}; // A vector of strings,
// whose elements are {"hello", "world"}.
std::vector<std::string> vs2 = {"hello", "world"}; // Equivalent to ↑

std::vector<int> v3(10); // A vector of ten ints, all initialized to 0.
std::vector<int> v4(10, 42); // A vector of ten ints, all initialized to 42.
```

`vector<T> v(n)` 这种构造方式会将 `n` 个元素都值初始化 (value-initialization) (类似于 C 中的“空初始化”)，而不是得到一串 indeterminate value。

- 对于类类型来说，“值初始化”几乎就是调用默认构造函数进行初始化。
- 标准库和我们同样讨厌未初始化的值！

26/42

创建一个 `std::vector` 对象

```
std::vector<int> v{2, 3, 5, 7};
std::vector<int> v2 = v; // v2 is a copy of v
std::vector<int> v3(v); // Equivalent
std::vector<int> v4{v}; // Equivalent
```

和 `std::string` 一样，`std::vector` 可以直接拷贝，不需要手动写循环逐元素拷贝。

```
std::vector<std::vector<int>> v;
```

“二维 `vector`”，也就是“`vector` of `vector`”，当然也是可以的。

`v.back()` 和 `v.front()`

分别获得最后一个元素、第一个元素的引用。

“引用”意味着你可以通过这两个成员函数修改它们：

```
v.front() = 42;
++v.back();
```

`v.back()`，`v.front()`，`v.pop_back()` 在 `v` 为空的情况下是 undefined behavior。

C++17 CTAD

Class Template Argument Deduction：只要你给出了足够的信息，编译器可以自动推导元素的类型。

```
std::vector v{2, 3, 5, 7}; // vector<int>
std::vector v2{3.14, 6.28}; // vector<double>
std::vector v3{10, 42}; // vector<int>
std::vector v4{10}; // Error: cannot deduce template argument type
```

- 怎样算是给出了“足够的信息”？你品。（具体规则细节略去）

基于范围的 `for` 语句

遍历一个 `std::vector`，同样可以使用基于范围的 `for` 语句：

```
std::vector<int> vi = some_values();
for (int x : vi)
    std::cout << x << std::endl;
std::vector<std::string> vs = some_strings();
for (const std::string &s : vs) // use reference-to-const to avoid copying
    std::cout << s << std::endl;
```

练习：使用基于范围的 `for` 语句，将一个 `vector<string>` 中的每个字符串的大写字母打印出来。

`std::vector` 的大小

`v.size()` 和 `v.empty()`

```
std::vector v{2, 3, 5, 7};
std::cout << v.size() << std::endl;
if (v.empty()) {
    // ...
}
```

清空 `std::vector`

`v.clear()`。不要写愚蠢的 `while (!v.empty()) v.pop_back();`

基于范围的 `for` 语句

练习：使用基于范围的 `for` 语句，将一个 `vector<string>` 中的每个字符串的大写字母打印出来。

```
for (const std::string &s : vs) {
    for (char c : s)
        if (std::isupper(c))
            std::cout << c;
    std::cout << std::endl;
}
```

向 `std::vector` 添加元素

`v.push_back(x)` 将元素 `x` 添加到 `v` 的末尾

```
int n;
std::cin >> n;
std::vector<int> v;
for (int i = 0; i != n; ++i) {
    int x;
    std::cin >> x;
    v.push_back(x);
}
```

使用下标访问

可以使用 `v[i]` 来获得第 `i` 个元素

- `i` 的有效范围是 $[0, N)$ ，其中 $N = v.size()$
- 越界访问是未定义行为，并且通常是严重的运行时错误。
- `std::vector` 的下标运算符 `v[i]` 并不检查越界，目的是为了效率。
 - 事实上标准库容器的大多数操作（比如刚才的 `front`，`back`，`pop_back`）都没有边界检查，为了效率。
- 一种检查越界的下标是 `v.at(i)`，它会在越界时抛出 `std::out_of_range` 异常。
 - 不妨自己试一试。

删除 `std::vector` 最后一个元素

`v.pop_back()`

练习：将末尾的偶数删掉，直到末尾是奇数为止

```
while (!v.empty() && v.back() % 2 == 0)
    v.pop_back();
```

`v.back()`：获得末尾元素的引用（这意味着什么？）

接口的统一性

事实上 `std::string` 也有 `.at()`，`.front()`，`.back()`，`.push_back(x)`，`.pop_back()`，`.clear()` 等函数。C++ 标准库的各种设施是讲究统一性的。

完整列表

练习：实现 Python 的 `rstrip` 函数，接受一个 `std::string`，返回它删去末尾的连续空白后的结果。

接口的统一性

练习：实现 Python 的 `rstrip` 函数，接受一个 `std::string`，返回它删去末尾的连续空白后的结果。

```
std::string rstrip(std::string str) {
    while (!str.empty() && std::isspace(str.back()))
        str.pop_back();
    return str;
}
```

在 C++17 下，这个 `return str;` 是会产生拷贝的，不必担心。

37/42

std::vector 动态增长带来的影响

我们已经看到，改变 `vector` 的大小可能会导致它所保存的元素“搬家”，这会使得所有指针、引用、迭代器失效。

- 最直接的影响：下面的代码是 undefined behavior，因为基于范围的 `for` 语句本质上依赖于迭代器。

```
for (int i : vec)
    if (i % 2 == 0)
        vec.push_back(i + 1);
```

不要在用基于范围的 `for` 语句遍历容器的同时改变容器的大小！

42/42

CS100 Recitation 7

GKxx

1/41

Contents

- 引用
- 左值和右值
- `new` 和 `delete`

2/41

引用 (Reference)

3/41

动机

练习：使用基于范围的 `for` 语句遍历一个字符串，将大写改为小写

std::vector 的增长策略

考虑像这样连续 `push_back` `n` 次得到一个 `vector` 的代码：

```
std::vector<int> v;
for (int i = 0; i != n; ++i)
    v.push_back(i);
```

`vector` 是如何做到快速增长的？

38/42

std::vector 的增长策略

假设现在有一片动态分配的内存，长度为 `i`。

当第 `i+1` 个元素到来时，朴素做法：

1. 分配一片长度为 `i+1` 的内存
2. 将原有的 `i` 个元素拷贝过来
3. 将新的元素放在后面
4. 释放原来的那片内存

但这需要拷贝 `i` 个元素。`n` 次 `push_back` 总共就需要 $\sum_{i=0}^{n-1} i = O(n^2)$ 次拷贝！

39/42

std::vector 的增长策略

假设现在有一片动态分配的内存，长度为 `i`。

当第 `i+1` 个元素到来时，

1. 分配一片长度为 `2*i` 的内存
2. 将原有的 `i` 个元素拷贝过来
3. 将新的元素放在后面
4. 释放原来的那片内存

而当第 `i+2`, `i+3`, ..., `2*i` 个元素到来时，我们不需要分配新的内存，也不需要拷贝任何对象！

40/42

std::vector 的增长策略

$0 \rightarrow 1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow \dots$

假设 $n = 2^m$ ，那么总的拷贝次数就是 $\sum_{i=0}^{m-1} 2^i = O(n)$ ，平均（“均摊”）一次

`push_back` 的耗时是 $O(1)$ （常数），可以接受。

使用 `v.capacity()` 来获得它目前所分配的内存的实际容量，看看是不是真的这样。

- 注意：这仅仅是一种可能的策略，标准并未对此有所规定。

* Lecture 8 `read_string` 也是这个策略。

41/42

4/41

动机

练习：使用基于范围的 `for` 语句遍历一个字符串，将大写改为小写

```
for (char c : str)
    c = std::tolower(c);
```

这样写是**不行**的：`for (char c : str)` 相当于让 `char c` 依次成为 `str` 中的每个字符的拷贝，在 `c` 上修改不会影响 `str` 的内容。

如同

```
for (std::size_t i = 0; i != str.size(); ++i) {
    char c = str[i];
    c = std::tolower(c);
}
```

5/41

动机

练习：使用基于范围的 `for` 语句遍历一个字符串，将大写改为小写

```
for (char &c : str)
    c = std::tolower(c);
```

`char &c` 定义了一个**绑定到** `char` 的引用，我们让 `c` 依次绑定到 `str` 中的每个字符。

如同

```
for (std::size_t i = 0; i != str.size(); ++i) {
    char &c = str[i];
    c = std::tolower(c); // effectively str[i] = std::tolower(str[i]);
}
```

6/41

引用即别名

```
int ival = 42;
int &iref = ival;
++iref;           // ival becomes 43
iref = 50;        // ival becomes 50
std::cout << ival << std::endl; // 50
std::cout << iref << std::endl; // 50
int &iref2 = iref; // Equivalent to int &iref2 = ival;
                // iref2 also bounds to ival.
```

可以写 `Type &r`，也可以 `Type& r`

- `r` 的类型是 `Type &`。
- 但 `Type& r1, r2, r3`；只有 `r1` 是引用。
- 如果想定义多个引用，需要 `Type &r1, &r2, &r3`；

引用必须初始化（即在定义时就指明它绑定到谁），并且这个绑定关系不可修改。

7/41

引用绑定到的类型必须是对象类型

普通的变量、数组、指针等都是对象

但“引用”本身只是一个傀儡，是一个对象的别名。

- 可以定义绑定到数组的引用：

```
int a[10];
int (&ar)[10] = a;
ar[0] = 42; // a[0] = 42
```

- 不能定义绑定到引用的引用：

```
int i = 42;
int &r = i;
int &&rr = r; // No such thing!
```

- 可以定义绑定到指针的引用：

```
int *p;
int *&pr = p;
pr = &ival; // p = &ival;
```

- 同样地，指针指向的类型、数组的元素类型也不能是引用。

```
int *&pr = &r; // No such thing!
int &ar[N]; // No such thing!
```

8/41

引用即别名

一个引用被声明（并初始化）之后，使用它就是在使用它所绑定到的对象。

```
std::string s;
std::string &r = s;
r.push_back('a'); // 如同 s.push_back('a');
r = "hello";      // 如同 s = "hello";
std::cout << r << '\n'; // 如同 std::cout << s << '\n';
std::string &r2 = r; // 如同 std::string &r2 = s; , r2 也被绑定到 s
```

- 初始化和赋值**对于引用类型有巨大的区别：初始化是设定它“绑定到谁”，而给一个引用赋值就是在给它所绑定到的对象赋值。

9/41

传引用参数 (pass-by-reference)

可以定义绑定到数组的引用：

```
void print_array(const int (&array)[10]) { // 只读，所以加 const
    for (int x : array) // 基于范围的 for 语句可以用来遍历数组！
        std::cout << x << ' ';
    std::cout << std::endl;
}
int a[10], ival = 42, b[20];
print_array(a); // Connect
print_array(&ival); // Error!
print_array(b); // Error!
```

这个 `array` 真的是（绑定到了）一个数组，而不是指向数组首元素的指针。

- `array` 只能绑定到 `int[10]`（可带 `const`），其它类型的参数都会导致 `Error`。

10/41

传引用参数 (pass-by-reference)

定义一个函数，接受一个字符串，将其中的大写字母改为小写字母

```
void to_lower(std::string &str) {
    for (char &c : str)
        c = std::tolower(c);
}
```

```
std::string hello = "Hello";
to_lower(hello); // std::string &str = hello;
```

- 还可以怎样设计？

11/41

传引用参数 (pass-by-reference)

定义一个函数，接受一个字符串，输出其中的大写字母。

```
void print_upper(std::string str) {
    for (char c : str)
        if (std::isupper(c))
            std::cout << c;
    std::cout << std::endl;
}
```

12/41

传引用参数 (pass-by-reference)

```
void print_upper(std::string str) {
    for (char c : str)
        if (std::isupper(c))
            std::cout << c;
    std::cout << std::endl;
}
std::string s = some_very_long_string();
print_upper(s);
```

传参的过程中发生了形如 `std::string str = s`；的**拷贝初始化**。如果 `s` 很长，这将非常耗时。

13/41

传引用参数 (pass-by-reference)

以**引用**的方式传参，避免拷贝：

```
void print_upper(std::string &str) {
    for (char c : str)
        if (std::isupper(c))
            std::cout << c;
    std::cout << std::endl;
}
std::string s = some_very_long_string();
print_upper(s); // std::string &str = s;
```

14/41

传引用参数 (pass-by-reference)

以引用的方式传参，避免拷贝：

```
void print_upper(std::string &str) {
    for (char c : str)
        if (std::isupper(c))
            std::cout << c;
    std::cout << std::endl;
}
std::string s = some_very_long_string();
print_upper(s); // std::string &str = s;
```

但这样有问题：

```
const std::string hello = "Hello";
print_upper(hello); // Error
```

- 这不合理，因为 `print_upper` 明明是个只读的操作。

传递常量引用 (reference-to- `const`)

最好的做法：传递 `reference-to-const`

```
void print_upper(const std::string &str) {
    for (char c : str)
        if (std::isupper(c))
            std::cout << c;
    std::cout << std::endl;
}
```

- 既避免了拷贝，又允许传递 `const` 对象，又能防止错误的修改。

左值 (lvalue) 和右值 (rvalue)

左值和右值

一个表达式在被使用时，有时我们使用的是它代表的**对象**，有时我们仅仅是使用了那个对象的**值**。

- `str[i] = ch` 中，我们使用的是表达式 `str[i]` 所代表的**对象**
- `ch = str[i]` 中，我们使用的是表达式 `str[i]` 所代表的对象的**值**。

左值和右值

一个表达式在被使用时，有时我们使用的是它代表的**对象**，有时我们仅仅是使用了那个对象的**值**。

- `str[i] = ch` 中，我们使用的是表达式 `str[i]` 所代表的**对象**
- `ch = str[i]` 中，我们使用的是表达式 `str[i]` 所代表的对象的**值**。

一个表达式本身带有**值类别** (value category) 的属性：它要么是左值，要么是右值

- 左值：它代表了一个实际的对象
- 右值：它仅仅代表一个值

左值和右值

一个表达式本身带有**值类别** (value category) 的属性：它要么是左值，要么是右值

- 左值：它代表了一个实际的对象
- 右值：它仅仅代表一个值

哪些表达式能代表一个实际的对象？

哪些表达式产生的仅仅是一个值？

左值和右值

一个表达式本身带有**值类别** (value category) 的属性：它要么是左值，要么是右值

- 左值：它代表了一个实际的对象
- 右值：它仅仅代表一个值

哪些表达式能代表一个实际的对象？

- `*addr`, `a[i]` 等等

哪些表达式产生的仅仅是一个值？

- `a + b`, `&val`, `cond1 && cond2` 等等。

* “左值”和“右值”这两个名字是怎么来的？

左值和右值

在 C 中，左值可以放在赋值语句的左侧，右值不能。

但在 C++ 中，二者的区别远没有这么简单。

目前已经见过的返回左值的表达式：`*p`, `a[i]`

特别地：在 C++ 中，前置递增/递减运算符返回**左值**，`++i = 42` 是合法的。

赋值表达式返回左值：`a = b` 的返回值是 `a` 这个对象。

- 试着解释表达式 `a = b = c` ？

左值和右值

在 C 中，左值可以放在赋值语句的左侧，右值不能。

但在 C++ 中，二者的区别远没有这么简单。

目前已经见过的返回左值的表达式：`*p`, `a[i]`

特别地：在 C++ 中，前置递增/递减运算符返回**左值**，`++i = 42` 是合法的。

赋值表达式返回左值：`a = b` 的返回值是 `a` 这个对象（的引用）。

- 赋值运算符**右结合**，表达式 `a = b = c` 等价于 `a = (b = c)`：先执行 `b = c`，然后相当于 `a = b`。

左值和右值

右值仅仅代表一个值，不代表一个实际的对象。常见的右值有**表达式执行产生的临时对象**和**字面值**。

```
std::string fun(); // a function that returns a std::string object
std::string a = fun();
```

- 函数调用 `fun()` 生成的临时对象是**右值**。
- 特别的例外：**字符串字面值** `"hello"` 是**左值**，它是长期存在于内存中的对象。
 - 相比之下，整数字面值 `42` 仅仅产生一个临时对象，是右值。

左值和右值

一种常见的右值：通过类型转换生成的临时对象

```
std::string &r1 = std::string("hello"); // Error
std::string &r2 = "hello"; // Error. This is equivalent to ↑
```

Functional-style cast expression: `Type(args...)`，会生成一个 `Type` 类型的临时对象。

- 对于类类型，这会调用一个适当的构造函数（或者类型转换运算符）
 - 例如 `std::string(10, 'c')`，`std::string("hello")`
- 对于内置类型，就是一个普通的拷贝或者类型转换
 - `int(x)` 会生成一个 `int` 类型的临时对象，其值由 `x` 初始化。

Reference-to- const

特殊规则：常量引用可以绑定到右值：

```
const int &cref = 42; // Correct
int fun();
const int &cref2 = fun(); // Correct
int &ref = fun(); // Error
```

当一个常量引用被绑定到右值时，实际上就是让它绑定到了一个临时对象。

- 这是合理的，反正你也不能通过常量引用修改那个对象的值

引用只能绑定到左值

```
int ival = 42;
int &iref = 42; // Error
int &iref2 = ival; // Correct
int &iref3 = ival + 42; // Error
int fun();
int &iref4 = fun(); // Error
```

C++11 引入了所谓的“右值引用”，我们在介绍移动的时候再讲。一般来说，“引用”指的是“左值引用”。

Pass-by-reference-to- const

```
void print_upper(std::string &str) {
    for (char c : str)
        if (std::isupper(c))
            std::cout << c;
    std::cout << std::endl;
}
print_upper("Hello"); // Error
```

当我们传递 “Hello” 给 `std::string` 参数时，实际上发生了一个由 `const char [6]` 到 `std::string` 的隐式转换，这个隐式转换产生右值，无法被 `std::string&` 绑定。

引用只能绑定到左值

```
int arr[10];
int &subscript(int i) { // function returning int&
    return arr[i];
}
subscript(3) = 42; // Correct.
int &ref = subscript(7); // Correct. ref bounds to arr[7]
```

Pass-by-reference-to- const

将参数声明为常量引用，既可以避免拷贝，又可以允许传递右值

```
void print_upper(const std::string &str) {
    for (char c : str)
        if (std::isupper(c))
            std::cout << c;
    std::cout << std::endl;
}
std::string s = some_very_long_string();
print_upper(s); // const std::string &str = s;
print_upper("Hello"); // const std::string &str = "Hello";, Correct
```

Reference-to- const

类似于“指向常量的指针”（即带有“底层 `const`”的指针），我们也有“绑定到常量的引用”

```
int ival = 42;
const int cival = 42;
const int &cival = cival; // Correct
const int &cref2 = ival; // Also correct.
int &ref = cival; // Error: Casting-away low-level const is not allowed.
int &ref2 = cref; // Error: Casting-away low-level const is not allowed.
int &ref3 = cref2; // Error: Even though cref2 bounds to non-const ('ival'),
// this is still casting-away low-level const.
```

一个 `reference-to- const` 自认为自己绑定到 `const` 对象，所以不允许通过它修改它所绑定的对象的值，也不能让一个不带 `const` 的引用绑定到它。（不允许“去除底层 `const`”）

Pass-by-reference-to- const

将参数声明为常量引用，既可以避免拷贝，又可以允许传递右值，也可以传递常量对象，也可以防止你不小心修改了它。

在 C++ 中声明函数的参数时，尽可能使用常量引用（如果你不需要修改它）。

（如果仅仅是 `int` 或者指针这样的内置类型，可以不需要常量引用）

Reference-to- const

指针既可以带顶层 `const`（本身是常量），也可以带底层 `const`（指向的东西是常量），但引用不谈“顶层 `const`”。

- 即，只有“绑定到常量的引用”。引用本身不是对象，不谈是否带 `const`。
- 从另一个角度讲，引用本身一定带有“顶层 `const`”，因为绑定关系不能修改。
- 在不引起歧义的情况下，通常用常量引用这个词来代表“绑定到常量的引用”。

真正的“值类别”

（语言律师需要掌握）

C++ 中的表达式依值类别被划分为如下三种：

英文	中文	has identity?	can be moved from?
lvalue	左值	yes	no
xvalue (expired value)	亡值	yes	yes
prvalue (pure rvalue)	纯右值	no	ves

`lvalue + xvalue = glvalue`（广义左值），`xvalue + prvalue = rvalue`（右值）

- 所以实际上“左值是实际的对象”是不严谨的，右值也可能是实际的对象（`xvalue`）。之后讲移动的时候我们会见到一个典型的 `xvalue`。

new 和 delete (初步)

new 表达式

动态分配内存，并构造对象

```
int *pi1 = new int; // 动态创建一个默认初始化的 int
int *pi2 = new int(); // 动态创建一个值初始化的 int
int *pi3 = new int{}; // 同上，但是更 modern
int *pi4 = new int{42}; // 动态创建一个 int，并初始化为 42
int *pi5 = new int{42}; // 同上，但是更 modern
```

对于内置类型：

- **默认初始化** (default-initialization)：就是未初始化，具有未定义的值
- **值初始化** (value-initialization)：类似于 C 中的“空初始化”，是各种零。

new[] 表达式

动态分配“数组”，并构造对象

```
int *pai1 = new int[n]; // 动态创建了 n 个 int，默认初始化
int *pai2 = new int[n](); // 动态创建了 n 个 int，值初始化
int *pai3 = new int[n]{}; // 动态创建了 n 个 int，值初始化
int *pai4 = new int[n]{2, 3, 5}; // 动态创建了 n 个 int，前三个元素初始化为 2,3,5
// 其余元素都被值初始化（为零）
// 如果 n<3，抛出 std::bad_array_new_length 异常
```

对于内置类型：

- **默认初始化** (default-initialization)：就是未初始化，具有未定义的值
- **值初始化** (value-initialization)：类似于 C 中的“空初始化”，是各种零。

delete 和 delete[] 表达式

销毁动态创建的对象，并释放其内存

```
int *p = new int{42};
delete p;
int *a = new int[n];
delete[] a;
```

- new 必须对应 delete，new[] 必须对应 delete[]，否则是 **undefined behavior**
- 忘记 delete：内存泄漏

——对应，不得混用

违反下列规则的一律是 undefined behavior:

- delete ptr 中的 ptr 必须等于某个先前由 new 返回的地址
- delete[] ptr 中的 ptr 必须等于某个先前由 new[] 返回的地址
- free(ptr) 中的 ptr 必须等于某个先前由 malloc, calloc, realloc 或 aligned_alloc 返回的地址。

new/delete vs malloc/free

C++ 的对象模型比 C 复杂得多，而 new/delete 也比 malloc/free 做了更多的事：

- new/new[] 表达式会**先分配内存，然后构造对象**。对于类类型的对象，它可能会调用一个合适的**构造函数**。
- delete/delete[] 表达式会**先销毁对象，然后释放内存**。对于类类型的对象，它会调用析构函数。

在 C++ 中，非必要不手动管理内存

- 当你需要创建“一列数”、“一列对象”，或者“一张表”、“一个集合”时，**优先考虑标准库容器等设施**，例如 std::string, std::vector, std::deque (双端队列), std::list / std::forward_list (链表), std::map / std::set (红黑树), std::unordered_map / std::unordered_set (哈希表)
- 当你需要动态创建单个对象时，应该优先考虑**智能指针** (std::shared_ptr, std::weak_ptr, std::unique_ptr)
- 只有在特殊情况下（例如手搓一个标准库没有的数据结构，并且对效率有极高的要求），使用 new/delete 来管理动态内存
- 当你对于内存分配本身也有特殊的要求时，才需要使用 C 的内存分配/释放函数，但通常也是用它们来**定制 new 和 delete**

CS100 Recitation 8

GKxx

class 初步

一个简单的 class

class 除了可以有数据成员，还可以有成员函数等。

```
class Student {
    std::string name;
    std::string id;
    int entranceYear;
    void printInfo() const {
        std::cout << "I am " << name << ", id " << id
            << ", entrance year: " << entranceYear << std::endl;
    }
    bool isGraduated(int year) const {
        return year - entranceYear >= 4;
    }
};
```

访问限制

```
class Student {
private:
    std::string name;
    std::string id;
    int entranceYear;
public:
    void printInfo() const {
        std::cout << "I am " << name << ", id " << id
            << ", entrance year: " << entranceYear << std::endl;
    }
    bool isGraduated(int year) const {
        return year - entranceYear >= 4;
    }
};
```

- private：外部代码不能访问，只有类内和 friend 代码可以访问。
- public：所有代码都可以访问。

访问限制

```
class Student {
    std::string name;
    std::string id;
    int entranceYear;
public:
    void printInfo() const;
    bool isGraduated(int year) const;
};
```

一个访问限制说明符的作用范围是从当前位置开始，一直到下一个访问限制说明符之前（或者到类的定义结束）。
开头有一段没有写访问限制的成员？

- 如果是 class，它们是 private。
- 如果是 struct，它们是 public。

- C++ struct 和 class 仅有两个区别，这里是区别之一。
- 为何需要将数据成员“藏起来”？《Effective C++》条款 22。

this 指针

访问一个普通成员的方法是 a.mem，其中 a 是该类类型的一个对象。

- 成员是属于一个对象的：每个学生都有一个姓名、学号、入学年份。你需要指明“谁的”姓名/学号...

```
Student s = someValue(); // 假设它具有一定的值
s.printInfo(); // 调用它的 printInfo() 输出相关信息
if (s.isGraduated(2023)) {
    // ...
}
```

注：我们现在暂时只讨论 non-static 成员，以下不再强调。

this 指针

```
class Student {
    // ...
public:
    bool isGraduated(int year) const;
};
Student s = someValue();
if (s.isGraduated(2023))
    // ...
```

isGraduated 函数有几个参数？

this 指针

```
class Student {
    // ...
public:
    bool isGraduated(int year) const;
};
Student s = someValue();
if (s.isGraduated(2023))
    // ...
```

isGraduated 函数有几个参数？

- 看似是一个，其实是两个：s 也是调用这个函数时必须知道的信息！

this 指针

```
class Student {
public:
    bool isGraduated(int year) const {
        return year - entranceYear >= 4;
    }
};
Student s = someValue();
if (s.isGraduated(2023))
    // ...
```

左边的代码就如同：

```
bool isGraduated(const Student *this,
    int year) {
    return year - this->entranceYear >=4;
}
Student s = someValue();
if (isGraduated(&s, 2023))
    // ...
```

this 指针

在成员函数内部，正有一个名为 this 的指针：它具有 X * 或者 const X * 类型（其中 X 是这个类的名字）。
在成员函数内部对任何成员 mem 的访问，实际上都是 this->mem。
你也可以显式地写 this->mem。

```
class Student {
public:
    bool isGraduated(int year) const {
        return year - this->entranceYear >= 4;
    }
};
```

许多语言都有类似的设施：Python 里有 self。（C++23 当然也可以有 self）

const 成员函数

在参数列表后面、函数体的 { 前面标一个 const

- 左边的代码如同：

```
class Student {
public:
    bool isGraduated(int year) const;
    void setID(const std::string &);
};

const Student s = someValue();
if (s.isGraduated(2023)) // OK
    // ...
s.setID("2024533123"); // Error
```

```
bool isGraduated(const Student *this,
    int year);
void setID(Student *this,
    const std::string &);
const Student s = someValue();
const Student s = someValue();
if (isGraduated(&s, 2023)) // OK
    // ...
setID(&s, "2024533123"); // Error
```

这个 const 表明 this 的类型是 const Student *。如果没有，则是 Student *。

const 成员函数

- 规则：
1. 如果 ptr 的类型是 const T *，T 具有类型为 U 的数据成员 member，则 ptr->member 的类型是 const U。
 - const ness 会在成员访问时传递下去。
 2. 在类 X 的 const 成员函数中，this 的类型是 const X *。
 - 在 non-const 成员函数中，this 的类型是 X *。
 3. 对成员的访问 member 实际上都是 this->member，这里的 member。
 - 这里的 member 既可以是数据成员，也可以是成员函数。
 4. 不允许去除底层 const。

const 成员函数

- 一些推论：
1. 在 const 对象上（包括用 reference-to-const 时），只能调用 const 成员函数。
 2. 在 const 成员函数内部，不允许修改自身的数据成员，不能调用自身的 non-const 成员函数。
 - 对于类类型的数据成员，也不能调用它的 non-const 成员函数。

[Best practice] 如果一个成员函数逻辑上不修改这个对象的状态，那么它就应该 是 const 成员函数。

《Effective C++》条款 3：尽可能使用 const。（下次讲）

控制数据成员的读写

方案一： Getters and setters

```
class Student {
    std::string name;
    std::string id;
    int entranceYear;
public:
    const std::string &getName() const { return name; }
    const std::string &getID() const { return id; }
    int getEntranceYear() const { return entranceYear; }
    void setName(const std::string &newName) { name = newName; }
    void setID(const std::string &newID) { id = newID; }
    void setEntranceYear(int year) { entranceYear = year; }
};
```

* 为什么要控制数据成员的读写？这和直接让数据成员变成 public 有什么区别？

14/34

控制数据成员的读写

和直接让数据成员变成 public 有什么区别？

- 你可以修改，但是我得知道你在改，而且我不能让你胡改。

```
class Student {
    std::string name;
    std::string id;
    int entranceYear;
public:
    void setID(const std::string &newID) {
        id = newID;
        entranceYear = std::stoi(id.substr(0, 4)); // 改 id 的同时更新 entranceYear
    }
    void setEntranceYear(int year) {
        if (year < 2013) // 制止不正确的改动
            throw std::invalid_argument{"Entrance year impossible!"};
        entranceYear = year;
    }
};
```

15/34

控制数据成员的读写

和直接让数据成员变成 public 有什么区别？

- 我可以修改实现细节，不会影响用户代码。

```
class Student {
    std::string name;
    std::string id; // 我不想存 entranceYear 了
                  // 没关系，用户本来也不能直接访问 entranceYear
public:
    int getEntranceYear() const { return std::stoi(id.substr(0, 4)); }
};
```

16/34

控制数据成员的读写

方案二：读/写重载

```
class Point2d {
    double mX;
    double mY;
public:
    double x() const { return mX; }
    double y() const { return mY; }
    void x(double newX) { mX = newX; }
    void y(double newY) { mY = newY; }
};
```

```
Point2d p = /* ... */;
p.x(3.14); // 将 x 改为 3.14
std::cout << p.x() << '\n'; // 访问 x
```

17/34

控制数据成员的读写

方案三： const 和 non-const 重载

- 实际上没有完全控制住“写”操作。
- 容器的下标运算符普遍采用这种方式。

```
class Point2d {
    double mX;
    double mY;
public:
    double x() const { return mX; }
    double y() const { return mY; }
    double &x() { return mX; }
    double &y() { return mY; }
};
```

```
Point2d p = /* ... */;
p.x() = 3.14;
```

18/34

构造函数

构造函数 (constructors, ctors) 负责对象的初始化方式。

- 构造函数往往是重载 (overloaded) 的，因为一个对象很可能具有多种合理的初始化方式。

```
class Student {
    std::string name;
    std::string id;
    int entranceYear;
public:
    Student(const std::string &name_, const std::string &id_, int ey)
        : name(name_), id(id_), entranceYear(ey) {}
    Student(const std::string &name_, const std::string &id_)
        : name(name_), id(id_), entranceYear(std::stoi(id_.substr(0, 4))) {}
};
Student a("Alice", "2020123123", 2020);
Student b("Bob", "2020123124"); // entranceYear = 2020
```

19/34

构造函数

```
class Student {
    std::string name;
    std::string id;
    int entranceYear;
public:
    Student(const std::string &name_, const std::string &id_)
        : name(name_), id(id_), entranceYear(std::stoi(id_.substr(0, 4))) {}
};
```

- 构造函数的函数名就是类名本身： `Student`
- 构造函数没有返回值类型，可以含有 `return`；但不能返回一个值。
- 上面这个构造函数的函数体是空的： `{}`

20/34

构造函数初始值列表

构造函数负责这个对象的初始化，包括所有成员的初始化。

一切成员的初始化过程都在进入函数体之前结束，它们的初始化方式（部分是）由构造函数初始值列表 (constructor initializer list) 决定：

```
class Student {
    // ...
public:
    Student(const std::string &name_, const std::string &id_)
        : name(name_), id(id_), entranceYear(std::stoi(id_.substr(0, 4))) {}
};
```

构造函数初始值列表由冒号 `:` 开头，依次给出各个数据成员的初始化器，用 `,` 隔开。

21/34

成员的初始化顺序

数据成员按照它们在类内声明的顺序进行初始化，而非它们在构造函数初始值列表中出现的顺序。

- 如果你的构造函数初始值列表中的成员的顺序和它们在类内的声明顺序不同，编译器会以 warning 的方式提醒你。

典型的错误：初始化 `entranceYear` 时用到了成员 `id`，而此时 `id` 还未初始化！

```
class Student {
    std::string name;
    int entranceYear;
    std::string id;
public:
    Student(const std::string &name_, const std::string &id_)
        : name(name_), id(id_), entranceYear(std::stoi(id.substr(0, 4))) {}
};
```

22/34

构造函数初始值列表

数据成员按照它们在类内声明的顺序进行初始化，而非它们在构造函数初始值列表中出现的顺序。

- 如果你的构造函数初始值列表中的成员的顺序和它们在类内的声明顺序不同，编译器会以 warning 的方式提醒你。
- 未在初始值列表中出现的成员：
 - 如果有类内初始值（稍后再说），则按照类内初始值初始化；
 - 否则，默认初始化。

默认初始化对于内置类型来说就是不初始化，但是对于类类型呢？

23/34

构造函数初始值列表

下面是典型的“先默认初始化，后赋值”：

```
class Student {
// ...
public:
    Student(const std::string &name_, const std::string &id_) {
        name = name_;
        id = id_;
        entranceYear = std::stoi(id_.substr(0, 4));
    }
};
```

“先默认初始化，后赋值”有什么坏处？

你的类需要一个默认构造函数吗？

如果一个类没有 user-declared 的构造函数，编译器会试图帮你合成一个默认构造函数。

```
struct X {}; // 什么构造函数都没写
X x; // OK: 调用了编译器合成的默认构造函数
```

编译器合成的默认构造函数的行为非常简单：逐个成员地进行默认初始化

- 如果有成员有类内初始值，则使用类内初始值。
- 如果有成员没有类内初始值，但又无法默认初始化，则编译器会放弃合成这个默认构造函数。

构造函数初始值列表

“先默认初始化，后赋值”有什么坏处？

- 不是所有类型都能默认初始化，不是所有类型都能赋值。（有哪些反例？）

你的类需要一个默认构造函数吗？

如果一个类有至少一个 user-declared 的构造函数，但没有默认构造函数，则编译器**不会**帮你合成默认构造函数。

- 在它看来，这个类的所有合理的初始化行为就是你给出的那几个构造函数，
- 因此不应该再画蛇添足地定义一个默认行为。

你可以通过 `= default;` 来显式地要求编译器合成一个具有默认行为的默认构造函数：

```
class Student {
public:
    Student(const std::string &name_, const std::string &id_, int ey)
        : name(name_), id(id_), entranceYear(ey) {}
    Student(const std::string &name_, const std::string &id_)
        : name(name_), id(id_), entranceYear(std::stoi(id_.substr(0, 4))) {}
    Student() = default;
};
```

构造函数初始值列表

“先默认初始化，后赋值”有什么坏处？

- 不是所有类型都能默认初始化，不是所有类型都能赋值。
 - 引用无法被默认初始化，无法被赋值。`const` 对象无法被赋值。
 - （内置类型的）`const` 对象无法被默认初始化。
 - 我们将在后面看到，类类型可以拒绝支持默认初始化和赋值，这取决于设计。
- 如果你把能默认初始化、能赋值的成员在函数体里赋值，其它成员在初始值列表里初始化，**你的成员初始化顺序将会非常混乱**，很容易导致错误。

[Best practice] 尽可能用构造函数初始值列表，按照成员的声明顺序逐个初始化它们。

你的类需要一个默认构造函数吗？

宁缺毋滥：如果它没有一个合理的默认构造行为，就不应该有默认构造函数！

- 而不是胡乱定义一个或者 `=default`，这会留下隐患。
- 对默认构造函数的调用应当引发**编译错误**，而不是随意地允许。

《Effective C++》条款 6：如果编译器合成的函数你不需要，应当显式地禁止。

默认构造函数

特殊的构造函数：不接受参数。

- 专门负责对象的**默认初始化**，因为调用它时不需要传递参数，相当于不需要任何初始化器。
- 类类型的**值初始化**（几乎）就是调用默认构造函数。

```
class Point2d {
    double x, y;
public:
    Point2d() : x(0), y(0) {}
    Point2d(double x_, double y_) : x(x_), y(y_) {}
};
Point2d p1;           // 调用默认构造函数, (0, 0)
Point2d p2{};         // value-initialization , 在这里也是调用默认构造函数, (0, 0)
Point2d p3(3, 4);     // 调用 Point2d(double, double), (3, 4)
Point2d p4();         // 这是在调用默认构造函数吗？
```

类内初始值

default member initializer, 或者叫 in-class initializer

可以在声明数据成员的时候顺便给它一个初始值。

```
struct Point2d {
    double x;
    double y = 0;
    Point2d() = default;
    Point2d(double x_) : x(x_) {} // y = 0
};
Point2d p; // x 具有未定义的值，而 y = 0
Point2d p2(3.14); // x = 3.14, y = 0
```

如果这个成员在某个构造函数初始值列表里没有出现，它会采用类内初始值，而不是默认初始化。

你的类需要一个默认构造函数吗？

首先，如果需要开数组，你几乎肯定需要默认构造函数：

```
Student s[1000]; // 全部默认初始化
Student s2[1000] = {a, b}; // 前两个元素是 a 和 b，后面的全部值初始化，
                           // 而值初始化也是调用默认构造函数
```

但关键问题是它是否具有一个合理的定义？

- 一个“默认的学生”应该具有什么姓名和学号？

类内初始值

但是类内初始值不能用圆括号...

```
struct X {
    // 错误：受限于编译器的设计，它会在语法分析阶段被认为是函数声明
    std::vector<int> v(10);

    // 这毫无疑问是声明一个函数，而非设定类内初始值
    std::string s();
};
```

成员初始化的方式

从特殊到一般：

- 1. 如果这个成员在当前构造函数的初始值列表中出现了，则遵循初始值列表中给它的初始化器。
- 2. 否则，如果这个成员具有类内初始值，则使用类内初始值。
- 3. 否则，如果它能默认初始化，则默认初始化。
- 4. 否则，这个成员在当前构造函数中无法被初始化。
 - 如果这是用户自己写的构造函数，则产生编译错误。
 - 如果是编译器合成的构造函数，这个构造函数就是 deleted 的。

Dynarray：构造函数

构造函数定义的是初始化的方式。

```
Dynarray a;           // 对应默认构造函数 Dynarray()
Dynarray b(n);        // 对应构造函数 Dynarray(std::size_t)
Dynarray c(n, x);     // 对应构造函数 Dynarray(std::size_t, int)

class Dynarray {
public:
    Dynarray();
    Dynarray(std::size_t);
    Dynarray(std::size_t, int);
};
```

CS100 Recitation 9

GKxx

Dynarray：构造函数

假如我们需要一个默认构造函数，它应当具有什么行为？

- 初始化为 0 个元素的数组？
 - 这样的数组能使用吗？
- 初始化为 DYNARRAY_DEFAULT_SIZE 个元素的数组？

class 的基础：以 Dynarray 为例

Dynarray：构造函数

假如我们需要一个默认构造函数，它应当具有什么行为？

- 初始化为 0 个元素的数组？
 - 在不知道如何拷贝的情况下，我们似乎没有什么能改变数组大小的操作。
 - 如果不能改变大小，那“0 个元素的数组”有什么用？
- 初始化为 DYNARRAY_DEFAULT_SIZE 个元素的数组？
 - 也许在某些应用场景下，这是合理的。

设计

实现一个“动态数组”：

```
int n; std::cin >> n;
Dynarray arr(n); // arr has n value-initialized ints.
for (int i = 0; i != n; ++i)
    std::cin >> arr.at(i); // arr.at(i) is just like arr[i]
                           // 如何做到 arr[i]? 以后再说
for (std::size_t i = 0; i != arr.size(); ++i)
    arr.at(i) += arr.at(i - 1) * 2;
// ...
```

最后 arr 应当自己释放自己所占用的内存。

默认构造函数 (default constructor)：复习

- 默认构造函数的“默认”体现在哪？
- 哪些使用方式依赖于默认构造函数？
- 如果没有自定义默认构造函数，这个类会有默认构造函数吗？
 - 如果有，其行为是什么？
- 默认构造函数 = default 意味着什么？

Dynarray：成员

```
class Dynarray {
    std::size_t m_size;
    int *m_storage;
public:
    bool empty() const {
        return m_size == 0u;
    }
    std::size_t size() const {
        return m_size;
    }
};
```

默认构造函数 (default constructor)：复习

- 默认构造函数的“默认”体现在哪？——无参数
- 哪些使用方式依赖于默认构造函数？——默认初始化和值初始化
 - ```
Dynarray a, b{};
Dynarray *c = new Dynarray, *d = new Dynarray(), *e = new Dynarray{};
Dynarray f[1000]; std::array<Dynarray, 1000> g; // 这是啥？
```
- 如果没有自定义默认构造函数，这个类会有默认构造函数吗？  
——如果有其它自定义的构造函数，编译器就不会合成默认构造函数
- 默认构造函数 = default 意味着什么？

默认构造函数 (default constructor): 复习

如果没有其它自定义的构造函数，或者我们显式地以 `= default` 要求编译器合成一个默认构造函数：

- 编译器会合成一个具有默认行为的默认构造函数。
- “默认行为”：按照成员声明顺序逐个初始化它们
  - 对于有类内初始值的成员，使用类内初始值
  - 对于其它成员，进行**默认初始化**

析构函数的具体行为

考虑一个问题： `Student` 是否拥有什么资源？

```
class Student {
 std::string name;
 std::string id;
 int entranceYear;
 // ...
};
```

析构函数 (destructor, dtor)

`Dynarray` 必须在自己不再被使用的时候释放所拥有的内存：

```
void fun() {
 Dynarray a(n);
 if (condition) {
 Dynarray b(m);
 // ...
 } // Now b should release the memory it allocated.
 // ...
} // Now a should release the memory it allocated.
```

析构函数的具体行为

`Student` 本身不拥有什么资源，但是 `std::string` 有。

- `std::string` 当然会在它自己的析构函数里释放内存。

```
class Student {
 std::string name;
 std::string id;
 int entranceYear;
 // ...
};
```

在一个 `Student` 被销毁的时候：

- 它显然应该销毁它的所有成员
- 销毁一个成员，显然应该调用那个成员的析构函数。

析构函数 (destructor, dtor)

**析构函数**是在这个对象被销毁的时候自动调用的函数。

- 它通常用来完成一些最后的清理
- 特别地，那些**拥有一定资源**的类通常在析构函数里释放它们所拥有的资源

析构函数的具体行为

**析构函数在执行完函数体之后，会自动销毁它的所有数据成员**

- 对于类类型成员，会调用它的析构函数来销毁它。
- 销毁成员的顺序？

定义一个析构函数

```
class Student {
 std::string name;
 std::string id;
 int entranceYear;
public:
 ~Student() {
 std::cout << "dtor of "
 << name << "called.\n";
 }
};

{
 Student alice("Alice", "2020123123");
 if (condition) {
 Student bob("Bob", "2020123124");
 // ...
 } // Output "dtor of Bob called."
 } // Output "dtor of Alice called."
```

- 析构函数的名字是 `~ClassName`
- 析构函数不接受参数，不声明返回值类型
- 一个类只能有一个析构函数

析构函数的具体行为

**析构函数在执行完函数体之后，会自动销毁它的所有数据成员**

- 对于类类型成员，会调用它的析构函数来销毁它。
- 按照成员的声明顺序**倒序**销毁它们。

对比一下构造函数：

- 在执行函数体**之前**初始化所有成员。
- 对于类类型成员，调用它的构造函数进行初始化。
- 初始化的顺序是成员的声明顺序。

Dynarray：析构函数

非常直接的实现

```
class Dynarray {
 std::size_t m_size;
 int *m_storage;
public:
 Dynarray(std::size_t n)
 : m_size(n), m_storage(new int[n]{}) {}
 ~Dynarray() {
 delete[] m_storage;
 }
};
```

析构函数的具体行为

对于成员的销毁是不需要我们写的，会自动完成。

```
class Student {
 std::string name;
 std::string id;
 int entranceYear;
public:
 ~Student() {} // 编译器会在最后插入代码来调用 id 和 name 的析构函数。
};
```

`Student` 类的析构函数只需一个空函数体即可。

等价的写法： `~Student() = default;`

**在任何一个成员函数的函数体中，这个对象的所有数据成员都处在已经初始化、未被析构的状态下。**

析构函数的具体行为

初学者典型的错误：

```
class Student {
 std::string name;
 std::string id;
 int entranceYear;
public:
 ~Student() {
 // 完全误解了析构函数的函数体的用途。
 delete this;
 }
};
```

可以，但没必要：

```
class Student {
 std::string name;
 std::string id;
 int entranceYear;
public:
 ~Student() {
 // 毫无意义的行为。
 name.clear(); id.clear();
 entranceYear = 0;
 }
};
```

析构函数还能干什么？

```
struct PrinterGuard {
 PrinterGuard() {
 hide_cursor();
 disable_input_echo();
 }
 ~PrinterGuard() {
 show_cursor();
 enable_input_echo();
 }
};

void drawUI(/* ... */) {
 PrinterGuard guard;
 // 画一些东西 ...
 // guard 被销毁，光标和输入显示被恢复
}
```

```
void loadMap(const std::string &path) {
 std::ifstream file(path);

 // ...

 // std::ifstream 的析构函数会关闭文件
}

void criticalSection(std::mutex &m) {
 std::lock_guard guard(m);
 // ...
 // std::lock_guard 的析构函数会解锁 m
}
```

析构函数的具体行为

初学者典型的错误：

```
class Dynarray {
 std::size_t m_size;
 int *m_storage;
public:
 ~Student() {
 delete[] m_storage;
 delete this; // 错误!
 }
};
```

可以，但没必要：

```
class Dynarray {
 std::size_t m_size;
 int *m_storage;
public:
 ~Student() {
 delete[] m_storage;
 // 下面的赋值毫无意义。
 m_size = 0;
 m_storage = nullptr;
 }
};
```

new 和 delete

```
std::string *p = new std::string("Hello");
std::cout << *p << std::endl;
delete p;
p = new std::string; // 调用默认构造函数，*p 为空串 ""
std::cout << p->size() << std::endl;
delete p;
```

new 表达式会先分配内存，然后构造对象。

- 如果这是一个类类型，它必然会调用一个构造函数来构造对象。在没有指定如何构造的情况下，它会调用默认构造函数。
- 相比之下，来自 C 的 malloc 只会分配内存，不构造任何对象（不做任何初始化）。calloc 会将这个内存清零，而非调用默认构造函数。

一个类不能没有析构函数，就像...

如果一个类的析构函数是不可调用的，就意味着它无法被销毁

- 就如同不可降解的塑料

C++ 不允许定义这样的类类型的对象！

什么情况下析构函数不可调用？

new 和 delete

```
std::string *p = new std::string("Hello");
std::cout << *p << std::endl;
delete p;
p = new std::string; // 调用了默认构造函数，*p 为空串 ""
std::cout << p->size() << std::endl;
delete p;
```

delete 表达式会先销毁对象，然后释放这片内存。

- 如果这是一个类类型，它必然会调用析构函数来销毁这个对象。
- 相比之下，free 只释放内存，不调用析构函数。

一个类不能没有析构函数，就像...

如果一个类的析构函数是不可调用的，就意味着它无法被销毁

C++ 不允许定义这样的类类型的对象！

什么情况下析构函数不可调用？

- 你可以显式地 ~ClassName() = delete; 将析构函数定义为“删除的”。
- 如果析构函数是 private 的，它就不能在类外、friend 外被调用。

如果一个类的析构函数是不可用的，创建这个类型的对象的唯一方式是 new，但你还没法 delete 它。

Dynarray：元素访问

.at(i) 应该返回什么？

```
class Dynarray {
 std::size_t m_size;
 int *m_storage;
public:
 ??? at(std::size_t n) {
 return m_storage[n];
 }
};
```

析构函数还能干什么？

```
void drawUI(/* ... */) {
 hide_cursor();
 disable_input_echo();

 // 画一些东西 ...

 // 要记得恢复光标和输入显示
 show_cursor();
 enable_input_echo();
}
```

```
void loadMap(const char *path) {
 FILE *file = fopen(path, "r");
 // 从文件中读取一些内容 ...
 fclose(file);
}

void criticalSection(std::mutex &m) {
 m.lock();
 // ...
 m.unlock();
}
```

Dynarray：元素访问

```
arr.at(0) = 42;
std::cout << arr.at(1);
```

如果我们希望通过 arr.at(i) 来修改这个元素，那必然要返回引用。

```
class Dynarray {
 std::size_t m_size;
 int *m_storage;
public:
 int &at(std::size_t n) {
 return m_storage[n];
 }
};
```

这样可以吗？

## Dynarray：元素访问

试一试：

```
void print(const Dynarray &arr) {
 for (std::size_t i = 0; i != arr.size(); ++i)
 std::cout << arr.at(i) << ' ';
}
```

无法编译：`at` 不是 `const` 成员函数，无法在 `const Dynarray &` 上调用！

## Dynarray：元素访问

```
class Dynarray {
public:
 const int &at(std::size_t n) const {
 return m_storage[n];
 }
 int &at(std::size_t n) {
 return m_storage[n];
 }
};
arr.at(i) = 42;

// 左边的代码就如同：
const int &at(const Dynarray *this,
 std::size_t n) {
 return this->m_storage[n];
}
int &at(Dynarray *this, std::size_t n){
 return this->m_storage[n];
}
at(&arr, i) = 42;
```

- 在 `const` 对象上，它只能调用 `const` 版本，得到 `reference-to-const`，无法修改
- 在非 `const` 对象上：两个版本都可以调用，但是
  - 调用 `non-const` 版本是完美匹配，调用 `const` 版本是添加底层 `const` 得到的匹配，因此前者是更好的匹配。

## 在 const vs non-const 重载中避免重复

假如我们要模仿标准库的行为的话，`at` 函数应该提供边界检查...

```
class Dynarray {
public:
 const int &at(std::size_t n) const {
 if (n >= m_length) // 为什么不需要判断 n < 0?
 throw std::out_of_range{"Dynarray subscript out of range."};
 return m_storage[n];
 }
 int &at(std::size_t n) {
 if (n >= m_length)
 throw std::out_of_range{"Dynarray subscript out of range."};
 return m_storage[n];
 }
};
```

## 在 const vs non-const 重载中避免重复

假如我们还想在 `at` 访问中做一些其它的记录和检查...

```
class Dynarray {
public:
 const int &at(std::size_t n) const {
 if (n >= m_length)
 throw std::out_of_range{"Dynarray subscript out of range."};
 log_access();
 verify_integrity();
 return m_storage[n];
 }
 int &at(std::size_t n) {
 if (n >= m_length)
 throw std::out_of_range{"Dynarray subscript out of range."};
 log_access();
 verify_integrity();
 return m_storage[n];
 }
};
```

## const 成员函数：复习

- `const` 写在哪？
- `const` 成员函数的 `const` 是作用于谁的？
- 在什么对象上能调用 `const` 成员函数？
- `const` 成员函数能做什么事？不能做什么事？

## const 成员函数：复习

- `const` 写在哪？——参数列表后
- `const` 成员函数的 `const` 是作用于谁的？
  - 加在隐式的 `this` 指针上的底层 `const`
  - 表示当前对象是 `const`，其所有数据成员也都是 `const`
- 在什么对象上能调用 `const` 成员函数？
  - 什么对象上都可以，因为添加底层 `const` 永远没问题。
- `const` 成员函数能做什么事？不能做什么事？
  - 不能修改数据成员，不能调用数据成员的 `non-const` 成员函数
  - 不能调用自身的 `non-const` 成员函数

## Dynarray：元素访问

加个 `const` ？

```
class Dynarray {
 std::size_t m_size;
 int *m_storage;
public:
 int &at(std::size_t n) const {
 return m_storage[n];
 }
};
```

结果是在 `const Dynarray` 上，你可以得到其中元素的 `non-const` 引用，进而修改它！

## 在 const vs non-const 重载中避免重复

将一模一样的代码编写两遍实在是太麻烦了... 有没有办法避免重复？

- `C++23 deducing-this` 能帮得上忙

假如没有额外的语法特性... 能不能让一个函数调用另一个？

- 让谁调用谁？

## 在 const vs non-const 重载中避免重复

将一模一样的代码编写两遍实在是太麻烦了... 有没有办法避免重复？

- `C++23 deducing-this` 能帮得上忙

假如没有额外的语法特性... 能不能让一个函数调用另一个？

假如我们让 `non-const` 版本的函数调用 `const` 版本的函数：

- 首先，我们需要显式地为 `this` 添加底层 `const`。
- `const` 版本的函数返回的是 `const int &`，我们得把它的底层 `const` 去除。

## Dynarray：元素访问

正确的解决方案：`const` 和 `non-const` 的重载

```
class Dynarray {
 std::size_t m_size;
 int *m_storage;
public:
 const int &at(std::size_t n) const {
 return m_storage[n];
 }
 int &at(std::size_t n) {
 return m_storage[n];
 }
};
```

- 在 `const` 对象上，它只能调用 `const` 版本，得到 `reference-to-const`，无法修改
- 在非 `const` 对象上会调用哪个？

30/42

35/42

31/42

36/42

32/42

37/42

33/42

38/42

34/42

39/42

在 const vs non-const 重载中避免重复

- 先用 static\_cast<const Dynarray \*>(this) 为 this 添加底层 const
- 这时调用 ->at(n), 就会匹配 const 版本的 at
- 将返回的 const int & 的底层 const 用 const\_cast 去除

```
class Dynarray {
public:
 const int &at(std::size_t n) const {
 if (n >= m_length)
 throw std::out_of_range{"Dynarray subscript out of range."};
 log_access();
 verify_integrity();
 return m_storage[n];
 }
 int &at(std::size_t n) {
 return const_cast<int &>(static_cast<const Dynarray *>(this)->at(n));
 }
};
```

40/42

在 const vs non-const 重载中避免重复

能不能反过来, 让 const 版本调用 non-const 版本?

```
class Dynarray {
public:
 int &at(std::size_t n) {
 if (n >= m_length)
 throw std::out_of_range{"Dynarray subscript out of range."};
 log_access();
 verify_integrity();
 return m_storage[n];
 }
 const int &at(std::size_t n) const {
 return const_cast<Dynarray *>(this)->at(n);
 }
};
```

41/42

在 const vs non-const 重载中避免重复

能不能反过来, 让 const 版本调用 non-const 版本?

- **不能!** const 成员函数里一定不会修改对象的状态, 但是 non-const 成员函数并没有这般的承诺!
- 如果在 non-const 版本的实现里一不小心修改了对象的状态, 让 const 版本调用它将导致灾难。

比较一下两种方法中对于“危险的”const\_cast 的使用?

- “先添加, 再去除”: OK
- “先去除, 再添加”: 危险

CS100 Recitation 10

GKxx

目录

- 左/右值与拷贝/移动
- 拷贝控制成员

左/右值与拷贝/移动

左/右值与拷贝/移动

假设 x 拥有一定的资源。获取它的资源的方式有以下两种:

- 拷贝: 将它的资源拷贝出来
- 移动: 将它的资源据为己有

左值持久, 右值短暂:

- “左值”通常代表具有长久的生命期的对象, 你不能随便移动它的资源。
- “右值”通常代表“即将死亡”的对象 (当然也有可能是“自杀”), 我们可以直接把它资源据为己有, 完全没有必要拷贝。

左/右值与拷贝/移动

一般而言, 我们希望拷贝左值, 移动右值。

- 但如果移动操作不可用, 我们也希望能正常拷贝右值。

```
class Message {
 std::string m_contents;

public:
 explicit Message(const std::string &contents) : m_contents{contents} {}
 explicit Message(std::string &&contents) : m_contents{std::move(contents)} {}
};

std::string s1 = foo(), s2 = bar();
Message m1(s1); // s1 经过 contents 被拷贝给 m_contents
Message m2(s1 + s2); // s1 + s2 产生的临时结果被移动给 m_contents
```

Pass-by-reference-to-const 不够好了!

我们已经习惯于将不修改的参数声明为 reference-to-const:

```
class Message {
 std::string m_contents;

public:
 explicit Message(const std::string &contents) : m_contents{contents} {}
};
```

但是如果传进来的字符串是右值, 它仍然会被拷贝。怎样移动这个右值?

一种解决方案: 像前一页那样提供一组重载, 分别接受左值和右值。

Pass-by-value, 然后移动

直接按值传递!

```
class Message {
 std::string m_contents;

public:
 explicit Message(std::string contents) : m_contents{std::move(contents)} {}
};
```

拷贝/移动会在对参数 contents 的初始化中自动决定, 而我们只需要把 contents 移给 m\_contents。

```
std::string s1 = foo(), s2 = bar();
Message m1(s1); // 用左值 s1 初始化参数 contents, 这是拷贝
Message m2(s1 + s2); // 用右值 s1 + s2 初始化 contents, 这是移动
```

## std::move

std::move(x) 并不移动 x 。但是假如有以下重载：

```
void foo(T &); // (1)
void foo(T &&); // (2)
```

那么对于一个 T 类型的变量 x，foo(x) 会选择 (1)，foo(std::move(x)) 会选择 (2)。

例：std::vector<T>::push\_back 提供一组重载，拷贝左值，移动右值。

```
std::vector<std::string> wordList;
for (int i = 0; i != n; ++i) {
 std::string word; std::cin >> word;
 wordList.push_back(std::move(word)); // word 被移动进 wordList，而非被拷贝进去。
 // word 的作用到此为止，所以我们应该将它移入 wordList，不应该拷贝它。
}
```

8/36

## 只要有名字，就是左值

```
struct X {
 std::string s;
 // /* bad */ X(X &&other) noexcept : s(other.s) {}
 /* good */ X(X &&other) noexcept : s(std::move(other.s)) {}
};
```

other 是一个左值，直接取 other.s 会得到一个左值，使得这个字符串被拷贝。

- “左值持久，右值短暂”：右值引用延长了右值的生命期，使用右值引用时就如同在使用一个普通的变量。

要想移动 other.s，必须套一层 std::move。

9/36

## 参数转发

回顾 std::make\_shared 和 std::make\_unique：

```
auto sp = std::make_shared<std::string>(10, 'c'); // "ccccccccc"
auto sp2 = std::make_shared<std::string>("hello"); // "hello"
auto up = std::make_unique<Student>("Alice", "2020123123");
```

甚至，如果传入右值，它们会移动构造那个对象：

```
auto sp3 = std::make_shared<std::string>(std::move(*sp));
std::cout << *sp << std::endl; // empty string
```

这种将参数转发给另一个函数，又能保持它们的值类别的操作叫做**完美转发** (perfect forwarding)

10/36

## 参数转发

std::make\_shared/unique<T>(…) 可以接受任意多个任意类型的参数，并将它们原封不动地转发给 T 的构造函数，不丢失值类别，不丢失 const。

等学了模板，就知道是咋回事了。

标准库很多函数都支持这样的操作，其中非常典型的是容器的 **emplace 系列操作**：

```
std::vector<Student> students;
students.emplace_back("Alice", "2020123123");
std::vector<std::string> words;
words.emplace_back(10, 'c');
```

11/36

## 标准库容器的 emplace

```
std::vector<Student> students;
students.emplace_back("Alice", "2020123123");
std::vector<std::string> words;
words.emplace_back(10, 'c');
```

emplace 系列操作利用传入的参数直接原地构造出那个对象，而不是将构造好的对象拷贝/移动进去。

- 提高效率。
- 对所存储的数据类型的要求进一步降低。尤其是 std::list<T>（链表）自 C++11 起不需要 T 具备任何拷贝/移动操作，只要有办法构造和析构即可。
- std::vector 由于需要搬家（增长时重新分配内存），无法存储不可拷贝、不可移动的元素，除非你不需要它搬家。

12/36

## 拷贝控制成员

### 拷贝控制成员

- 拷贝构造函数 copy constructor
- 拷贝赋值运算符 copy assignment operator
- 移动构造函数 move constructor
- 移动赋值运算符 move assignment operator
- 析构函数 destructor

虽然后三个名字里没有“拷贝”，但也属于“copy control members”。

两个移动操作是 C++11 开始有的。

13/36

### 何时需要

首先，分清初始化和赋值。

- 初始化是在变量声明语句中的，它需要调用构造函数。
- 赋值是一个运算符，它必然在表达式中。
- 构造和赋值有一定相关性，但归根结底调用的是不同的函数。

拷贝左值，移动右值。但如果移动操作不可用，右值也被拷贝。

析构函数的调用意味着对象生命期的结束。

- 超出作用域时，程序结束时，以及 delete / delete[] 表达式

15/36

### 默认行为

在某些情况下（包括用 = default 显式要求时），编译器会合成一个具有默认行为的拷贝控制成员。

- 默认行为：**先父类，后自己的成员**，且成员按**声明顺序**，逐个执行对应的操作。
  - 析构顺序相反。
- 默认的移动行为：等同于将 std::move 作用于每个成员。
  - 并不是苛求每个成员或父类都采用移动操作。
  - 能移动就移动，不能移动就拷贝。

如果默认行为中涉及的任何一個操作无法正常进行（不存在或不可访问），这个函数就是删除的 (deleted function)。

### = delete

删除的函数 (deleted function)

- 仍然参与重载决议，
- 但如果被匹配到，就是 error。
- 任何函数都可以是删除的，并不局限于特殊成员函数。

特别例外：如果编译器合成了一个删除的移动操作，它不会参与重载决议，这是为了让右值被拷贝。[\[CWG1402\]](#)

- 《C++ Primer》在这个问题上没有及时更新。



17/36

### 三/五法则

C++11 以前是“三”，C++11 以后是“五”。

如果你认为有必要自定义这五个函数中的任何一个，通常意味着这五个你都应该定义。

"Define zero or five or them."

18/36

### 三/五法则

根据“五法则”：如果五个函数中的任何一个具有用户自定义 (user-provided) 的版本，编译器就不应该再合成其它那些用户没有定义的函数。

- 重要例外：一个类不能没有析构函数 就像...
- 另一个例外：兼容旧的代码
  - 在 C++98 时代，“三法则”并未在编译器的行为上予以体现。
  - 如果一个类有自定义的拷贝构造函数或析构函数，而没有自定义拷贝赋值运算符，C++98 编译器会合成这个拷贝赋值运算符。（拷贝构造函数同理）
  - 为了兼容旧的代码，不能直接禁止这种行为，只能将它判定为 deprecated。

19/36

### Dynarray 的拷贝赋值运算符

这个对不对？

```
class Dynarray {
public:
 Dynarray &operator=(const Dynarray &other) {
 delete[] m_storage;
 m_storage = new int[other.m_length];
 std::copy_n(other.m_storage, other.m_length, m_storage);
 m_length = other.m_length;
 return *this;
 }
};
```

20/36

### Dynarray 的拷贝赋值运算符

这个对不对？

```
class Dynarray {
public:
 Dynarray &operator=(const Dynarray &other) {
 delete[] m_storage;
 m_storage = new int[other.m_length];
 std::copy_n(other.m_storage, other.m_length, m_storage);
 m_length = other.m_length;
 return *this;
 }
};
```

错误！没有做到自我赋值安全。

21/36

### Dynarray 的拷贝赋值运算符

这个对不对？

```
class Dynarray {
public:
 Dynarray &operator=(const Dynarray &other) {
 auto new_data = new int[other.m_length];
 std::copy_n(other.m_storage, other.m_length, new_data);
 m_storage = new_data;
 m_length = other.m_length;
 return *this;
 }
};
```

22/36

### Dynarray 的拷贝赋值运算符

```
class Dynarray {
public:
 Dynarray &operator=(const Dynarray &other) {
 // if (this != &other) { // 加个判断也可以
 auto new_data = new int[other.m_length];
 std::copy_n(other.m_storage, other.m_length, new_data);
 delete[] m_storage;
 m_storage = new_data;
 m_length = other.m_length;
 // }
 return *this;
 }
};
```

23/36

### Dynarray 的移动操作

```
class Dynarray {
public:
 Dynarray(Dynarray &&other) noexcept
 : m_length{other.m_length}, m_storage{other.m_storage} {
 // 为何要有以下两步？
 other.m_length = 0; other.m_storage = nullptr;
 }
 Dynarray &operator=(Dynarray &&other) noexcept {
 if (this != &other) {
 delete[] m_storage;
 m_length = other.m_length; m_storage = other.m_storage;
 // 为何要有以下两步？
 other.m_length = 0; other.m_storage = nullptr;
 }
 }
};
```

移动操作应将被移动的对象置于有效的、可被安全析构的状态。

24/36

### Dynarray 的移动操作

```
class Dynarray {
public:
 Dynarray(Dynarray &&other) noexcept
 : m_length{std::exchange(other.m_length, 0)},
 m_storage{std::exchange(other.m_storage, nullptr)} {}
 Dynarray &operator=(Dynarray &&other) noexcept {
 if (this != &other) {
 delete[] m_storage;
 m_length = std::exchange(other.m_length, 0);
 m_storage = std::exchange(other.m_storage, nullptr);
 }
 }
};
```

25/36

### Copy-and-swap

能不能写出一个简单的 swap 函数，交换两个 Dynarray 对象的值？

```
class Dynarray {
public:
 void swap(Dynarray &) noexcept;
};
```

26/36

### Copy-and-swap

能不能写出一个简单的 swap 函数，交换两个 Dynarray 对象的值？

```
class Dynarray {
public:
 void swap(Dynarray &other) noexcept {
 std::swap(m_storage, other.m_storage);
 std::swap(m_length, other.m_length);
 }
};
```

直接交换 m\_storage 指针，就可以快速交换两个“动态数组”。这个 swap 甚至是 noexcept 的，它远远好过传统的 auto tmp = a; a = b; b = tmp; 写法。

27/36

## Copy-and-swap

赋值 = 拷贝构造 + 析构：拷贝新的数据，销毁原有的数据。

能不能利用拷贝构造函数和析构函数写出一个拷贝赋值运算符？

28/36

## Copy-and-swap

赋值 = 拷贝构造 + 析构：拷贝新的数据，销毁原有的数据。

为 `other` 建立一个拷贝 `tmp`，直接将自己和 `tmp` 交换！

```
class Dynarray {
public:
 Dynarray &operator=(const Dynarray &other) {
 auto tmp = other;
 swap(tmp);
 return *this;
 }
};
```

- 拷贝构造函数会负责正确拷贝 `other`。
- `tmp` 的析构函数会正确销毁旧的数据。

29/36

## Copy-and-swap

更简洁些：

```
class Dynarray {
public:
 Dynarray &operator=(const Dynarray &other) {
 Dynarray(other).swap(*this); // C++23: auto{other}.swap(*this);
 return *this;
 }
};
```

自我赋值安全吗？

30/36

## Copy-and-swap

```
class Dynarray {
public:
 Dynarray &operator=(const Dynarray &other) {
 Dynarray(other).swap(*this); // C++23: auto{other}.swap(*this);
 return *this;
 }
};
```

不仅好写，还自我赋值安全，还提供强异常安全保证！

31/36

## "Copy"-and-swap

更进一步，直接在传参的时候做好“拷贝”。

```
class Dynarray {
public:
 Dynarray &operator=(Dynarray other) noexcept {
 swap(other);
 return *this;
 }
};
```

且慢——传参的时候真的发生了拷贝吗？

32/36

## "Copy"-and-swap

```
class Dynarray {
public:
 Dynarray &operator=(Dynarray other) noexcept {
 swap(other);
 return *this;
 }
};
```

如果参数是右值，`other` 将被移动初始化，而不是拷贝初始化。

也就是说，这个赋值运算符既是一个拷贝赋值运算符，又是一个移动赋值运算符！

33/36

## Copy-and-swap

通过实现一个快速、`noexcept` 的 `swap` 函数，一举多得。

利用这个 `swap` 实现赋值运算符：不需要额外做任何操作。

- 自我赋值安全
- 异常安全（提供强异常安全保证）
- 同时获得拷贝赋值运算符和移动赋值运算符

34/36

## 再看 Dynarray

我们将数据成员 `m_length` 和 `m_storage` “藏起来”，并定义了各种接口。有什么好处？

```
struct Dynarray { std::size_t length; int *storage; };
void createDynarray(Dynarray &array, size_t length) {
 array.length = length; array.storage = new int[length]{};
}
void destroyDynarray(Dynarray &array) { delete[] array->storage; }
void dynarrayAssign(Dynarray &lhs, const Dynarray &rhs) { /* ... */ }
```

```
{ // 正确使用：小心翼翼
 Dynarray a;
 createDynarray(a, 10);
 Dynarray b;
 dynarrayAssign(b, a);
 destroyDynarray(a);
 destroyDynarray(b);
}
```

```
// 一不小心就用错
Dynarray a;
createDynarray(a, 10);
Dynarray b = a; // !!
// ...
```

```
// 我没看见 createDynarray
// 但反正都是 public 的，
// 我自己动手！
Dynarray a;
a.length = 1;
a.storage = new int{10};
destroyDynarray(a); // !!
```

35/36

## Dynarray 的 class invariants

A class invariant is a property that must always hold for an object of that class.

- 指针 `m_storage` 要么是空指针，要么指向由 `new[]` 分配的一片内存。
- `m_length` 总是表示 `m_storage` 指向的内存的长度，或者（在 `m_storage` 是空指针时）为零。

用户只能使用我们定义的接口，不能直接访问 `m_length` 或 `m_storage`，所以不能轻易破坏 class invariants

- 构造函数和析构函数在对象被创建和被销毁时会被自动调用，更加保证了这一点，也防止了内存泄漏。

36/36

## CS100 Recitation 11

GXxx

1/39

## 返回多个值

想返回 (min,max) 两个值, 怎么办?

```
??? getMax(const std::vector<int> &vec) {
 int min = INT_MAX, max = INT_MIN;
 for (auto x : vec) {
 // ...
 }
 return ???
}
```

2/39

## 返回多个值

C 里讲过的办法: 创建一个结构体

- 在 C++ 里就是一种简单的类。

```
struct MinAndMax {
 int min;
 int max;
};
MinAndMax getMax(const std::vector<int> &vec) {
 int min = INT_MAX, max = INT_MIN;
 for (auto x : vec) {
 // ...
 }
 return /* 这里怎么写? */
}
```

3/39

## 聚合类 (aggregate)

聚合类是那些所有 non-`static` 数据成员都是 `public`、没有自定义构造函数且满足一些其它条件的类。

- 核心特点一: 数据成员都是 `public`, 看起来就如同是几个变量打包在一起, 这些变量本身就是接口。
- 核心特点二: 没有自定义构造函数, 初始化 = 对所有成员进行初始化。
- 其它条件, 例如基类都是 `public` 继承, 没有虚函数等。

这样的类从使用的角度讲, 与 C 的 `struct` 非常接近。

4/39

## 使用大括号列表

```
struct MinAndMax {
 int min;
 int max;
};
MinAndMax result{a, b};

MinAndMax getMax(/* ... */) {
 int min = INT_MAX, max = INT_MIN;
 // ...
 return {min, max};
}
```

一般地, 如果用 {e1, e2, ...} 初始化一个 `T` 类型的对象, 例如 `T x{a, b, c};`:

- 如果 `T` 是一个聚合体, 那么这是一种**聚合初始化 (aggregate initialization)**: `e1`, `e2`, ... 依次用来初始化第一个子对象、第二个子对象、.....
- 否则, 如果 `T` 含有接受 `std::initializer_list<V>` 的构造函数, 则这个列表被传给这个构造函数。
- 否则, 就如同 `T x(a, b, c);`, 走正常的构造函数 (**直接初始化 (direct initialization)**)。

5/39

## 使用大括号列表

如果 `MinAndMax` 有一个用户提供的构造函数:

```
struct MinAndMax {
 int min;
 int max;
 MinAndMax(int min_, int max_) : min{min_}, max{max_} {}
};
```

那么 `MinAndMax` 就不再是聚合类, 但是我们仍然可以用 {a, b} 初始化一个该类型的对象, 也仍然可以 `return {min, max};`

总之, 下面这种做法一定是可以避免、需要避免的:

```
MinAndMax result;
result.min = a;
result.max = b;
```

6/39

## 结构化绑定 (Structured binding)

```
struct MinAndMax { int min, max; };
MinAndMax getMax(const std::vector<int> &);
```

调用者怎么获得这个返回值?

```
auto result = getMax(numbers);
std::cout << result.min << ' ' << result.max << '\n';
```

这当然可以, 但是有更好的办法: **结构化绑定**

```
auto [minVal, maxVal] = getMax(numbers);
std::cout << minVal << ' ' << maxVal << '\n';
```

7/39

## 结构化绑定 (Structured binding)

结构化绑定的声明可以带有 `const`、引用等, 也可以放在基于范围的 `for` 语句里:

```
enum class Gender { Male, Female };
struct PersonInfo {
 std::string name;
 Gender gender;
 int birthYear;
};
void foo(const std::vector<PersonInfo> &persons) {
 for (const auto &[n, g, by] : persons) {
 // ...
 }
}
```

8/39

## std::pair

定义在 `<utility>` 中

一个快速、随意的数据结构

```
std::pair<T1, T2> p1{a, b}; // 用圆括号也可以
auto p2 = std::make_pair(a, b); // 等价的写法
std::pair p3{a, b}; // 编译器根据 a 和 b 的类型来推导模板参数
// 这里用圆括号也可以, 但 {} 更 modern
```

C++17 有了 CTAD, 几乎不再需要 `std::make_pair` 了。

9/39

## std::pair

返回两个值: 装进 `pair` 里就行

```
std::pair<int, std::string> foo() {
 // ...
 return {42, "hello"};
}
```

访问 `pair` 中的元素: `p.first`, `p.second`

```
auto p = foo();
std::cout << foo.first << ", " << foo.second << std::endl;
```

10/39

## std::pair

结构化绑定也可以用于 `std::pair`:

```
auto [ival, sval] = foo();
std::cout << ival << ", " << sval << std::endl;
```

`std::pair` 有比较运算符: 按照 `.first` 为第一关键字、`.second` 为第二关键字比较。

不要滥用 `std::pair`!

```
using Complex = std::pair<double, double>; // bad idea!
using Student = std::pair<std::string, int>; // bad idea!
```

11/39

## 重载运算符 (Operator overloading)

### 总结：技术

运算符重载的方式是定义一个名为 `operator@` 的函数：

- 各个运算对象从左向右依次作为这个函数的参数。
- 如果是成员函数，则最左侧的运算对象绑定到 `*this`。

不能发明新的运算符。不能为内置类型定义运算符。

不能被重载：

- `?:`, `,`, `::` 等，具有特别意义的运算符

不建议被重载：（为什么？）

- `&&`, `||`, `,`, `&` (address-of)

### 总结：技术

运算符重载的方式是定义一个名为 `operator@` 的函数：

- 各个运算对象从左向右依次作为这个函数的参数。
- 如果是成员函数，则最左侧的运算对象绑定到 `*this`。

不能发明新的运算符。不能为内置类型定义运算符。

不能被重载：

- `?:`, `,`, `::` 等，具有特别意义的运算符
- `?:` 无法被重载，因为它有一个运算对象不被求值，这一点用函数传参不可能做到

不建议被重载：（为什么？）

- `&&`, `||`, `,`, `&` (一元取地址运算符)
- `&&` 和 `||` 的短路求值会失效。`,` 和 `&` 本来就能作用于一切类型。

### 总结：技术

重载的行为需要和内置的行为保持某种程度上的一致，除非你有很好的理由不这么做

- `++i` 返回 `i` 的引用，`i++` 返回递增前的 `i` 的一份拷贝。
- 赋值、复合赋值都返回左侧运算对象的引用。
- 解引用 `*p` 通常返回左值引用
- 比较运算符需要符合逻辑：`a == b` 意味着 `b == a`，`a != b` 意味着 `!(a == b)`。

C++20 的比较运算符的相关规则发生了巨大的变化：

- 允许用 `=default` 让编译器合成
- 新增了 `operator<=>` 用来定义数学上对应的序关系（partial ordering, strong ordering, weak ordering）
- 定义了 `operator==` 和 `operator<=>` 后编译器会合成其它关系运算符，还会照顾对称性。

### 总结：技术

通常是成员：`++`, `--`, `*` (解引用), `->`, `=` (赋值，必须是成员), 各种复合赋值运算符 (`+=`, `-=`, ...)

成员或非成员皆可：比较运算符 `<`, `<=`, `>`, `>=`, `==`, `!=`, 算术运算符等

如果需要左侧运算对象也能隐式转换，则它不能是成员。

- `r == 1` 被视为 `r.operator==(1)`，即 `r.operator==(Rational(1))`
- 但 `1 == r` 会被视为 `1.operator==(r)`（显然是错误），不会被进一步推断为 `Rational(1).operator==(r)`

### 不要漏 `const`！

```
class Rational {
public:
 bool operator==(const Rational &) const;
};
bool operator!=(const Rational &, const Rational &);
```

非成员有两个 `const`，成员也必然有两个 `const`，只是其中一个变成了这个成员函数的 qualifier。

### 特殊的运算符：后置递增 `i++`

```
class Rational {
public:
 Rational &operator++() { /* ... */ return *this; }
 Rational operator++(int) {
 // 若无特殊情况，后置递增运算符几乎总是这样写
 auto tmp = *this; // 拷贝原来的对象
 ++*this; // 真正的“递增”由前置版本完成
 return tmp; // 返回递增前的拷贝
 }
};
```

- `int` 参数：仅仅是为了区分前置版本和后置版本，没有实际意义，也不需要用到，自然也就没有名字。
  - 如果一个参数没有被用到，它的名字可以不被写出来，也可以标上 `[[maybe_unused]]`
- `++i` 被视为 `i.operator++()`，`i++` 被视为 `i.operator++(0)`。

### 特殊的运算符：`->`

```
class SharedPtr {
public:
 Object &operator*() const;
 Object *operator->() const {
 // 若无特殊情况，箭头运算符几乎总是这样写，以使 p->mem 和 (*p).mem 等价
 return std::addressof(this->operator*());
 }
};
```

- 为了让 `p->mem` 和 `(*p).mem` 等价，`operator->` 几乎总是应该这样定义。
  - 这个运算符有点怪（比如，它是一元的还是二元的？）。不用太纠结，实在好奇可以上网查。
- 注意在本例中 `operator*` 和 `operator->` 都是 `const`：它们都允许在 `const SharedPtr` 上调用。

### 避免重复：比较运算符

定义 `operator==` 和 `operator<`，让剩下四个依赖于它们。

```
bool operator!=(const Rational &lhs, const Rational &rhs) {
 return !(lhs == rhs);
}
bool operator>(const Rational &lhs, const Rational &rhs) {
 return rhs < lhs;
}
bool operator<=(const Rational &lhs, const Rational &rhs) {
 return !(lhs > rhs); // 你只会 lhs < rhs || lhs == rhs ?
}
bool operator>=(const Rational &lhs, const Rational &rhs) {
 return !(lhs < rhs);
}
```

### 避免重复：算术运算符

定义一元负号 `-` 和 `+=`，那么 `+`，`-`，`-=` 都可以用它们实现。

```
class Rational {
public:
 Rational &operator-=(const Rational &rhs) {
 return *this += -rhs;
 }
};
Rational operator+(const Rational &lhs, const Rational &rhs) {
 return Rational(lhs) += rhs;
}
Rational operator-(const Rational &lhs, const Rational &rhs) {
 return Rational(lhs) -= rhs;
}
```

## 输入、输出运算符: `operator<<` 和 `operator>>`

首先搞清楚: 输入流 (例如 `std::cin`) 的类型是 `std::istream`, 输出流 (例如 `std::cout`) 的类型是 `std::ostream`。

这两个运算符应该是成员还是非成员?

22/39

## 输入、输出运算符

`operator<<` 和 `operator>>`

首先搞清楚: 输入流 (例如 `std::cin`) 的类型是 `std::istream`, 输出流 (例如 `std::cout`) 的类型是 `std::ostream`。

这两个运算符只能是非成员: 你无法给 `std::istream` 和 `std::ostream` 添加成员。

- 若一个运算符是成员, 它只可能是最左侧运算对象的成员。

`std::istream` 和 `std::ostream` 都不能拷贝, 必须按引用传递, 而且不能是常量引用。

```
std::istream &operator>>(std::istream &, Rational &r);
std::ostream &operator<<(std::ostream &, const Rational &r);
```

- 为了支持连续的输入输出 `std::cin >> a >> b >> c`, 这两个运算符需要把左侧运算对象返回出来。

23/39

## 输入、输出运算符

输入运算符需要考虑输入错误的情况。

```
struct Vec3 {
 double x_, y_, z_;
 double l2_norm_;
};
std::istream &operator>>(std::istream &is, Vec3 &v) {
 is >> v.x_ >> v.y_ >> v.z_;
 if (!is) // 如果输入发生错误, 要将对象置于有效的状态。
 v.x_ = v.y_ = v.z_ = 0;
 v.l2_norm_ = std::sqrt(v.x_ * v.x_ + v.y_ * v.y_ + v.z_ * v.z_);
 return is;
}
```

24/39

## 输入、输出运算符

不能直接将 `is`、`os` 替换为 `std::cin` 和 `std::cout`: 除它们之外还有其它的流对象

```
std::ifstream file("infile.txt");
int x, y, z;
file >> x >> y >> z;
```

由于 `std::ifstream` 继承自 `std::istream`, 它可以被 `std::istream &` 绑定

25/39

## 避免重载运算符的滥用

函数的名称可以透露很多信息, 而运算符不行:

- `a * b` 和 `dot_product(a, b)`
- `a < b` 和 `compare_by_id(a, b)`
- `a + b` 和 `concat(a, b)`

避免滥用重载运算符: 除非这个运算符有唯一明确合理的定义, 否则不要定义它。

- 为什么 `std::string` 可以用 `+` 连接, 而 `std::vector` 不行?
- MATLAB 的 `a * b` 和 `a .* b` 的区别

`operator+` 应该加而不是减, `operator<` 应该是“小于”而非“大于”。

26/39

## 函数调用运算符 `operator()`

```
struct Adder {
 int operator()(int a, int b) const {
 return a + b;
 }
};
std::cout << Adder{}(2, 3) << std::endl; // 5
```

- `Adder{}` 创建了一个 `Adder` 类型的对象。
  - 也可以写作 `Adder()`。
- `Adder{}(2, 3)` 相当于 `Adder{}.operator()(2, 3)`

一般地, `f(arg1, arg2, ...)` 被视为 `f.operator()(arg1, arg2, ...)`。

**\* 分清这里的各个括号的含义!** 要理解每个括号所发挥的功能, 不要瞎猜。

27/39

## 类型转换运算符

除了接受单个参数的构造函数, 我们还有另一种方式自定义类型转换:

```
struct Rational {
 int n, d;
 operator double() const { // 定义了从 Rational 到 double 的类型转换
 return static_cast<double>(n) / d;
 }
};

Rational r{3, 2};
double d = r; // 1.5
```

28/39

## 类型转换运算符

```
struct Rational {
 int n, d;
 operator double() const {
 return static_cast<double>(n) / d;
 }
};
Rational r{3, 2};
double d = r; // 1.5
```

- 函数名是 `operator Type`
- 不接受参数, 不写返回值类型 (因为返回值类型就是 `Type`)
- 通常是 `const`: 类型转换一般不应改变这个对象本身。

29/39

## 类型转换运算符

`std::istream` 对象可以放在条件部分, 来判断这个输入流正不正常:

```
std::cin >> a >> b;
if (!std::cin)
 handle_input_failure();
```

所以标准库定义了 `std::istream` 向 `bool` 的类型转换, 对吗?

```
class istream {
 bool fail, bad;
public:
 operator bool() const {
 return !fail && !bad;
 }
};
```

30/39

## 类型转换运算符

标准库定义了 `std::istream` 向 `bool` 的类型转换, 对吗?

```
class istream {
 bool fail, bad;
public:
 operator bool() const { return !fail && !bad; }
};
istream cin;
```

下面的代码居然也乐呵呵地编译了! 为什么?

```
int ival;
cin << ival; // 写反了!
```

31/39

类型转换运算符

标准库定义了 `std::istream` 向 `bool` 的类型转换，对吗？

```
class istream {
 bool fail, bad;
public:
 operator bool() const { return !fail && !bad; }
};
istream cin;
```

下面的代码居然也乐呵呵地编译了！为什么？

```
int ival;
cin << ival; // cin 隐式转换成 bool 类型，又提升成 int
// 这个 << 其实是作用于两个 int 的移位运算符！
```

32/39

类型转换运算符

为了避免 `std::cin << ival` 通过编译，在 C++11 以前标准库定义的是向 `void *` 的类型转换：在输入流正常的时候返回一个非空指针，不正常的时候返回空指针。

(since C++11): `explicit` 类型转换运算符

```
struct Rational {
 int n, d;
 explicit operator double() const {
 return static_cast<double>(n) / d;
 }
};
Rational r{3, 2};
double d = r; // 错误：隐式转换
double d2 = static_cast<double>(r); // 正确
```

33/39

类型转换运算符

为了避免 `std::cin << ival` 通过编译，在 C++11 以前标准库定义的是向 `void *` 的类型转换：在输入流正常的时候返回一个非空指针，不正常的时候返回空指针。

(since C++11): `explicit` 类型转换运算符

```
class istream {
 bool fail, bad;
public:
 explicit operator bool() const { return !fail && !bad; }
};
istream cin;
```

从 `istream` 向 `bool` 的类型转换必须显式发生。

那 `if (cin)` 还能用吗？

34/39

Contextual conversions

下列情况中，`e` 向 `bool` 的类型转换允许使用 `explicit` 类型转换运算符：

- `if (e)`, `while (e)`, `for (xxx; e; xxx)`
- `!e`, `e && e`, `e || e`, `e ? a : b`
- (你们没学过的) `static_assert(e)`, `noexcept(e)`, `explicit(e)` (since C++20)

因此 `if (cin)` 也就没问题了。

35/39

避免类型转换运算符的滥用

```
struct Rational {
 int n, d;
 operator double() const {
 return static_cast<double>(n) / d;
 }
 operator std::string() const {
 return std::to_string(n) + '/' + std::to_string(d);
 }
};

Rational r{3, 2};
std::cout << r << std::endl; // 1.5 还是 3/2 ?
```

36/39

避免类型转换运算符的滥用

更好的设计：定义成普通的函数

```
struct Rational {
 int n, d;
 auto to_double() const {
 return static_cast<double>(n) / d;
 }
 auto to_string() const {
 return std::to_string(n) + '/' + std::to_string(d);
 }
};
```

一般来说，只为某些特殊的行为定义类型转换，尤其是不要轻易定义向内置类型的类型转换！

37/39

类型转换引发的二义性

```
struct A {
 A(const B &);
};
struct B {
 operator A() const;
};

B b{};
A a = b; // 到底是调用了 `A::A(const B &)` 还是 `B::operator A() const`?
```

38/39

类型转换引发的二义性

当类型转换遇上重载决议，情况就彻底乱套了...

```
struct Rational {
 operator double() const;
};
struct X {
 X(const Rational &);
};
void foo(int);
void foo(X);
Rational r{3, 2};
foo(r); // 到底是先转成 double 然后转成 int 匹配 foo(int)
// 还是直接转成 X 匹配 foo(X)?
```

避免这样的情况发生！不要滥用类型转换运算符。

39/39

CS100 Recitation 12

GKxx

1/35

Contents

- STL 迭代器和算法
- 继承、多态

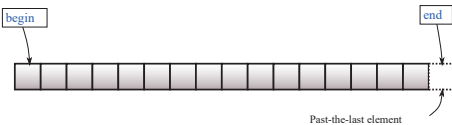
2/35

## STL 迭代器和算法

### 迭代器

一种用来访问容器中的元素的广义“指针”。

- 每个容器（以及 `std::string`）都有迭代器，其类型为 `Container::iterator`
- `c.begin()` 返回“首迭代器”：指向开头
- `c.end()` 返回“尾后迭代器”：指向尾后 (off-the-end) 位置
- 通常用一对迭代器表示一个左闭右开区间：`[begin, end)`



### 迭代器

并非所有容器都像 `std::vector` 或数组这样连续存储、支持下标访问。

- 链表，二叉树，哈希表.....

“广义指针”：扩展了指针的“访问元素”和“遍历序列”的功能，在不同的数据结构上 `++iter` 会采用不同的方式“前进到下一位置”。

### 迭代器的基本操作

所有迭代器类型都支持以下操作：

- `*it`：返回 `it` 所指向的元素的引用
- `it->mem`：等价于 `(*it).mem`
- `++it, it++`：将 `it` 前进一步。
- `it1 == it2, it1 != it2`：检查 `it1` 和 `it2` 是否指向相同的位置。

### 迭代器型别 (Iterator categories)

一个容器的迭代器型别归根到底取决于这个容器的性质。

- `ForwardIterator` 前向迭代器：支持基本操作。
- `BidirectionalIterator` 双向迭代器：在前向迭代器的基础上还支持 `--it` 和 `it--`。
- `RandomAccessIterator` 随机访问迭代器：在双向迭代器的基础上还支持算术运算和大小比较（类似于指针）
  - `it + n, n + it, it - n, it += n, it -= n, it[n], it1 - it2, <, <=, >, >=`
  - `std::string::iterator` 和 `std::vector<T>::iterator` 都是随机访问迭代器

数组 `T[N]` 当然可以被视为一种容器，其迭代器是 `T*`，也是随机访问迭代器。

实际上还有比 `ForwardIterator` 更低级的 `InputIterator`，以及特殊的 `OutputIterator`。

### 从迭代器范围初始化

`std::string` 以及大多数容器都支持从一个迭代器范围初始化：

```
std::vector<char> v = {'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i'};
std::vector v2(v.begin() + 2, v.end() - 3); // {'c', 'd', 'e', 'f'}
std::string s(v.begin(), v.end()); // "abcdefghi"
```

- CTAD 同样能发挥作用：`v2` 的类型是 `std::vector<char>`，这里的元素类型 `char` 是根据我们提供的迭代器推断出来的。

### 基于范围的 for 语句

基于范围的 `for` 语句本质上是用迭代器遍历：

```
std::vector<int> v = something();
for (auto x : v) {
 do_something_with(x);
}

std::vector<int> v = something();
{
 auto begin = v.begin();
 auto end = v.end();
 for (; begin != end; ++begin) {
 do_something_with(*begin);
 }
}
```

\* 如何让 `Dynarray` 也能用基于范围的 `for` 语句遍历？

### 标准库算法

通常接受一对迭代器表示范围 `[begin, end)`

```
std::sort(a, a + N); // a 可能是一个数组
std::copy(v.begin(), v.end(), a); // v 可能是一个 std::vector<...>
```

带 `_n` 后缀的函数使用 `[begin, begin + n)`

```
class Dynarray {
public:
 Dynarray(const Dynarray &other)
 : m_length(other.size()), m_storage(new int[other.size()]) {
 std::copy_n(other.m_storage, other.size(), m_storage);
 }
 Dynarra(std::size_t n, int x) : m_length(n), m_storage(new int[n]) {
 std::fill_n(m_storage, m_length, x);
 }
};
```

### 标准库算法

通常用迭代器表示位置

```
auto pos = std::find(v.begin(), v.end(), val); // pos 指向 val 第一次出现的位置
auto maxPos = std::max_element(v.begin(), v.end()); // maxPos 指向最大值所在的位置
```

部分算法对迭代器型别有要求，比如

- `std::sort` 接受 `RandomAccessIterator`，比较严格。
- `std::copy` 接受任何 `InputIterator` 作为前两个参数，要求第三个参数是 `OutputIterator`。

部分算法对元素类型有要求，比如

- `std::sort` 要求元素类型具有 `<` 运算符。
- `std::equal` 要求元素类型具有 `==` 运算符。

### 算法不修改容器

标准库算法绝不会改变容器的大小（除非传给它的迭代器是某些特殊的迭代器适配器）

例如，`std::copy(begin, end, dest)` 只是复制元素，但并不向容器插入元素！

```
std::vector<int> a = someValues();
std::vector<int> b(a.size());
std::vector<int> c{};
std::copy(a.begin(), a.end(), b.begin()); // 正确
std::copy(a.begin(), a.end(), c.begin()); // 未定义行为！
```

某种神秘的迭代器适配器：它的 `++` 和赋值操作十分不同寻常，会将 `*iter++ = x` 变为 `c.push_back(x)`

```
std::copy(a.begin(), a.end(), std::back_inserter(c)); // 这是可以的
```

一些常见算法

- 不修改序列: `count`, `find`, `find_end`, `find_first_of`, `search` 等
- 修改序列: `copy`, `fill`, `reverse`, `unique` 等
- 划分、排序、归并: `partition`, `sort`, `nth_element`, `merge` 等
- 二分查找: `lower_bound`, `upper_bound`, `binary_search`, `equal_range` 等
- 堆相关: `make_heap`, `push_heap`, `pop_heap`, `sort_heap` 等
- 大小比较: `min` / `max`, `min_element` / `max_element`, `equal`, `lexicographical_compare` 等
- 数值运算 (`<numeric>`): `accumulate`, `inner_product` 等

13/35

谓词 (Predicate)

许多算法接受一个谓词 (predicate), 即一个返回 `bool` 的**可调用对象**, 来定制操作。

- 需要比较元素的算法通常默认采用 `<` 进行比较, 但也提供接受一个二元谓词 `cmp` 的版本, 用 `cmp(a, b)` 代替 `a < b`。
  - `std::sort(begin, end, cmp)`
  - `std::max_element(begin, end, cmp)`
- 带有 `_if` 后缀的函数接受一个一元谓词 `cond`, 只关心那些 `cond(element)` 为真的元素 `element`。
  - `std::find_if(b, e, [](int x) { return x % 2 == 0; })` 找第一个偶数
  - `std::copy_if(b, e, d, [](int x) { return x % 2 == 0; })` 只拷贝偶数

14/35

可调用对象

C++ 中的可调用对象有函数、函数指针、类型为重载了调用运算符 `operator()` 的类类型的对象。

Lambda 本质上是让编译器帮你合成一个重载了 `operator()` 的类型并创建一个该类型的对象。

```
auto cmp =
 [](const std::string &a,
 const std::string &b) {
 return a.size() < b.size();
 };

struct LambdaType_cmp {
 bool operator()(const std::string &a,
 const std::string &b) {
 return a.size() < b.size();
 }
};
LambdaType_cmp cmp;
```

15/35

继承 (Inheritance)、多态 (Polymorphism)

继承

- 一个子类对象里一定有一个完整的父类对象\*
- 禁止“坑爹”: 继承不能破坏父类的封装性

17/35

继承

- 一个子类对象里一定有一个完整的父类对象\*
  - 父类的**所有成员** (除了构造函数和析构函数) 都被继承下来, 无论能否访问
  - 子类的构造函数必然先调用父类的构造函数来初始化父类的部分, 然后再初始化自己的成员
  - 子类的析构函数在析构了自己的成员之后, 必然调用父类的析构函数
- 禁止“坑爹”: 继承不能破坏父类的封装性
  - 子类不可能改变继承自父类成员的访问限制级别
  - 子类不能随意初始化父类成员, 必须经过父类的构造函数
    - 先默认初始化后赋值是可以的

\* 除非父类是空的, 这时编译器会做“空基类优化 (Empty Base Optimization, EBO)”。

18/35

子类的构造函数

必然先调用父类的构造函数来初始化父类的部分, 然后再初始化自己的成员

```
class DiscountedItem : public Item {
public:
 DiscountedItem(const std::string &name, double price,
 int minQ, double discount)
 : Item(name, price), m_minQuantity(minQ), m_discount(discount) {}
};
```

可以在初始值列表里调用父类的构造函数

- 如果没有调用?

19/35

子类的构造函数

必然先调用父类的构造函数来初始化父类的部分, 然后再初始化自己的成员

```
class DiscountedItem : public Item {
public:
 DiscountedItem(const std::string &name, double price,
 int minQ, double discount)
 : Item(name, price), m_minQuantity(minQ), m_discount(discount) {}
};
```

可以在初始值列表里调用父类的构造函数

- 如果没有调用, 则自动调用父类的默认构造函数
  - 如果父类不存在默认构造函数, 则报错。

合成的默认构造函数的行为?

20/35

子类的构造函数

必然先调用父类的构造函数来初始化父类的部分, 然后再初始化自己的成员

```
class DiscountedItem : public Item {
public:
 DiscountedItem(const std::string &name, double price,
 int minQ, double discount)
 : Item(name, price), m_minQuantity(minQ), m_discount(discount) {}
};
```

可以在初始值列表里调用父类的构造函数

- 如果没有调用, 则自动调用父类的默认构造函数
  - 如果父类不存在默认构造函数, 则报错。

合成的默认构造函数: 先调用父类的默认构造函数, 再 .....

21/35

子类的析构函数

析构函数在执行完函数体之后:

- 先按成员的声明顺序倒序销毁所有成员。对于含有 non-trivial destructor 的成员, 调用其 destructor。
- 然后调用父类的析构函数销毁父类的部分。

22/35

子类的拷贝控制

自定义：不要忘记拷贝/移动父类的部分

```
class Derived : public Base {
public:
 Derived(const Derived &other) : Base(other), /* ... */ { /* ... */ }
 Derived &operator=(const Derived &other) {
 Base::operator=(other);
 // ...
 return *this;
 }
};
```

合成的拷贝控制成员（不算析构）的行为？

纯虚函数

通过将一个函数声明为 `=0`，它就是一个**纯虚函数** (pure virtual function)。

一个类如果有某个成员函数是纯虚函数，它就是一个**抽象类**。

- 不能定义抽象类的对象，不能调用无定义的纯虚函数\*。

\* 事实上一个纯虚函数仍然可以拥有一份定义，阅读《Effective C++》条款 34（必读）

子类的拷贝控制

合成的拷贝控制成员（不算析构）：先父类，后子类自己的成员。

- 如果这个过程中调用了任何不存在/不可访问的函数，则合成为 implicitly deleted

纯虚函数

纯虚函数通常用来定义**接口**：这个函数在所有子类里都应该有一份自己的实现。

如果一个子类继承了某个纯虚函数而没有 override 它，这个成员函数就仍然是纯虚的，这个类仍然是抽象类，无法被实例化。

动态绑定

向上转型：

- 一个 `Base *` 可以指向一个 `Derived` 类型的对象
  - `Derived *` 可以向 `Base *` 类型转换
- 一个 `Base &` 可以绑定到一个 `Derived` 类型的对象
- 一个 `std::shared/unique_ptr<Base>` 可以指向一个 `Derived` 类型的对象
  - `std::shared/unique_ptr<Derived>` 可以向 `std::shared/unique_ptr<Base>` 类型转换

运行时类型识别 (RTTI)

`dynamic_cast` 可以做到“向下转型”：

- 它会在运行时检测这个转型是否能成功
- 如果不能成功，`dynamic_cast<T*>` 返回空指针，`dynamic_cast<T&>` 抛出 `std::bad_cast` 异常。
- 非常非常慢**，你几乎总是应该先考虑用一组虚函数来完成你想要做的事。

`typeid(x)` 可以获取表达式 `x`（忽略顶层 `const` 和引用后）的动态类型信息

- 通常用 `if (typeid(*ptr) == typeid(A))` 来判断这个动态类型是否是 `A`。

[https://quick-bench.com/q/E0LS3gJgAHIQK0Em\\_6XzkRzEjnE](https://quick-bench.com/q/E0LS3gJgAHIQK0Em_6XzkRzEjnE)

根据经验，如果你需要获取某个对象的动态类型，通常意味着设计上的缺陷，你应当修改设计而不是硬着头皮做 RTTI。

虚函数

继承父类的某个函数 `foo` 时，我们可能希望在子类提供一个新版本（override）。

我们希望在 一个 `Base *`，`Base &` 或 `std::shared/unique_ptr<Base>` 上调用 `foo` 时，可以根据**动态类型**来选择正确的版本，而不是根据静态类型调用 `Base::foo`。

在子类里 override 一个虚函数时，函数名、参数列表、`const` ness 必须和父类的那个函数**完全相同**。

- 返回值类型必须**完全相同**或者随类型协变 (covariant)。

加上 `override` 关键字：让编译器帮你检查它是否真的构成了 override

设计

`public` 继承建模出“is-a”关系：A discounted item is an item.

但有些时候英语上的“is-a”具有欺骗性：

- Birds can fly. A penguin is a bird.
- A square is a rectangle. 但矩形的长宽可以随意更改，而正方形不可以。

阅读《Effective C++》条款 32（必读）。

虚函数

除了 override，不要以其它任何方式定义和父类中某个成员同名的成员。

- 阅读以下章节，你会看到违反这条规则带来的后果。
  - 《Effective C++》条款 33：避免遮掩继承而来的名称
  - 《Effective C++》条款 36：绝不重新定义继承而来的 non-`virtual` 函数
  - 《Effective C++》条款 37：绝不重新定义继承而来的缺省参数值

继承的访问权限

这个 `public` 是什么意思？

```
class DiscountedItem : public Item {};
```

继承的访问权限

```
class DiscountedItem : public Item {};
```

继承的访问权限：这个继承关系（“父子关系”）是否对外公开。

如果采用 `private` 继承，则在外人眼里他们不是父子，任何依赖于这一父子关系的行为都将失败（有哪些？）。

IOStream 库

注：以下所有来自标准库的名字可能会省去 `std::`，但不代表我在代码里也会省去。

继承的访问权限

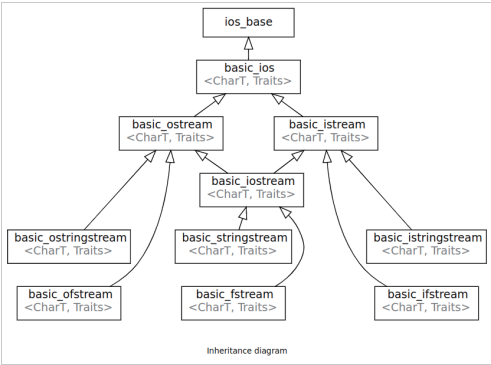
```
class DiscountedItem : public Item {};
```

继承的访问权限：这个继承关系（“父子关系”）是否对外公开。

如果采用 `private` 继承，则在外人眼里他们不是父子，任何依赖于这一父子关系的行为都将失败。

- 访问继承而来的成员（本质上也是向上转型）
- 向上转型（包括动态绑定等等）、向下转型

`private` 继承：建模“is-implemented-in-terms-of”。阅读《Effective C++》条款 38、39（选读）。



继承的访问权限

```
struct A : B {}; // public inheritance
class C : B {}; // private inheritance
```

`struct` 和 `class` 仅有两个区别：

- 默认的成员访问权限是 `public` / `private`。
- 默认的继承访问权限是 `public` / `private`。

IOStream

`ifstream`、`ofstream`：文件输入流，文件输出流，定义在 `<fstream>` 里

一般的文件流（默认情况下既可输入又可输出）：`fstream`

从文件读入：

```
std::ifstream ifs("path/to/myfile.ext");
int ival; std::string s;
ifs >> ival >> s;
```

向文件写入：

```
std::ofstream ofs("path/to/myfile.ext");
ofs << "hello world" << std::endl;
```

fstream

打开文件的模式：`app`、`binary`、`in`、`out`、`trunc`、`ate`、(since C++23) `noreplace`

[https://en.cppreference.com/w/cpp/io/basic\\_ifstream#Member\\_types\\_and\\_constants](https://en.cppreference.com/w/cpp/io/basic_ifstream#Member_types_and_constants)

通过 `|` 可以将多个模式并起来，例如 `std::ios::out | std::ios::binary`。

默认情况下：`ifstream` 使用 `in`，`ofstream` 使用 `out`，`fstream` 使用 `in | out`。

- 假如你传入的打开模式是 `mode`，`ifstream` 会以 `mode | in` 打开，`ofstream` 会以 `mode | out` 打开。
- `out` 会清空这个文件原来的内容，而 `app` 是将输出追加到原来的内容的后面。

```
std::ofstream file("myfile.txt", std::ios::app);
file << "hello world" << std::endl;
```

fstream

可以在构造的时候打开文件，并同时指定打开模式；也可以在稍后用 `open` 打开一个文件，或用 `close` 关闭打开的文件：

```
std::ifstream file("myfile.txt", std::ios::binary);
if (!file.is_open()) {
 // myfile.txt is not found or cannot be opened.
 // handle this error.
 file.open("myfile.txt", std::ios::binary);
}
// do something with the file
file.close();
file.open("another_file.txt");
// ...
```

`file` 的析构函数会关闭打开的文件。

CS100 Recitation 13

GKxx

Contents

- IOStream
- STL
  - 容器
  - 迭代器
  - 迭代器适配器

stringstream

将一个 `std::string` 作为输入内容或输出结果

`istringstream`, `ostringstream`, `stringstream`: 定义在 `<sstream>` 里。

```
std::ostringstream oss;
oss << 42 << 3.14 << "hello world";
std::string str = oss.str(); // "423.14hello world"
```

STL 总结、补充

注：以下所有来自标准库的名字可能会省去 `std::`，但不代表我在代码里也会省去。

STL 概览

诞生于 1994

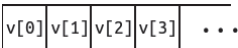
- 容器 containers: 顺序 (sequence) 容器, 关联 (associative) 容器
- 迭代器 iterators
- 算法 algorithms
- 适配器 adaptors: iterator adaptors, container adaptors
- 仿函数 function objects (functors)
- 空间分配器 allocators

容器

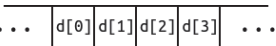
- 顺序容器 sequence containers
  - `vector`, `deque`, `list`, `array`, `forward_list`
- 关联容器 associative containers
  - `map`, `set`, `multimap`, `multiset`
  - `unordered_map`, `unordered_set`, `unordered_multimap`, `unordered_multiset`

顺序容器

- `vector<T>`: 可变长数组, 支持在末尾快速地添加、删除元素, 支持随机访问 (random access)



- `deque<T>`: 双端队列 (double ended queue), 支持在开头和末尾快速地添加、删除元素, 支持随机访问



顺序容器

- `list<T>`: 双向链表, 支持在任意位置快速地添加、删除元素, 支持双向遍历, 不支持随机访问



- `forward_list<T>`: 单向链表, 支持在任意位置快速地添加、删除元素, 仅支持单向遍历, 不支持随机访问



- `array<T, N>`: 内置数组 `T[N]` 的套壳, 提供和其它 STL 容器一致的接口 (包括迭代器等), 可以直接拷贝, 不会退化为 `T*`。
- 特别的: `string` 不是容器, 但非常像一个容器

统一的接口: 构造

- `Container c`: 一个空的容器
- `Container c(n, x)`: `n` 个 `x`
- `Container c(n)`: `n` 个元素, 每个元素都被值初始化
  - `string` 不支持这一操作, 为什么?
- `Container c(begin, end)`: 从迭代器范围 `[begin, end)` 中拷贝元素。

`array<T, N>` 只支持 1, 为什么?

统一的接口: 完整列表

相同的功能在所有容器上都具有相同的接口 (除了 `forward_list` 有一点点特殊)

- 拷贝构造、拷贝赋值、移动构造、移动赋值
- `c.size()`, `c.empty()`, `c.resize(n)`
- `c.capacity()`, `c.reserve(n)`, `c.shrink_to_fit()`
- `c.push_back(x)`, `c.emplace_back(args...)`, `c.pop_back()`
- `c.push_front(x)`, `c.emplace_front(args...)`, `c.pop_front()`
- `c.at(i)`, `c[i]`, `c.front()`, `c.back()`
- `c.insert(pos, ...)`, `c.emplace(pos, args...)`, `c.erase(...)`, `c.clear()`

统一的接口

记忆的关键:

- 理解每一种容器的底层数据结构, 自然就明白为何支持/不支持某种操作
  - 为何链表不支持下标访问? 为何 `vector` 不支持在开头添加元素?
- 记住这些接口的名字
- 如果一个操作应该被支持, 它就必然叫那个名字、是那个用法

string 和 vector 的“容量”

`string` 大概率和 `vector` 采用类似的增长方式, 分配的内存可能比当前存储的元素所占用的内存大。

- 当前所拥有的内存能放下多少个元素, 称为“容量” (capacity), 可以通过 `c.capacity()` 查询。
- `c.reserve(n)` 为至少 `n` 个元素预留内存。
  - 如果 `n <= c.capacity()`, 什么都不会发生。
  - 否则, 重新分配内存使得 `c` 能装得下至少 `n` 个元素。
  - 务必区分 `reserve` 和 `resize`。
- `c.shrink_to_fit()`: 请求 `c` 释放多余的容量 (可能重新分配一块更小的内存)
  - 这个函数在 `deque` 上也有。

## insert 和 erase

`c.insert(pos, ...)`, `c.emplace(pos, args...)`, 其中 `pos` 是一个迭代器。

在 `pos` 所指的位置之前添加元素。

- `insert` 有很多种写法, 可以 `c.insert(pos, x)`, `c.insert(pos, begin, end)`, `c.insert(pos, {a, b, c, ...})` 等等, 用到的时候再查。

`c.erase(...)` 也有很多种写法, 用到的时候再查。

18/41

## 特殊的 forward\_list

`forward_list` 的功能完全被 `list` 包含, 那为何我们还需要 `forward_list`?

19/41

## 特殊的 forward\_list

`forward_list` 的功能完全被 `list` 包含, 那为何我们还需要 `forward_list`?

为了省时间, 省空间。

- 单向链表的结点比双向链表的结点少存一个指针
- 维护单向链表上的链接关系也比维护双向链表少一些操作

因此, `forward_list` 采取最简的实现: 能省则省

- 它不能 `push_back` / `pop_back`。
  - 如果需要, 你可以自己保存指向末尾元素的迭代器, 然后用 `insert_after`
- 它甚至不支持 `size()`。如果需要, 你可以自己用一个变量记录。

20/41

## 特殊的 forward\_list

`insert`, `emplace` 和 `erase` 变成了 `insert_after`, `emplace_after` 和 `erase_after`

- 单向链表上, 操作“下一个元素”比操作“当前元素”或“前一个元素”更方便。

21/41

## 越界检查

`c.at(i)` 在越界时抛出 `std::out_of_range` 异常

`c[i]`, `c.front()`, `c.back()`, `c.pop_back()`, `c.pop_front()` 统统不检查越界, 一切为了效率。

但是我们在自己设计自己的容器时, 不一定要采用标准库的这种方式。

- 也许下面这种设计更合理?

```
auto &operator[](size_type n) {
 assert(n < size());
 return data[n];
}
```

- `assert` 在 Debug 模式下生效, 而在 Release 模式下 (定义了 `NDEBUG` 宏) 无效, 不会影响 Release 模式下的效率。

22/41

## 选择正确的容器

顺序容器:

- 能维持元素的先后顺序
- 某些情况下的插入、删除、查找可能较慢

关联容器 (不带 `unordered` 的):

- 元素总是有序的, 默认是升序 (因为底层数据结构通常是红黑树)
- 支持  $O(\log n)$  地插入、删除、查找元素

无序关联容器 (`unordered`):

- 元素是无序的 (因为底层数据结构是哈希表)
- 支持平均情况下  $O(1)$  地插入、删除、查找元素

23/41

## 迭代器

### 复习: 假如没有迭代器...

不同的容器, 根据底层数据结构不同, 遍历方式自然也不同:

```
for (std::size_t i = 0; i != a.size(); ++i)
 do_something(a[i]);
// 可能的方式: 通过指向结点的“句柄” (指针) 遍历一个链表
for (node_handle node = l.first_node(); node; node = node.next())
 do_something(node.value())
```

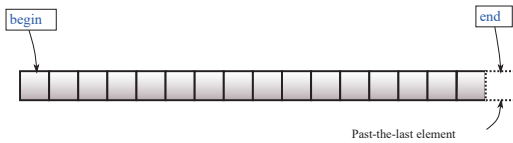
如果是更复杂的容器呢, 比如基于哈希表/红黑树实现的关联容器?

25/41

### 复习: 使用统一的方式访问元素、遍历容器

所有容器都有其对应的迭代器类型 `Container::iterator`, 例如 `std::string::iterator`, `std::vector<int>::iterator`。

所有容器都支持 `c.begin()`, `c.end()`, 分别返回指向首元素和指向尾后位置的迭代器。



\* 我们总是使用左闭右开区间 `[begin, end)` 表示一个“迭代器范围”

26/41

### 复习: 使用统一的方式访问元素、遍历容器

- `it1 != it2` 比较两个迭代器是否相等 (指向相同位置)
- `++it` 让 `it` 指向下一个位置。
- `*it` 获取 `it` 指向的元素的引用。

```
for (auto it = c.begin(); it != c.end(); ++it)
 do_something(*it);
```

基于范围的 `for` 语句: 完全等价于上面的使用迭代器的遍历

```
for (auto &x : c)
 do_something(x);
```

27/41

const\_iterator

带有“底层 const”的迭代器。

- Container::const\_iterator
- c.cbegin(), c.cend()
- 在一个 const 对象上, c.begin() 和 c.end() 也返回 const\_iterator。

对 const\_iterator 解引用会得到 const T & 而非 T &, 无法通过它修改元素的值。

begin, end, cbegin, cend

再次出现了 const 和 non-const 的重载。

```
class MyContainer {
public:
 using iterator = /* unspecified */;
 using const_iterator = /* unspecified */;
 iterator begin();
 const_iterator begin() const;
 iterator end();
 const_iterator end() const;
 const_iterator cbegin() const;
 const_iterator cend() const;
};
```

复习：迭代器型别 (iterator category)

- ForwardIterator: 支持 \*it, it->mem, ++it, it++, it1 == it2, it1 != it2
- BidirectionalIterator: 在 ForwardIterator 的基础上, 支持 --it, it--。
- RandomAccessIterator: 在 BidirectionalIterator 的基础上, 支持 it[n], it + n, it += n, n + it, it - n, it -= n, it1 - it2, <, <=, >, >=。

vector, string, array, deque 的迭代器是 RandomAccessIterator; list 的迭代器是 BidirectionalIterator; forward\_list 的迭代器是 ForwardIterator。

复习：迭代器型别 (iterator category)

还有两种迭代器型别: InputIterator 和 OutputIterator。

- InputIterator 表示可以通过这个迭代器获得元素 (不要求能修改它所指向的元素)
- OutputIterator 表示可以通过这个迭代器写入元素 (不要求能获得它所指向的元素)
- A ForwardIterator is an InputIterator.

稍后我们会看到一些例子。

迭代器的辅助函数

- std::advance(iter, n) 将 iter 前进 n 步。n 也可以是负的。
- std::distance(iter1, iter2) 返回 iter 和 iter2 之间的距离: 相距几个元素。
- std::next(iter) 返回 iter 的“下一个位置”, std::prev(iter) 返回 iter 的“上一个位置”, 它们都不会改变 iter 本身。
  - 用来代替手动的 auto tmp = iter; ++tmp;。

以上函数对于各类迭代器型别都有支持, 并且会针对不同的型别提供不同的实现。

- 不管 iter1 和 iter2 是什么型别, 你都可以用 std::distance(iter1, iter2) 获取它们的距离, 不用担心 iter1 - iter2 这个表达式能不能编译之类的问题。

迭代器适配器 (iterator adaptors)

迭代器适配器 (iterator adaptors)

一种用起来像迭代器的东西

- 可能是根据某个迭代器和/或一些别的东西“变出来”的
- 用起来像迭代器, 但多多少少有点区别, 比如
  - 反向迭代: ++ 实际上是 --
  - 帮助移动元素: \*iter 返回右值引用
  - 看似迭代, 实则插入元素: \*iter++ = x 会被“变”成 c.push\_back(x)
  - 将某些特殊的过程也抽象为“迭代”: x = \*iter++ 被“变”成 std::cin >> x
  - .....

反向迭代器 reverse\_iterator

每个容器都有自己的反向迭代器: Container::reverse\_iterator 和 Container::const\_reverse\_iterator, 通常被这样定义:

```
using reverse_iterator = std::reverse_iterator<iterator>;
using const_reverse_iterator = std::reverse_iterator<const_iterator>;
```

- c.rbegin(), c.rend(), c.crbegin(), c.crend()
- ++ 和 -- 在反向迭代器上都是反的。

```
std::vector v{1, 2, 3, 4, 5};
for (auto rit = v.rbegin(); rit != v.rend(); ++rit)
 std::cout << *rit << ' ';
```

输出: 5 4 3 2 1

移动迭代器 move\_iterator

- std::make\_move\_iterator(iter) 从一个普通的迭代器 iter 变出一个“移动迭代器”
- \*mit 会得到右值引用而非左值引用, 从而元素更可能被移动而非被拷贝。

```
std::vector<std::string> words = someValues();
std::vector<std::string> v(words.size());
std::copy(std::make_move_iterator(words.begin()),
 std::make_move_iterator(words.end()), v.begin());
```

words 中的每个 string 被移动进了 v, 而不是拷贝。

从迭代器向迭代器的映射

对于一个给定的迭代器类型 Iter, std::reverse\_iterator<Iter> 和 std::move\_iterator<Iter> 都是一个新的迭代器类型:

- 它们是 Iter 的“wrapper”: 在内部保存一个 Iter 类型的原始迭代器, 可以通过 rit.base() 访问。
- 它们基于 Iter 的接口定义自己的接口: ++rit 会调用 --it, \*mit 会返回 std::move(\*it)

C++23 还有个 std::basic\_const\_iterator<Iter>, 用类似的方式定义了 Iter 的带有底层 const 的版本。

你当然可以用类似的方式实现自己的特殊迭代器。

插入迭代器

```
insert_iterator, front_insert_iterator, back_insert_iterator
```

典型的 OutputIterator:

- 只可向 \*iter 写入元素, 不能从 \*iter 读取元素
- 它们会调用容器的 insert, push\_front 或 push\_back, 将“写入”的元素插入容器

```
std::vector<int> numbers = someValues();
std::vector<int> v;
std::copy(numbers.begin(), numbers.end(), v.begin()); // 错误!
std::copy(numbers.begin(), numbers.end(), std::back_inserter(v)); // 正确
```

std::back\_inserter(c) 生成一个 std::back\_insert\_iterator<Container>, 它内部保存一个 Container &r = c, 会不断调用 r.push\_back(x) 将向它“写入”的元素添加进 c。

Contents

- 模板 (总结、补充)
  - 基础知识
  - 可变参数模板
  - 一些由模板编译引发的问题
  - 实现一个 std::distance
  - 认识模板元编程
- 总结

流迭代器

读入一串数存进一个 vector<int>:

```
std::vector<int> v(std::istream_iterator<int>(std::cin),
std::istream_iterator<int>{});
```

利用 std::copy 将 v 中的元素输出, 并且每个后面跟一个 ", ":

```
std::copy(v.begin(), v.end(), std::ostream_iterator<int>(std::cout, ", "));
```

- istream\_iterator 是一种 InputIterator, 它不断从输入流中获取元素
- ostream\_iterator 是一种 OutputIterator, 它不断将向它“写入”的元素写进输出流

基础知识

迭代器型别

InputIterator 和 ForwardIterator 都要求支持 ++it, it++, \*it, it->mem, ==, !=。

这两类迭代器的区别究竟是什么?

模板

有模板函数、类模板、变量模板、别名模板。

```
template <typename T>
const T &max(const T &a, const T &b) {
 return a < b ? b : a;
}

template <typename T> class vector { /* ... */ };

template <typename T>
inline constexpr bool is_abstract_v = /* ... */;

template <typename T>
using MyPair = std::pair<T, T>; // MyPair<T> is an alias of std::pair<T, T>,
 // for any type T.
```

迭代器型别

InputIterator 和 ForwardIterator 都要求支持 ++it, it++, \*it, it->mem, ==, !=。

这两类迭代器的区别究竟是什么? —— ForwardIterator 提供 multi-pass guarantee

```
auto original = iter; // 对当前的 iter 做个拷贝
auto value = *iter; // 现在 iter 指向的元素是 value
++iter;
assert(*original == value);
```

对于一个 ForwardIterator 来说, original 指向了 iter 在递增之前指向的位置, 那个位置上的值始终是 value。

InputIterator 不这么认为, 它只保证能“input”: ++iter 就意味着我们打算获取“下一个值”了, 先前的值也就无法再被获取了。(考虑“输入”的过程)

模板不是 ... , 而是 ...

模板函数不是真正的函数, 它只是提供给编译器的一份指导方案, 编译器将根据该“方案”根据需要合成真正的函数。

当编译器看到一个模板函数时:

- 它会对这个函数做基本的语法检查, 缺少分号、自己发明运算符等错误会被发现。
- 它会对这个函数中与模板参数无关的部分做一些细致的语法、语义分析。
- 除此之外, 什么都不做, 也不会生成任何的代码。

当这个模板函数被实例化时 (伴随着相应的模板参数被提供):

- 编译器将模板实参代入模板函数中, 得到一个真正的函数。
- 编译器对这个真正的函数做彻底的检查和分析, 并生成相应的代码。

<https://godbolt.org/z/b7TP5v8ca>

CS100 Recitation 14

GKxx

模板不是 ... , 而是 ...

模板函数不是真正的函数, 它只是提供给编译器的一份指导方案, 编译器将根据该“方案”根据需要合成真正的函数。

类模板、变量模板、别名模板也是类似。

对于类模板, 未被用到的成员函数也不会被实例化, 哪怕这个类被实例化了。

```
template <typename T> struct A {
 T x;
 void foo() { ++x; } // 不是什么类型都支持 ++ 的
};
A<std::string> a; // T = std::string 显然不支持 ++ ,
 // 但是没关系, 我们并未调用 a.foo()
```

模板实例化 (instantiation)

比较常见的是**隐式实例化**：当某个实例被需要时，它会被自动实例化出来。

- 比如，模板函数的某个实例被调用时，创建了类模板的某个实例的对象时。

```
template <typename T> void f(T x);
f(0); // 隐式实例化 f<int>(int)
```

也可以**显式实例化**：可能是我预判到这个实例将被用到，所以先要求编译器把它实例化出来。

```
template void f<double>(double); // 显式实例化 f<double>(double)
template void f<char>(char); // 显式实例化 f<char>(char) , 模板参数被推断
template void f(int); // 显式实例化 f<int>(int) , 模板参数被推断
```

模板是编译时的游戏

模板实例化完全发生在**编译时**。左边这个 C++ 函数与右边的 Python （动态类型）函数有**根本的区别**。

```
template <typename T>
T add(T a, T b) {
 return a + b;
}
```

```
def add(a, b):
 return a + b
```

所有模板参数（在尖括号 <> 之间的内容）必须在**编译时已知**。

可变参数模板

可变参数模板

```
template <typename First, typename... Rest> // Rest 是一个模板参数包
void read(First &first, Rest &...rest) { // rest 是一个函数参数包
 std::cin >> first;
 if (/* rest 不是空的 */)
 read(rest...); // 包展开
}
int i; double d; std::string s;
read(i); // First = int, Rest 和 rest 都是空的
```

read(i, d, s) 实例化出以下函数：

- void read(int &first, double &rest\_1, std::string &rest\_2)
- read(rest...) 又会调用 read(rest\_1, rest\_2) , 导致 void read(double &first, std::string &rest\_1) 被实例化，而它的 read(rest\_1) 会导致 void read(std::string &first) 被实例化。

sizeof...(pack)

一个参数包里有几个参数？用 sizeof... 运算符。这个运算符在编译时求值。

**\* 务必区分声明参数包时的 ...、包展开时的 ... 和 sizeof... 中的 ... ！**

```
template <typename First, typename... Rest> // Rest 是一个模板参数包
void read(First &first, Rest &...rest) { // rest 是一个函数参数包
 std::cin >> first;
 if (sizeof...(Rest) > 0)
 read(rest...); // 包展开
}
```

报了个编译错误？它说我试图调用 read()

```
a.cpp: In instantiation of 'void read(First&, Rest& ...) [with First = int; Rest = {}]':
a.cpp:12:7: required from here
a.cpp:7:9: error: no matching function for call to 'read()'
 7 | read(rest...); // 包展开
```

if constexpr

不妨试着写出当 Rest = {} 时的函数长什么样：

```
template <typename First>
void read(First &first) {
 std::cin >> first;
 if (false) // sizeof...(Rest) == 0
 read(); // Oops! read 接受至少一个参数!
}
```

问题出在这个 **if 是运行时的控制流**，哪怕这个条件 100% 是 false，这个部分也必须能编译才行！

if constexpr

if constexpr (condition)：编译时的 if (since C++17)

- condition 必须能在编译时求值
- 只有在 condition 为 true 时，statements 才会被编译。

```
if constexpr (condition)
 statements
```

- 根据 condition 的值来决定编译 statementsTrue 还是 statementsFalse。

```
if constexpr (condition)
 statementsTrue
else
 statementsFalse
```

if constexpr

```
template <typename First, typename... Rest>
void read(First &first, Rest &...rest) {
 std::cin >> first;
 if constexpr (sizeof...(Rest) > 0)
 read(rest...);
}
```

如果没有 if constexpr，我们就需要通过重载来完成：（这里的重载决议不用搞清楚）

```
template <typename T> // 为一个参数的情况单独定义
void read(T &x) { std::cin >> x; }
template <typename First, typename... Rest>
void read(First &first, Rest &...rest) {
 read(first); read(rest...);
}
```

一些由模板编译引发的问题

实现 Dynarray<T> 的 operator<

不就是给各个地方都加上 <T> 么，我会！

```
template <typename T>
class Dynarray {
 friend bool operator<(const Dynarray<T> &, const Dynarray<T> &);
};
template <typename T> // 别忘了模板声明
bool operator<(const Dynarray<T> &, const Dynarray<T> &) {
 // 实现这个函数...
}

Dynarray<int> a, b;
if (a < b) // ld 报错 undefined reference to operator< ???
 // ...
```

## 非模板的情形

```
class Dynarray {
 friend bool operator<(const Dynarray &, const Dynarray &); // (1)
};
bool operator<(const Dynarray &, const Dynarray &) { // (2)
 // 实现这个函数...
}
```

毫无疑问，(1) 处的声明和 (2) 处的定义是同一个函数。

17/48

## 模板情形

```
template <typename T> class Dynarray {
 friend bool operator<(const Dynarray<T> &, const Dynarray<T> &); // (1)
};
template <typename T>
bool operator<(const Dynarray<T> &, const Dynarray<T> &) { /* ... */ } // (2)
```

- 当我们使用 `Dynarray<int>` 时，这个类被实例化出来。
- 这时它顺带声明了 (1) `friend bool operator<(const Dynarray<int> &, const Dynarray<int> &)`，这个函数**不是模板**，而 (2) 是一个函数模板，**编译器不认为 (1) 和 (2) 是同一个函数**。
- 表达式 `a < b` 对于 `operator<` 做重载决议时，既能找到 (1) 又能找到 (2)，但是其它条件相同的情况下**非模板优于模板**，所以它选择了 (1) 而不是 (2)。
- 接下来压力给到链接器：(1) 只有声明而没有定义，遂报错。

18/48

## 解决方案 1：不用 friend

```
template <typename T>
class Dynarray {
 // 不声明 operator< 为 friend
};
template <typename T>
bool operator<(const Dynarray<T> &, const Dynarray<T> &) {
 // 不访问 Dynarray<T> 的私有成员
}
```

如果真的可以不借助 `friend` 实现它（而且不产生额外的代价），这当然也是个办法。

这时 `a < b` 的 `operator<` 正对应了这个模板函数，能够正确编译和链接。

这里只有一个 `operator<`，不存在两个 `operator<` 争宠的情况。

19/48

## 解决方案 2：在声明 friend 的同时定义它

```
template <typename T>
class Dynarray {
 friend bool operator<(const Dynarray<T> &, const Dynarray<T> &) {
 // 直接在这里实现它
 }
};
```

这里也只有一个 `operator<`。

20/48

## 解决方案 3：告诉编译器“真相”

```
template <typename T> class Dynarray {
 friend bool operator<(const Dynarray<T> &, const Dynarray<T> &); // (1)
};
template <typename U> // 为了不引起混淆，这里用 U
bool operator<(const Dynarray<U> &, const Dynarray<U> &) { /* ... */ } // (2)
```

编译器认为：(1) 不是模板，(2) 是模板，(1) 和 (2) 不是同一个函数。

真相：(1) 和 (2) 应当是同一个函数。在 `T` 给定的情况下，(1) 其实是 (2) 的 `U = T` 情形的实例。

21/48

## 解决方案 3：告诉编译器“真相”

```
// 1. 在 `friend` 声明之前声明这个模板函数
// 为了声明 operator< 的参数，还得再为 class Dynarray 补充一个声明
template <typename T> class Dynarray;
template <typename T>
bool operator<(const Dynarray<T> &, const Dynarray<T> &);
template <typename T> class Dynarray {
 // 2. 声明 friend 时在函数名后面加上 <> (或 <T>)
 // 说明它是先前声明过的一个模板函数的一个实例
 friend bool operator< <>(const Dynarray<T> &, const Dynarray<T> &);
};
// 3. 正常给出 `operator<` 的定义
template <typename T>
bool operator<(const Dynarray<T> &, const Dynarray<T> &) {
 // ...
}
```

22/48

## 实现一个 `std::distance`

### `std::distance`

定义于 `<iterator>` 中。相关的函数还有 `std::advance`，`std::next` 等。

```
template <typename Iterator>
auto distance(Iterator first, Iterator last);
```

计算从 `first` 到 `last` 的“距离”：

- `Iterator` 至少得是 `InputIterator`。
- 对于 `RandomAccessIterators`，返回 `last - first`。
- 对于一般的 `InputIterator`，从 `first` 开始不断 `++`，直到碰到 `last` 为止。

24/48

## 返回值类型是什么？

通常情况下，一个迭代器应当具有一个类型别名成员 `difference_type`，表示两个迭代器的“距离”的类型。

这个类型通常是 `std::ptrdiff_t`（和指针相减的类型相同），但这**并不一定**。

为了写出最通用的 `distance`，我们应该使用这个 `difference_type`：

```
template <typename Iterator>
typename Iterator::difference_type distance(Iterator first, Iterator last);
```

开头的这个 `typename` 是啥？如果 `Iterator` 是个指针怎么办？一会儿再说...

25/48

## 如何知道迭代器的型别？

通常情况下，一个迭代器应当具有一个类型别名成员 `iterator_category`，它是以下五个类型之一的别名：

```
namespace std {
 struct input_iterator_tag {};
 struct output_iterator_tag {};
 struct forward_iterator_tag : input_iterator_tag {};
 struct bidirectional_iterator_tag : forward_iterator_tag {};
 struct random_access_iterator_tag : bidirectional_iterator_tag {};
}
```

26/48

## Tag dispatch

将两种不同的实现写在两个函数里，分别加上一个 tag 参数

```
template <typename Iterator>
auto distance_impl(Iterator first, Iterator last,
 std::random_access_iterator_tag); // (1)

template <typename Iterator>
auto distance_impl(Iterator first, Iterator last,
 std::input_iterator_tag); // (2)

template <typename Iterator>
auto distance(Iterator first, Iterator last) {
 using category = typename Iterator::iterator_category;
 // 传一个 category 类型的对象作为第三个参数
 // 如果 category 是 std::random_access_iterator_tag, 就会匹配 (1), 否则匹配 (2)
 return distance_impl(first, last, category{});
}
```

27/48

## Tag dispatch

```
template <typename Iterator>
auto distance_impl(Iterator first, Iterator last,
 std::random_access_iterator_tag) { // (1)
 return last - first;
}
template <typename Iterator>
auto distance_impl(Iterator first, Iterator last,
 std::input_iterator_tag); { // (2)
 typename Iterator::difference_type result = 0;
 while (first != last) { ++first; ++result; }
 return result;
}
template <typename Iterator>
auto distance(Iterator first, Iterator last) {
 using category = typename Iterator::iterator_category;
 return distance_impl(first, last, category{});
}
```

28/48

## 指针怎么办?

以上实现依赖于 `Iterator::difference_type` 和 `Iterator::iterator_category`，如果 `Iterator` 根本不是类类型怎么办？

当然可以直接为指针做一个重载：

```
template <typename T>
auto distance(T *first, T *last) {
 return last - first;
}
```

但事实上有很多函数都面临这个问题，全都多加一份重载也太麻烦了。

29/48

## Traits 技术

```
template <typename Iterator> // 一般情况: Iterator 是一个类类型
struct Traits {
 using difference_type = typename Iterator::difference_type;
 using iterator_category = typename Iterator::iterator_category;
};
template <typename T> // 为指针做特化
struct Traits<T*> {
 using difference_type = std::ptrdiff_t;
 using iterator_category = std::random_access_iterator_tag;
};
```

使用 `Traits<Iterator>::difference_type` 和 `Traits<Iterator>::iterator_category`，即可处理所有情况。

30/48

## iterator\_traits

上面的这个 `Traits` 正对应了标准库 `std::iterator_traits`。

```
namespace std {
 template <typename Iterator> // 一般情况: Iterator 是一个类类型
 struct iterator_traits {
 using value_type = typename Iterator::value_type;
 using pointer = typename Iterator::pointer;
 using reference = typename Iterator::reference;
 using difference_type = typename Iterator::difference_type;
 using iterator_category = typename Iterator::iterator_category;
 };
}
```

31/48

## iterator\_traits

上面的这个 `Traits` 正对应了标准库 `std::iterator_traits`。

```
namespace std {
 template <typename T> // 为指针做特化
 struct iterator_traits<T*> {
 using value_type = std::remove_cv_t<T>; // 这是啥?
 using pointer = T*;
 using reference = T&;
 using difference_type = std::ptrdiff_t;
 using iterator_category = std::random_access_iterator_tag;
 };
}
```

`std::remove_cv_t<T>`：是 `T` 去除可能的顶层 `const` 或 `volatile` 后的类型，定义于 `<type_traits>`。

32/48

## 用 if constexpr 实现

能不能直接这样写？

```
template <typename Iterator>
auto distance(Iterator first, Iterator last) {
 using category = typename std::iterator_traits<Iterator>::iterator_category;
 if constexpr (/* category == std::random_access_iterator_tag */)
 return last - first;
 else {
 typename std::iterator_traits<Iterator>::difference_type result = 0;
 while (first != last) {
 ++first; ++result;
 }
 return result;
 }
}
```

如何判断两个类型相同？

33/48

## 用 if constexpr 实现

`std::is_same_v<T, U>`：bool 类型的编译期常量，当 `T` 和 `U` 是同一个类型时为 `true`，否则为 `false`。定义于 `<type_traits>`。

```
template <typename Iterator>
auto distance(Iterator first, Iterator last) {
 using category = typename std::iterator_traits<Iterator>::iterator_category;
 if constexpr (std::is_same_v<category, std::random_access_iterator_tag>)
 return last - first;
 else {
 typename std::iterator_traits<Iterator>::difference_type result = 0;
 while (first != last) {
 ++first; ++result;
 }
 return result;
 }
}
```

34/48

## 认识模板元编程 (Template Metaprogramming)

### Hello world

```
template <unsigned N>
struct Factorial {
 static const auto result = N * Factorial<N - 1>::result;
};
template <>
struct Factorial<0u> {
 static const auto result = 1u;
};
int main() {
 const auto n = Factorial<10>::result; // 3628800
 int a[n]; // 正确。n 是一个编译时常量，可以用来开数组。
}
```

虽然在 modern C++ 它可以完全被 `constexpr` 函数替代，但是这仍是 TMP 的最经典的 hello world。

36/48

## std::tuple：一个编译期容器

可能的实现：

```
template <typename First>
class tuple<First>;
template <typename First, typename... Rest>
class tuple<First, Rest...> : public tuple<Rest...>;
// 例: tuple<A, B, C> 继承自 tuple<B, C>

std::tuple<int, double, int> t{0, 4.0, 42};
```

## std::ratio：编译期有理数类

标准库 `<chrono>` 利用 `std::ratio` 来表示各种时间单位，并且保证了量纲的正确性

- 例如，微秒和毫秒不能在数值上直接相加。

一个经典的例子：将七大基本物理单位对应于七个模板参数

```
template <int mass, int length, int time, int charge,
 int temperature, int intensity, int amount_of_substance>
struct quantity;
```

- 例如，力（牛顿）就是 `quantity<1, 1, -2, 0, 0, 0, 0>`，即  $\text{kg} \cdot \text{m} / \text{s}^2$ 。
- 正确定义 `quantity` 之间的运算，就可以在**编译时**杜绝量纲错误。

## 更疯狂的例子（不止局限于模板元编程）

C++ 的编译时计算能力非常强大 (turing-complete)。

在 `r14` 文件夹下有

- `merge_sort`：编译时归并排序
- `ctjson`：编译时 json parser（需要 C++20）

更疯狂的：

编译时正则表达式

编译时 raytracer

## 其它可能的应用

- 表达式模板 expression templates
- 序列化 serialization
- Embedded Domain Specific Language
- ...

事实上很多使用模板的程序都需要一些 TMP 技术，哪怕只是非常简单的 specialization 或 `SFINAE`。

## 总结

## 代码是你的作品

我们在本学期的几次作业中不断地传达这个思想（虽然没有明确地点出）。

- 尝试**读懂**整个代码项目中的所有部分，而不是只看自己要写的一个函数或类。
  - 理解程序的各个部分之间如何协作、有哪些隐式的约定。（记得 quiz1 的那个问题吗？）
- 对你自己写的代码负责：**保证你的代码的正确性应该是你自己的事。**
  - “未经测试的代码都是错的。”——蒋炎岩
- 代码不仅会被运行，**还会被阅读。**
  - 要写可读性高的代码。**可读性高的代码才更可能是正确的代码**，而不是“先用笨办法把代码搞对，再去考虑可读性的问题”。
  - 格式化。
  - 正确、规范地维护注释和文档。

## 工具很重要

工欲善其事，必先利其器。工具的使用也是编程的一部分。

- VSCode 的那些 json 文件都是什么意思？每次“一键编译”甚至“一键编译运行”的时候，到底发生了什么？“Problems”、“Output”等面板里都是些啥？
- 编译指令怎么写？各个选项都是什么含义？
- 为什么你的一键格式化和我的一键格式化效果不同？

为了舒适地写代码，花费一些时间在学习工具上是**完全值得的**，因为你的工具会陪伴你很多年。

“差生文具多”的观念并不可取。

## 编程语言是工具吗？

有人说，编程语言不过是工具，不需要花太多时间学习语言特性；也有人认为不仅如此。你怎么看？

- 语言特性**很重要**。就算你认为它只是工具，工具也很重要。
- 编程语言（PL）是一个很值得研究的领域，虽然在上科大搞的人很少。
- 语言特性的演化往往代表着“好的编程风格”的演化。

```
// C++98
for (std::vector<int>::reverse_iterator rit = v.rbegin();
 rit != v.rend(); ++rit) {}

// C++11
for (auto rit = v.rbegin(); rit != v.rend(); ++rit) {}
// C++20
for (auto x : v | std::views::reverse) {}
```

## 提问的智慧

<https://stackoverflow.com/help/how-to-ask>

- 搜索引擎/StackOverflow/文档/手册/教科书上能查到的问题不要问。
  - 再强调一下 StackOverflow（以及 StackExchange 的各个分站）。不要因为它是英语的“或者“加载缓慢”而拒绝它们，这些论坛里高手如云，回答的质量往往超乎想象。
- 你的编译器和编译指令是什么？系统是什么？某个库的版本是什么？
  - 必要时，完整地贴出相关信息。
  - 不要只会说“报错了”。究竟报了什么**错**？尝试理解报错信息。
- 正确使用 AI。
  - 谨慎判断 AI 所给出的回答。
  - 不要让 AI 代替你本应经历的锻炼过程。

## 这些东西几乎没有课程教你，但往往默认你会

- Git
- Python 等脚本语言
- Markdown
  - 如果没有 Marp，很难想象这 28 个 Lecture 和 14 次习题课的 slides 要怎样在无比急迫的情况下做出来。
- $LATEX$
- 学术写作

学习终究是自己的事情。

保持好的心态

不要被一些你改变不了的事情牵扯注意力。想清楚自己究竟应该学会什么。

•